Inteligență Artificială

Bogdan Alexe
bogdan.alexe@fmi.unibuc.ro

Radu Ionescu
raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

Recapitulare – cursul trecut

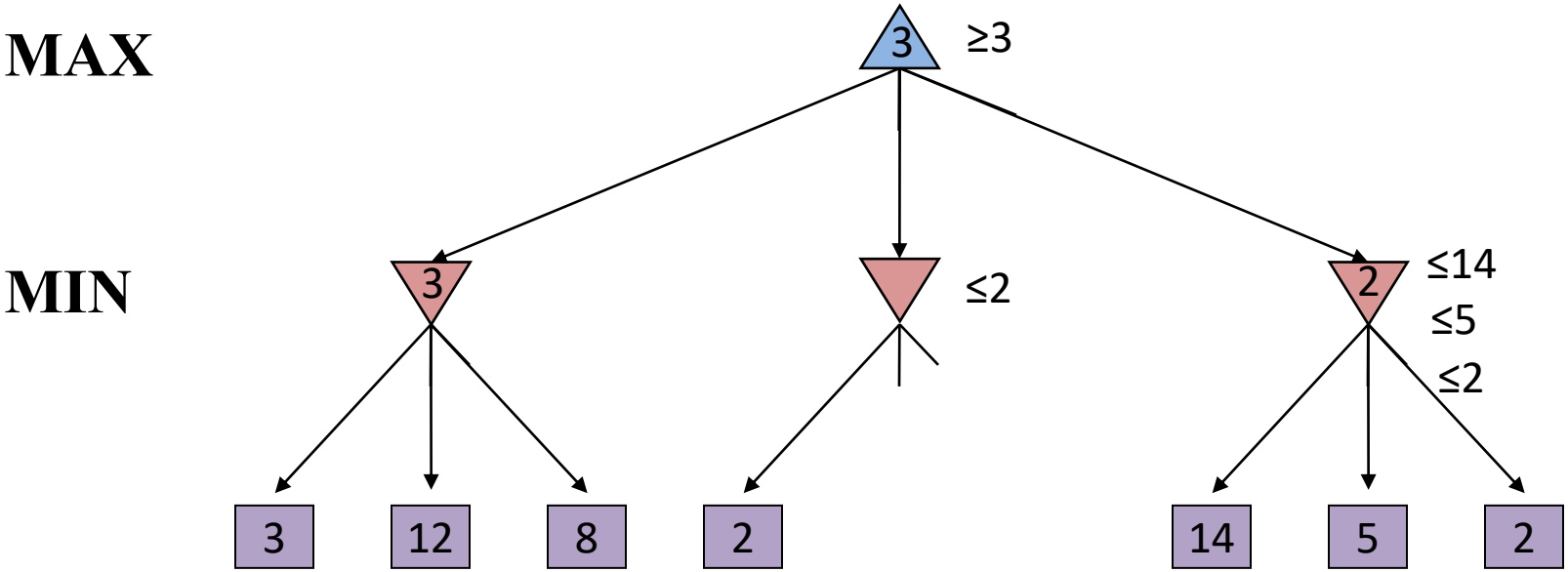
1. Crearea de euristici admisibile

2. Căutare adversarială:
   - jocuri adversariale
   - algoritmul MINIMAX
   - algoritmul α-β retezare (α-β prunning)

Cuprinsul cursului de azi

1. Algoritmul α-β retezare (α-β prunning)
2. Căutare ExpectiMax
3. Probleme de satisfacere de constrângeri
4. Algoritmi de căutare locală
5. Algoritmi genetici

Retezare alfa-beta



Tehnica de alfa-beta retezare, când este aplicată unui arbore de tip Minimax standard, **va întoarce aceeași mutare pe care ar furniza-o și Algoritmul MiniMax, dar într-un timp mai scurt**, întrucât realizează o retezare a unor ramuri (subarbori) ale arborelui care nu pot influența decizia finală și care nu mai sunt vizitate.

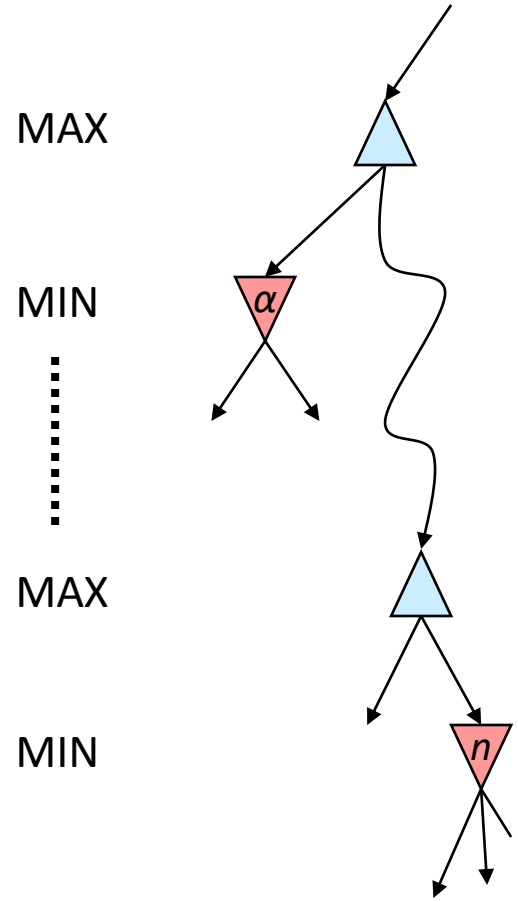
Retezare alfa-beta

înainte

după

Retezare alfa-beta (alfa-beta pruning)

- Cazurile când se aplică (pentru noduri de tip MIN)
  - calculăm pentru jucătorul MIN valoarea nodului *n*
  - iterăm după toți succesorii nodului *n* de la stânga la dreapta
  - valoarea estimată a nodului *n* va scădea pe măsură ce îi vizităm toți succesorii
  - cine este interesat de valoarea nodului *n*? MAX
  - fie α cea mai bună valoare curentă (garantată) pe care MAX o poate obține după ce a vizitat subarborii din stânga subarborelui actual vizitat pornind de la rădăcină
  - dacă valoarea curentă estimată a lui *n* devine mai mică decât α, MAX nu va alege acest drum, deci putem să renunțăm (să retezăm) să evaluăm valorile pentru celelalte noduri succesor ale lui *n*.



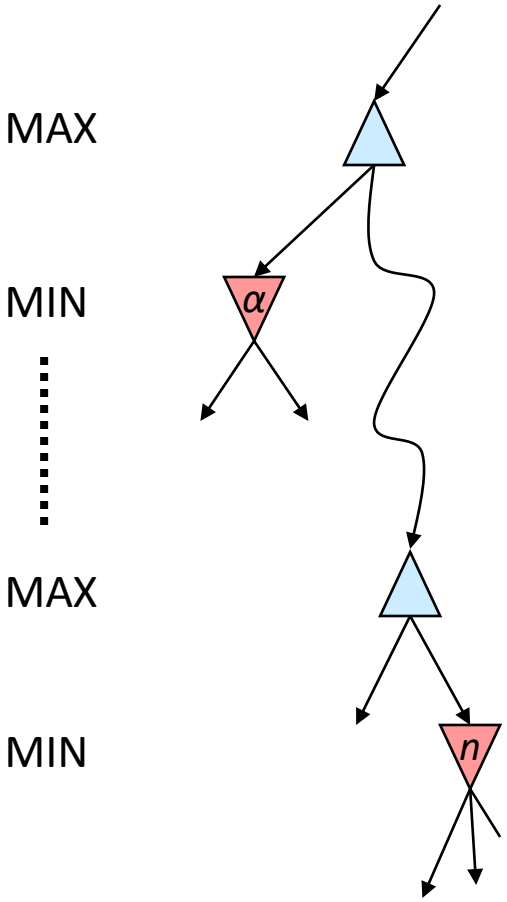
- Cazurile când se aplică (pentru noduri de tip MAX)
  - simetric, se definește în mod similar valoarea β ca fiind cea mai bună valoare (garantată) pe care MIN o poate obține.
  - MIN nu mai trebuie să mai ia în considerare nicio valoare internă mai mare sau egală cu β care este asociată oricărui nod intern *n* de tip MAX. Putem tăia (reteza) acea ramură a arborelui de joc.

Retezare alfa-beta (alfa-beta pruning)

- Încep cu parametri $\alpha = -\infty$, $\beta = +\infty$ din rădăcină pe care îi transmit (***ca valoare nu ca referință***) în subarbori și după ce îi parcurg pe aceștia actualizez valorile α și β.

- Pentru nodurile de tip MAX: după ce parcurg prima ramură din stânga (sub-arborele stâng) am o valoare minimă α. Pot atunci pentru sub-arborii care urmează să nu mai vizitez (să retez) sub-arbori din nodurile de tip MIN pentru care avem $\beta \leq \alpha$.

- Pentru nodurile de tip MIN: după ce parcurg prima ramură din stânga (sub-arborele stâng) am o valoare maximă β. Pot atunci pentru sub-arborii care urmează să nu mai vizitez (să retez) sub-arbori din nodurile de tip MAX pentru care avem $\alpha \geq \beta$.



Implementare Alfa-Beta retezare

α: opțiunea curentă cea mai bună a lui MAX (MAX vrea să maximizeze α) de la nodul curent la nodul rădăcină

β: opțiunea curentă cea mai bună a lui MIN (MIN vrea să minimizeze β) de la nodul curent la nodul rădăcină

Întotdeauna vom avea α ≤ β

```
def max-value(stare, α, β):   //nodul n e de tip MAX, MAX are α, MIN are β
    initializeaza v = -∞
    pentru fiecare succesor al stării:          //pentru fiecare succesor al lui n
        v = max(v, value(successor, α, β))   //calculăm valoarea v și actualizăm max
        if v ≥ β return v    //dacă v depășește limita β a lui MIN nu mai continuăm
                             // întoarcem în acest caz v  = o estimare a lui n
        α = max(α, v)        // actualizăm valoarea α
    return v
```

Implementare Alfa-Beta retezare

α: opțiunea curentă cea mai bună a lui MAX (MAX vrea să maximizeze α) de la nodul curent la nodul rădăcină

β: opțiunea curentă cea mai bună a lui MIN (MIN vrea să minimizeze β) de la nodul curent la nodul rădăcină

Întotdeauna vom avea $\alpha \leq \beta$

```
def max-value(stare, α, β):
    initializeaza v = -∞
    pentru fiecare succesor al stării:
        v = max(v, value(successor, α, β))
        if v ≥ β returneaza v
        α = max(α, v)
    returneaza v
```

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```

Implementare MiniMax

```
def value(stare):
    dacă stare este o stare terminală: returnează utilitatea stării
    dacă următorul agent care mută este MAX: returnează max-value(stare)
    dacă următorul agent care mută este MIN: returnează min-value(stare)
```

```
def max-value(stare):
    initializeaza v = -∞
    pentru fiecare successor al starii:
        v = max(v, value(successor))
    returneaza v
```

```
def min-value(stare):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor))
    returneaza v
```

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

$$V(s) = \min_{s' \in \text{Successors}(s)} V(s')$$

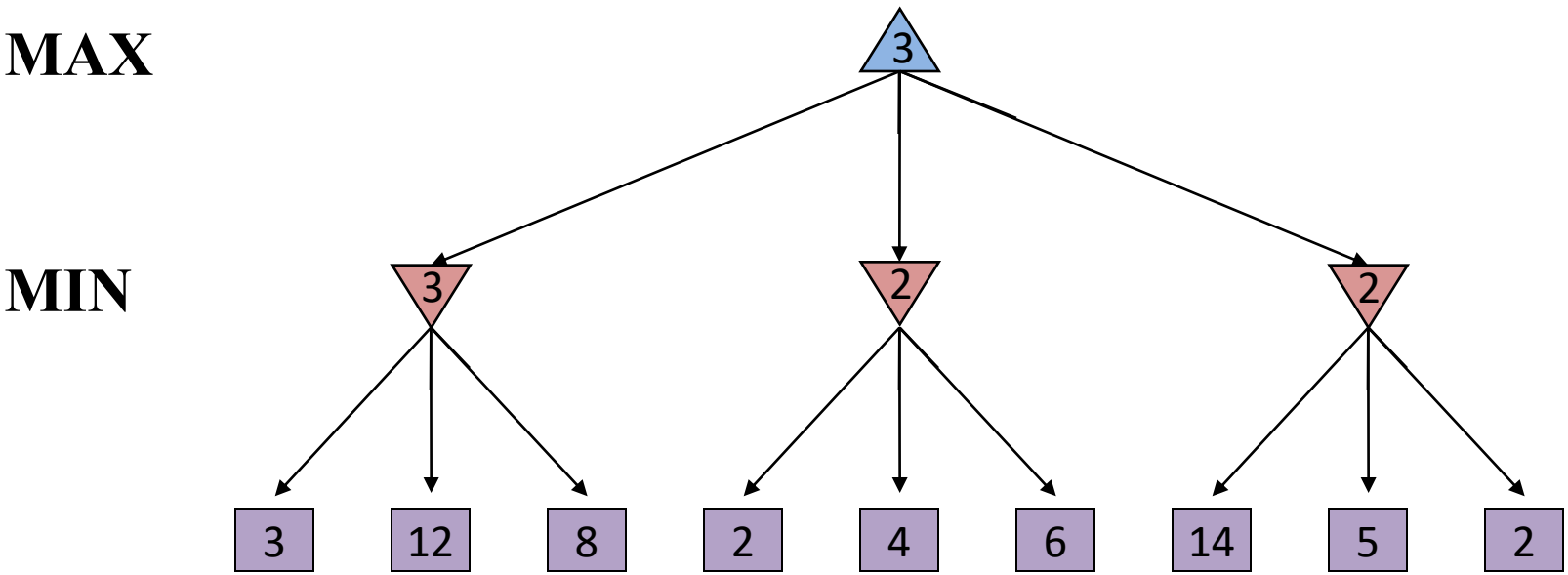
Implementare Alfa-Beta retezare

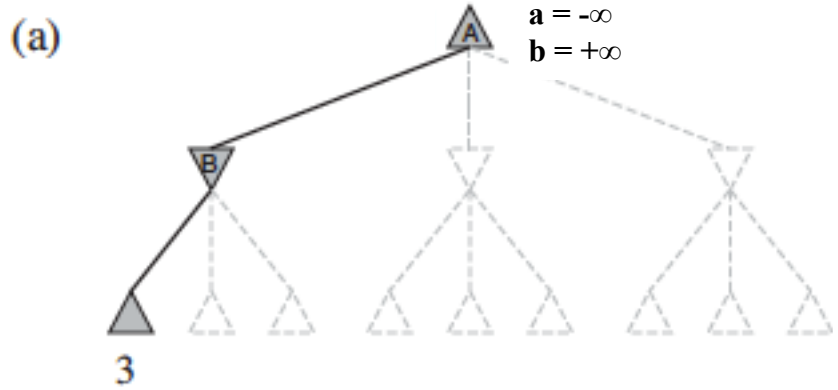
```
def value(stare, α, β):
    dacă stare este o stare terminală: returnează utilitatea
        stării
    dacă următorul agent care mută este MAX:
        returnează max-value(stare, α, β)
    dacă următorul agent care mută este MIN: returnează
        min-value(stare, α, β)
```

```
def max-value(stare, α, β):
    initializeaza v = -∞
    pentru fiecare succesor al stării:
        v = max(v, value(successor, α, β))
        if v ≥ β returneaza v
        α = max(α, v)
    returneaza v
```

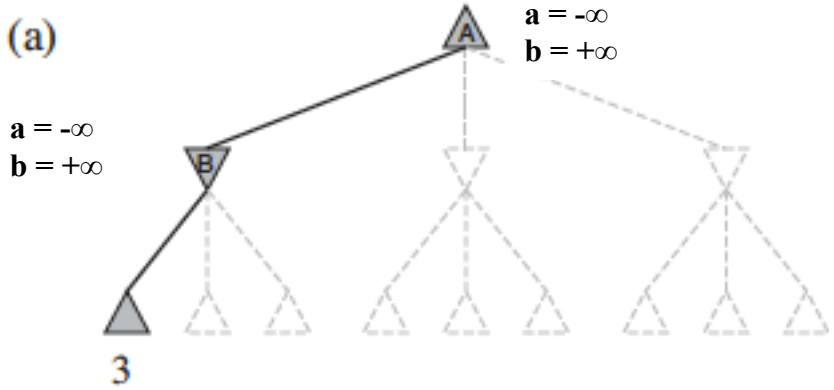
```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```

Exemplu MiniMax

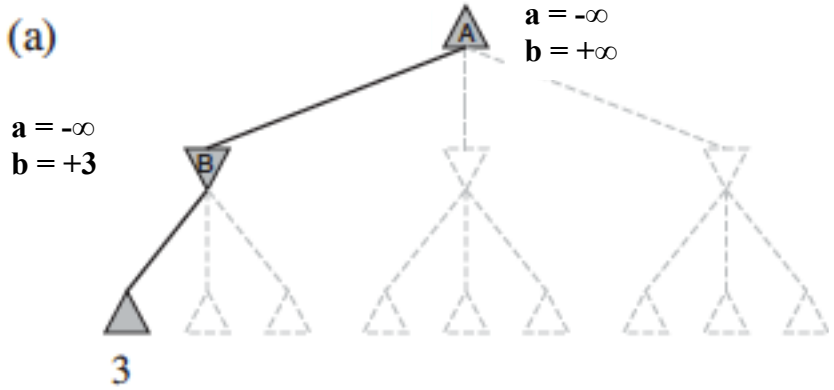




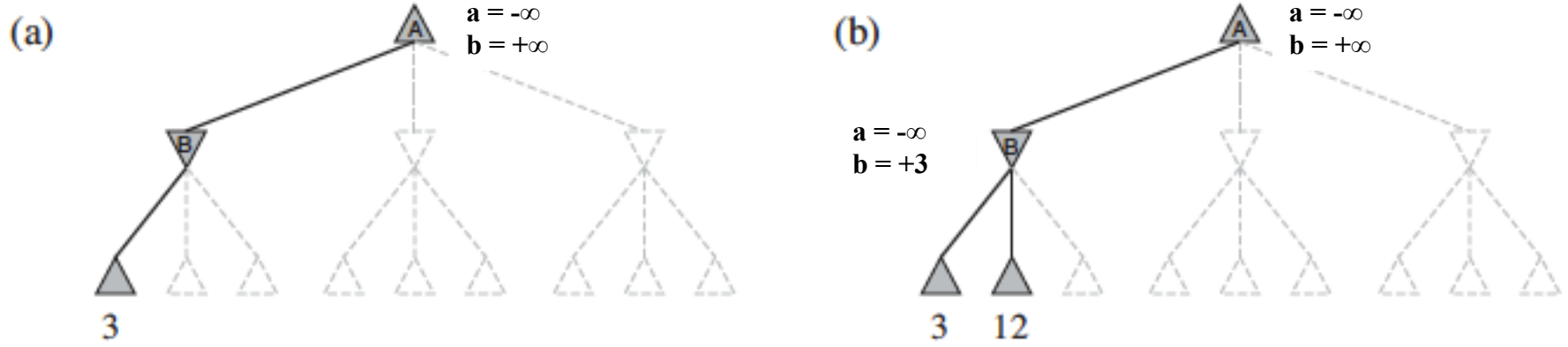

```
def max-value(stare, α, β):
    initializeaza v = -∞
    pentru fiecare succesor al stării:
        v = max(v, value(successor, α, β))
        if v ≥ β returneaza v
        α = max(α, v)
    returneaza v
```



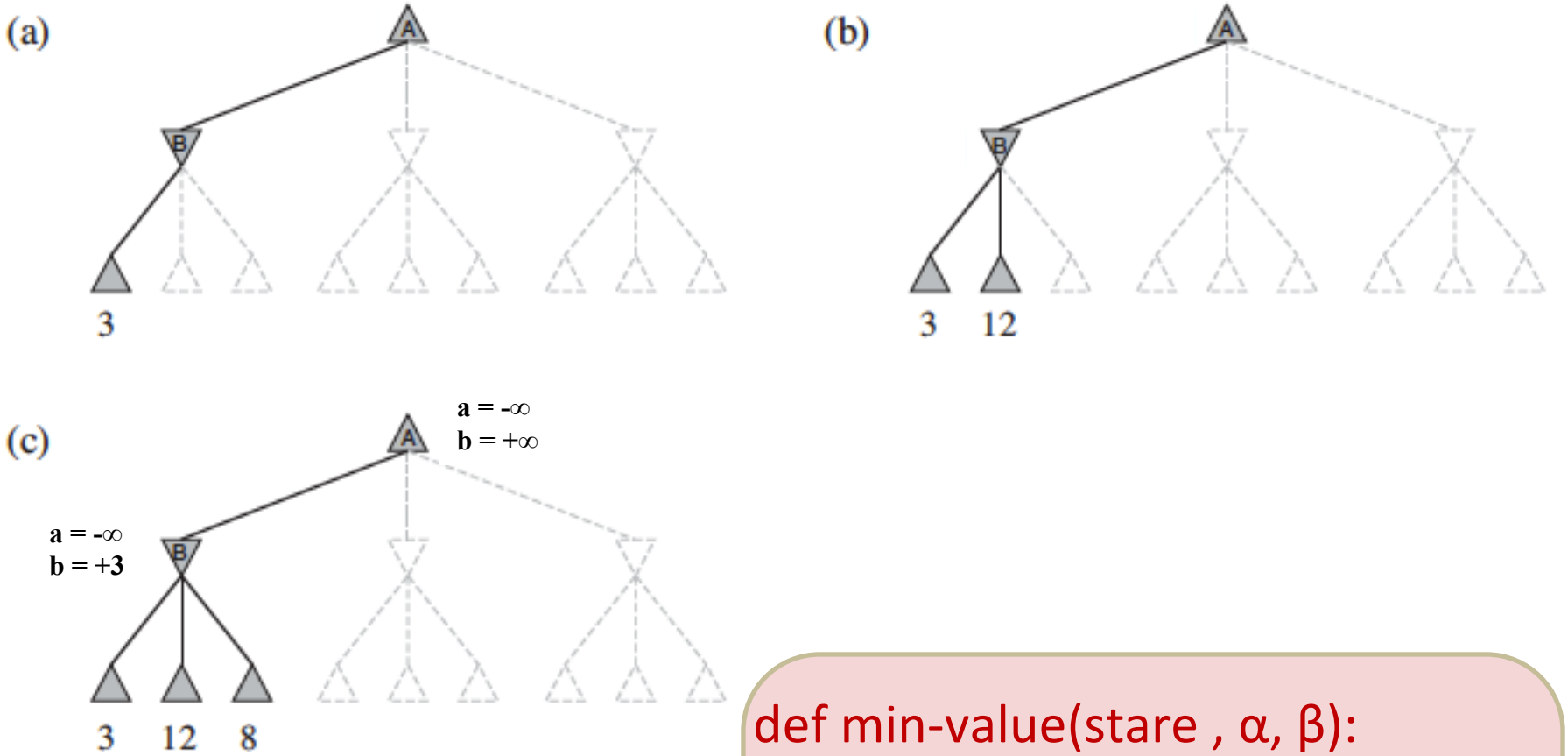

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```



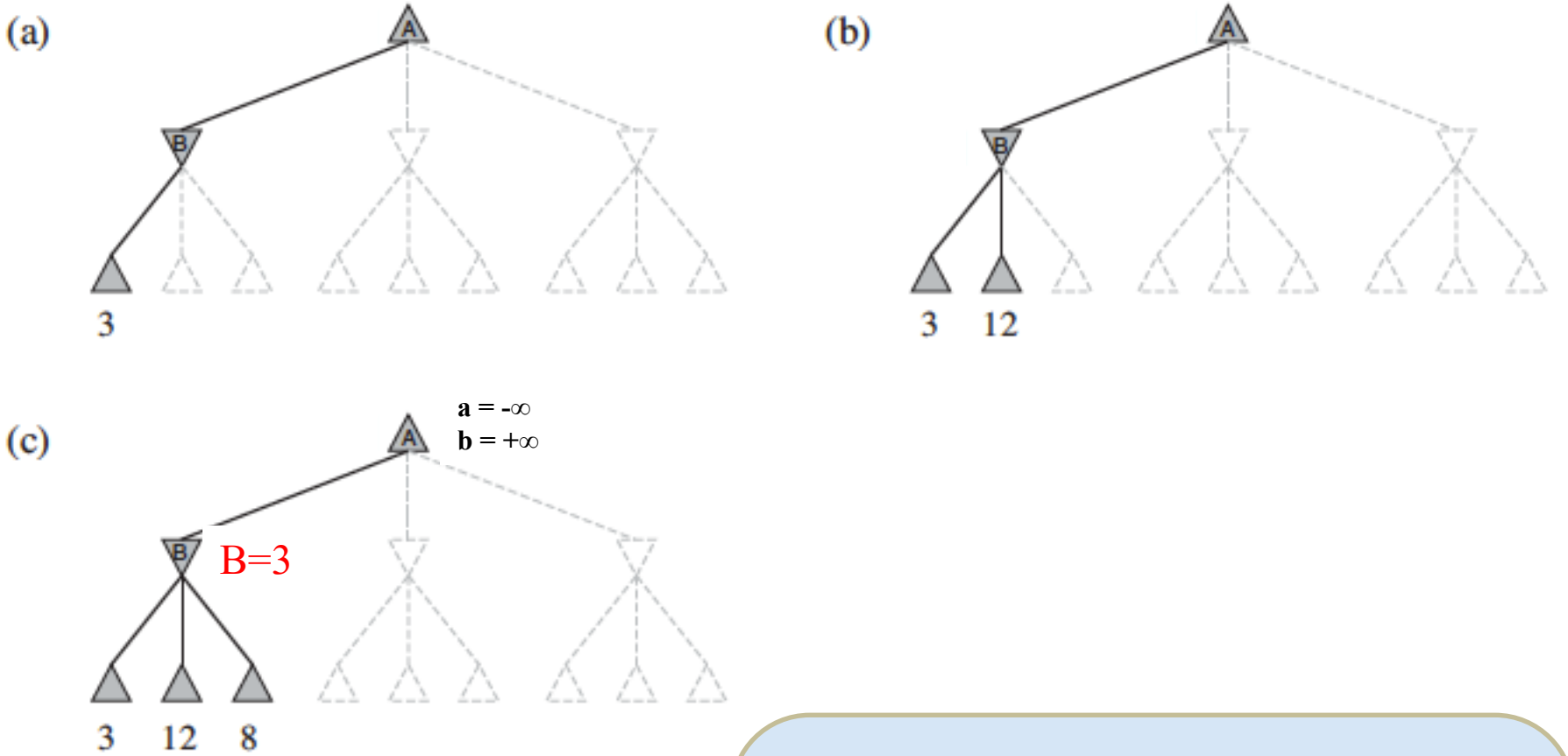

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```



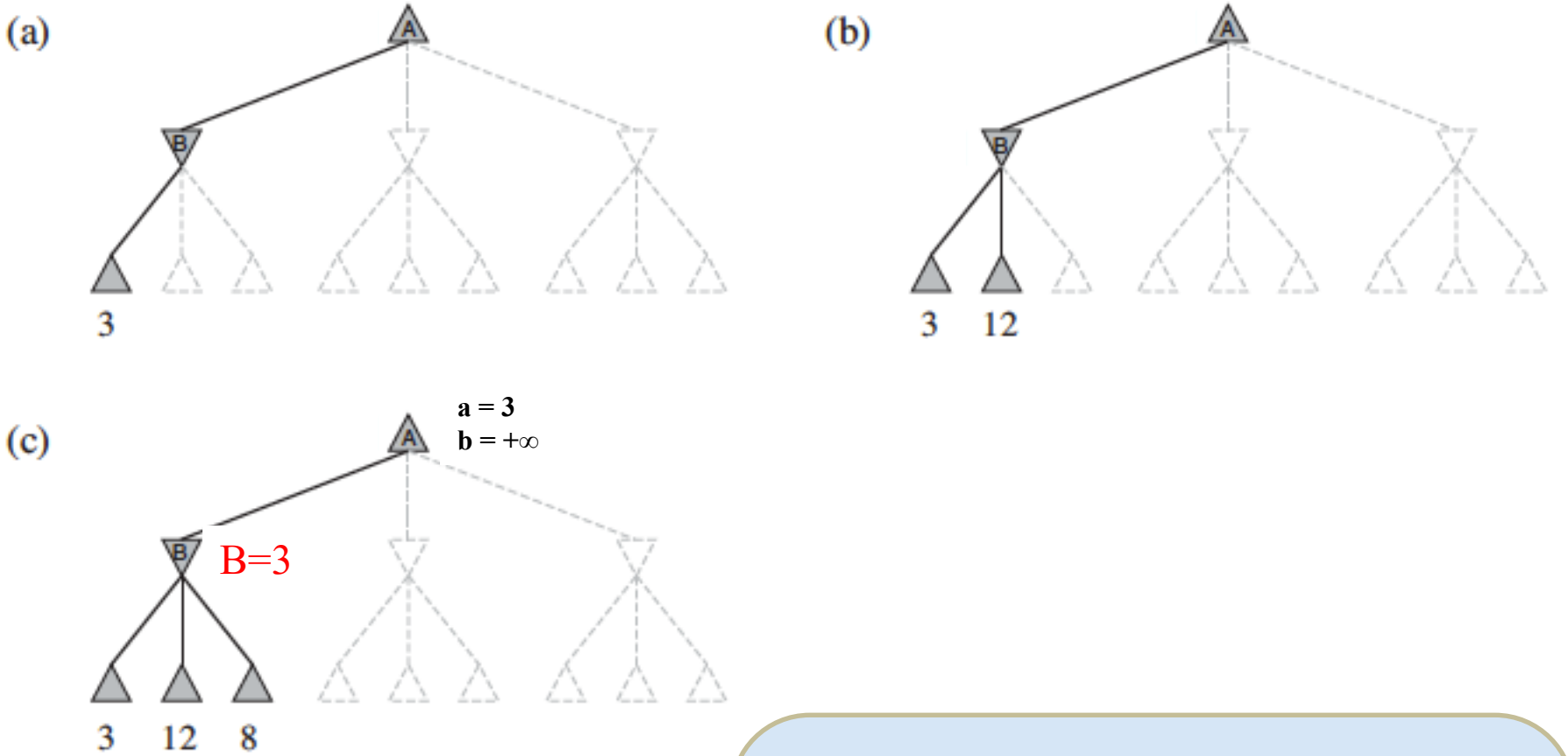

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```



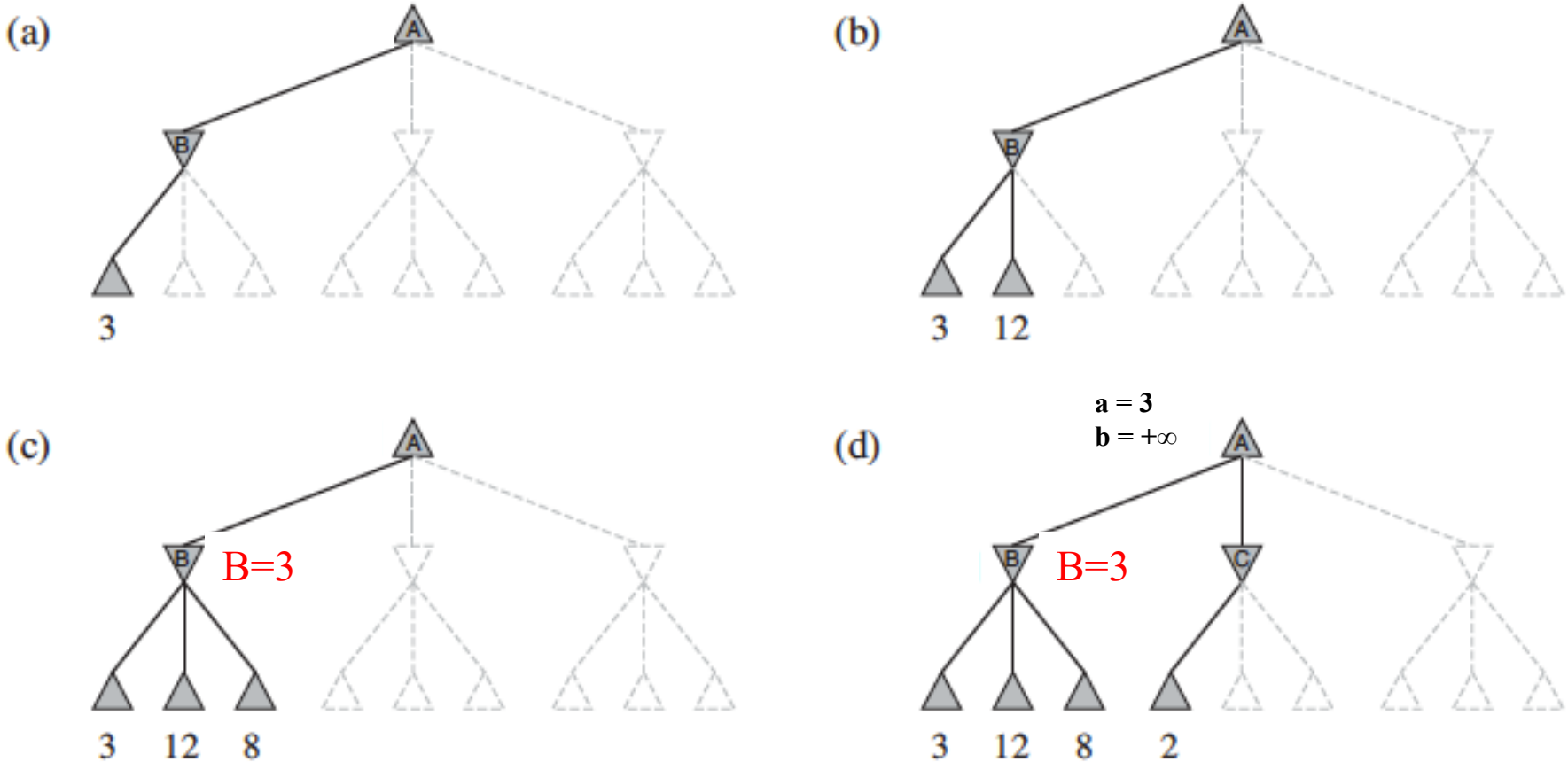

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```



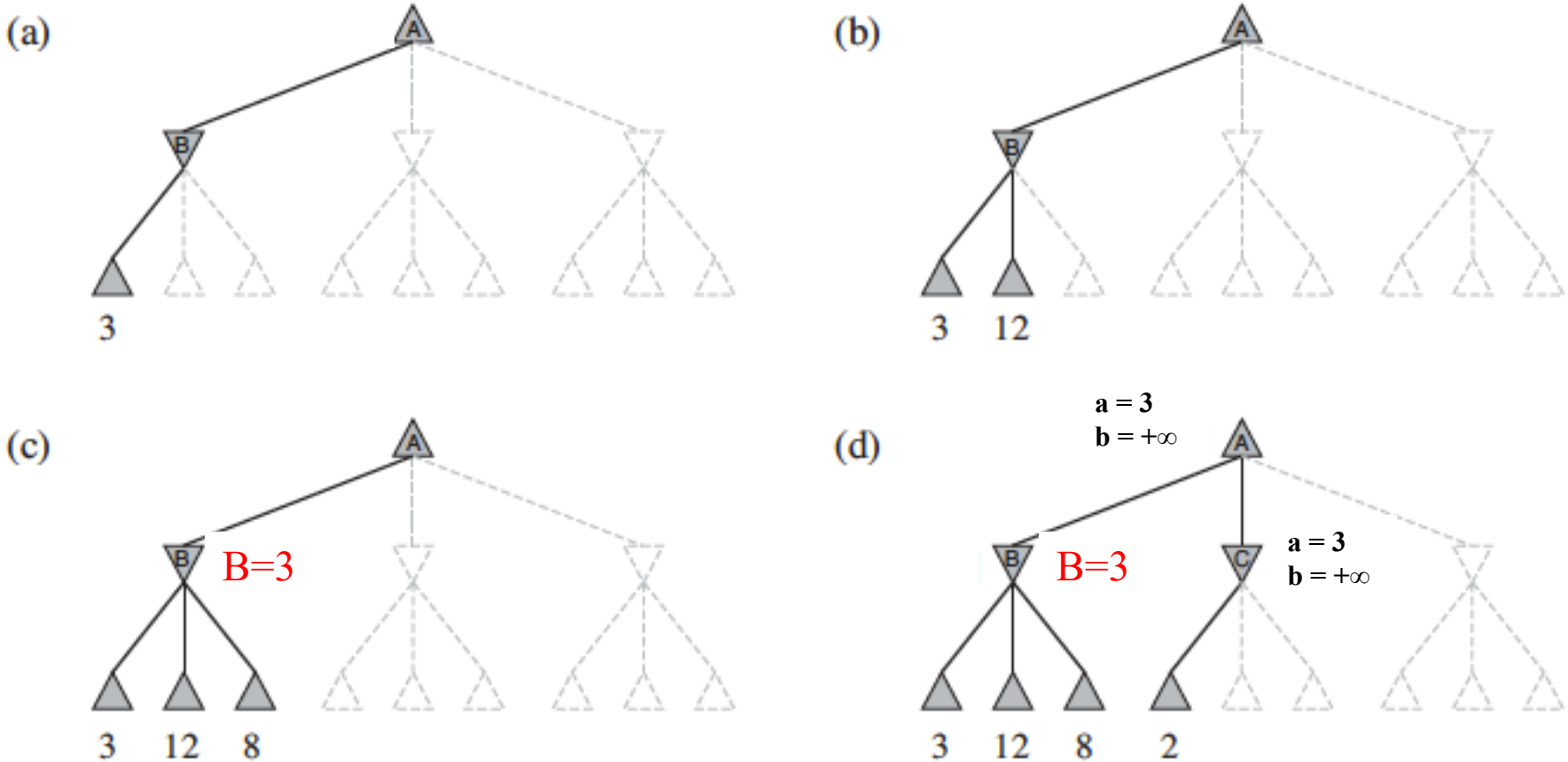

```
def max-value(stare, α, β):
    initializeaza v = -∞
    pentru fiecare succesor al stării:
        v = max(v, value(successor, α, β))
        if v ≥ β returneaza v
        α = max(α, v)
    returneaza v
```



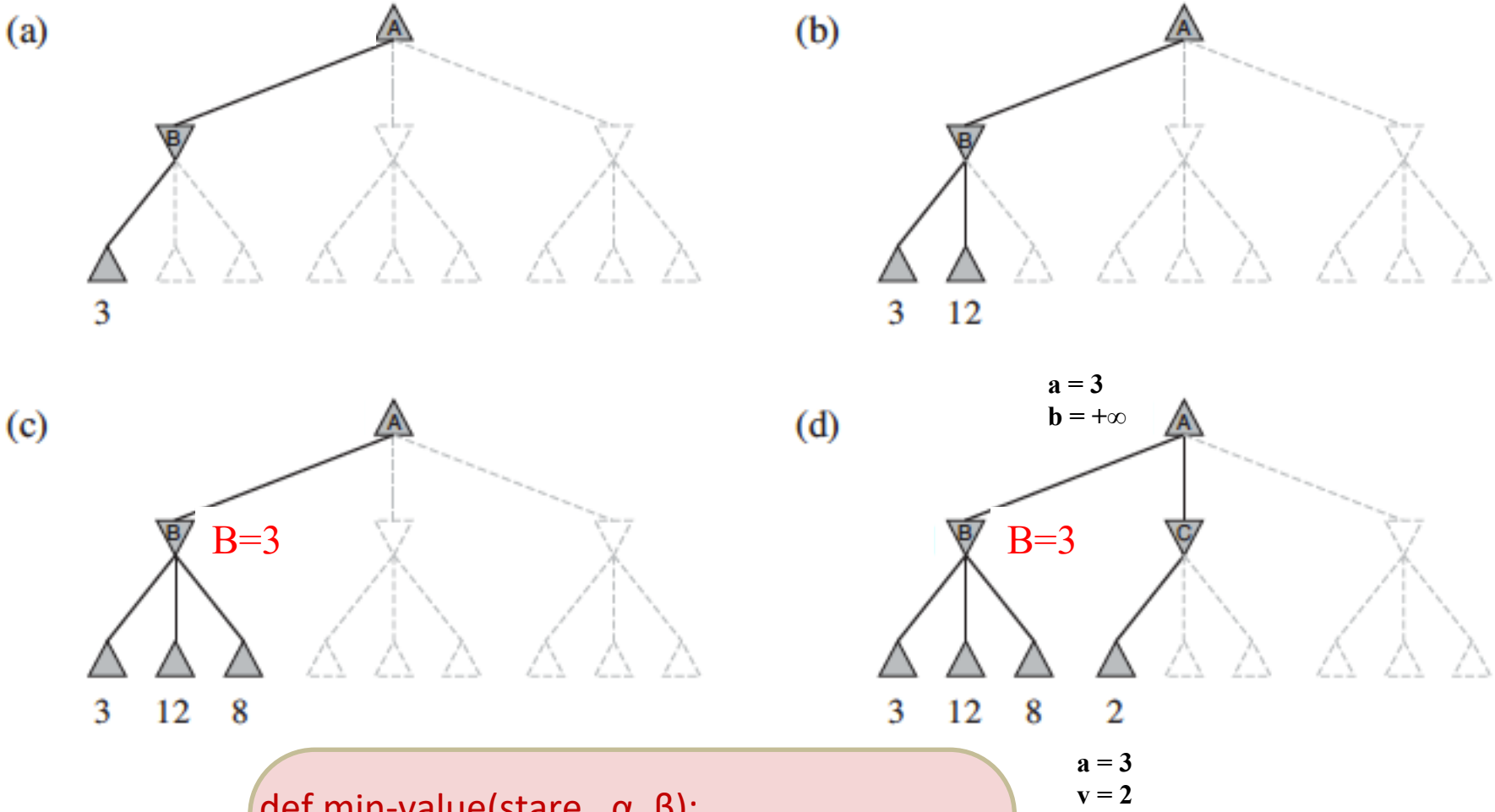

```
def max-value(stare, α, β):
    initializeaza v = -∞
    pentru fiecare succesor al stării:
        v = max(v, value(successor, α, β))
        if v ≥ β returneaza v
        α = max(α, v)
    returneaza v
```



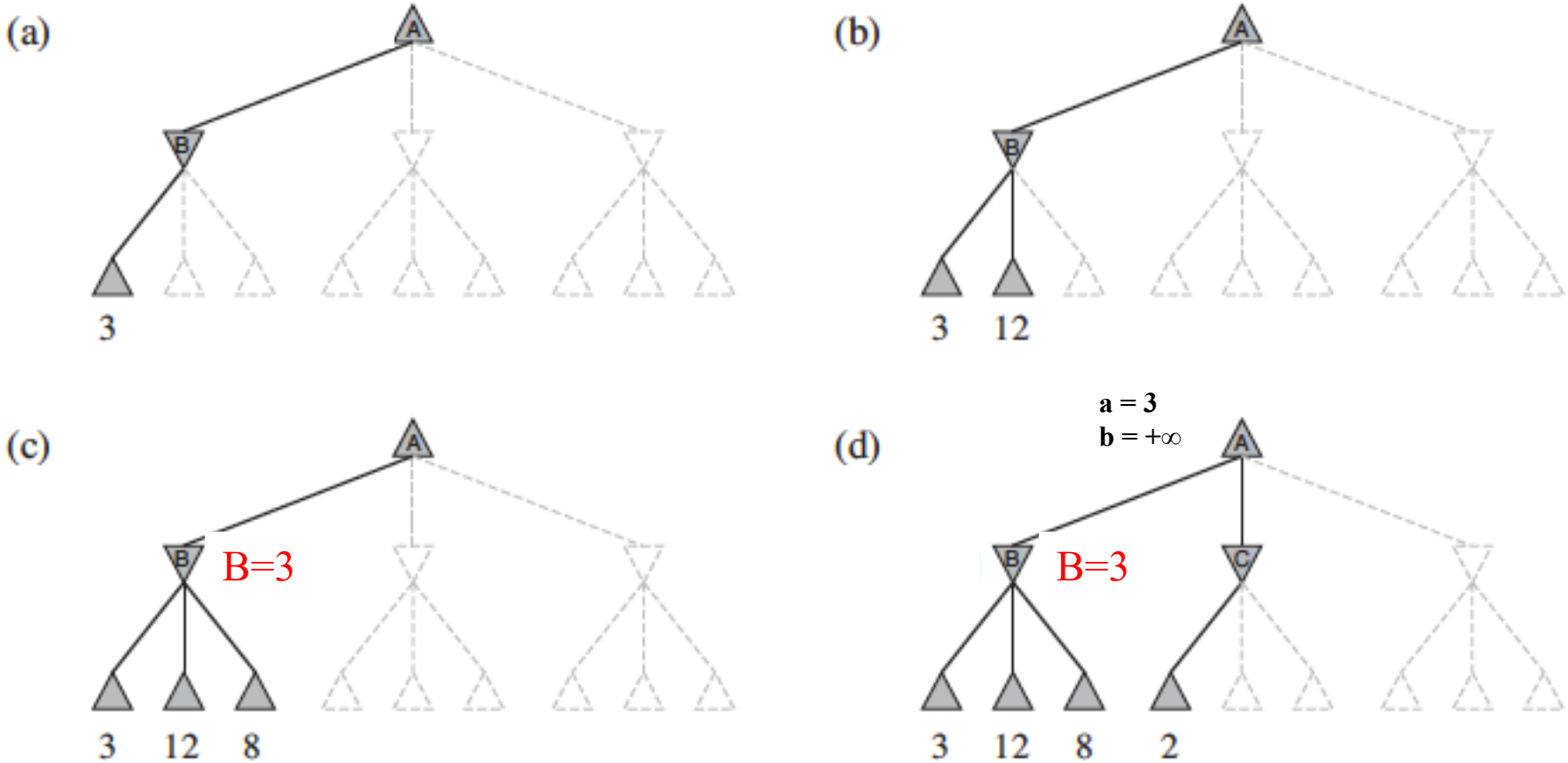

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```



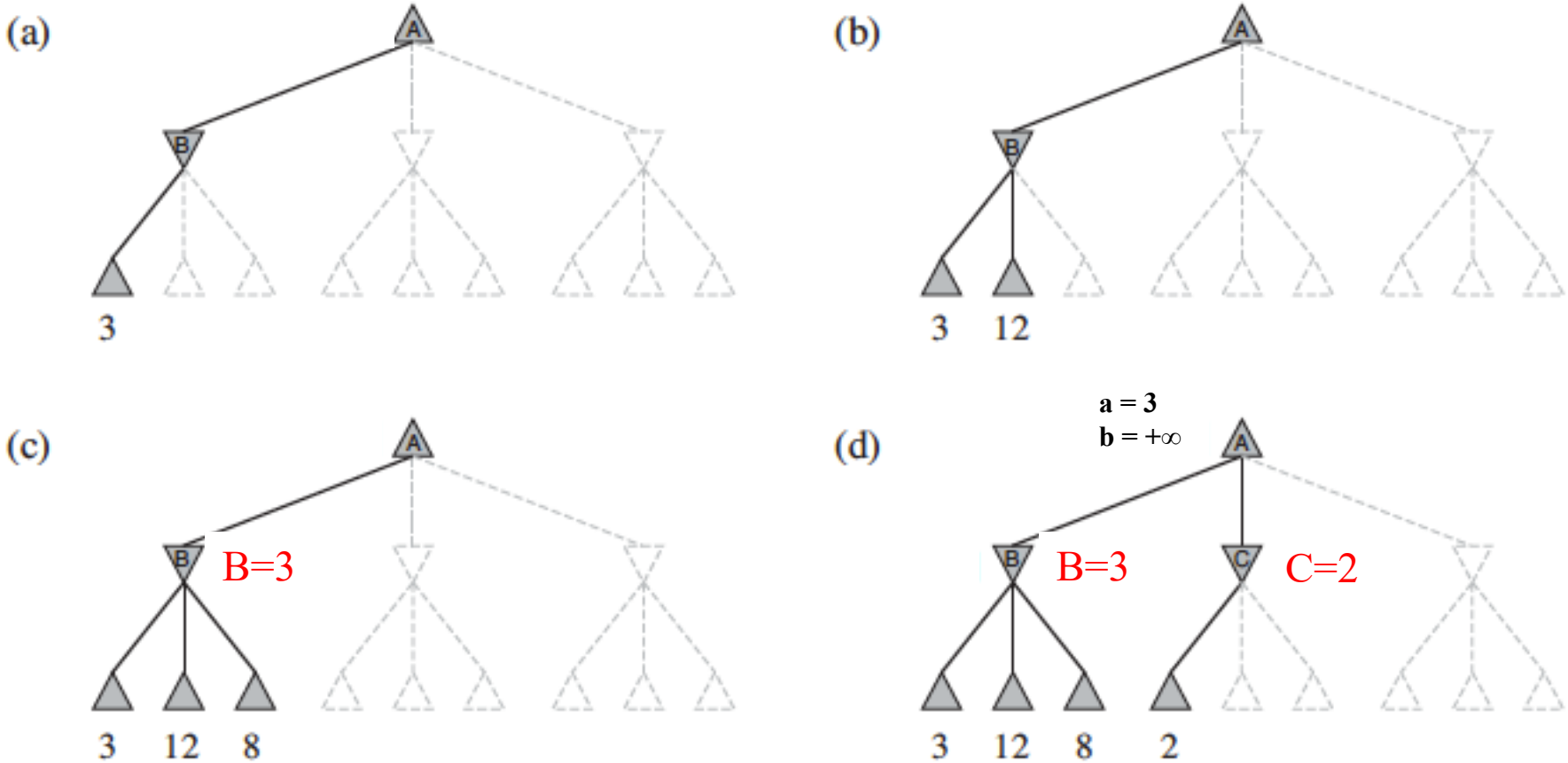

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```




```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```

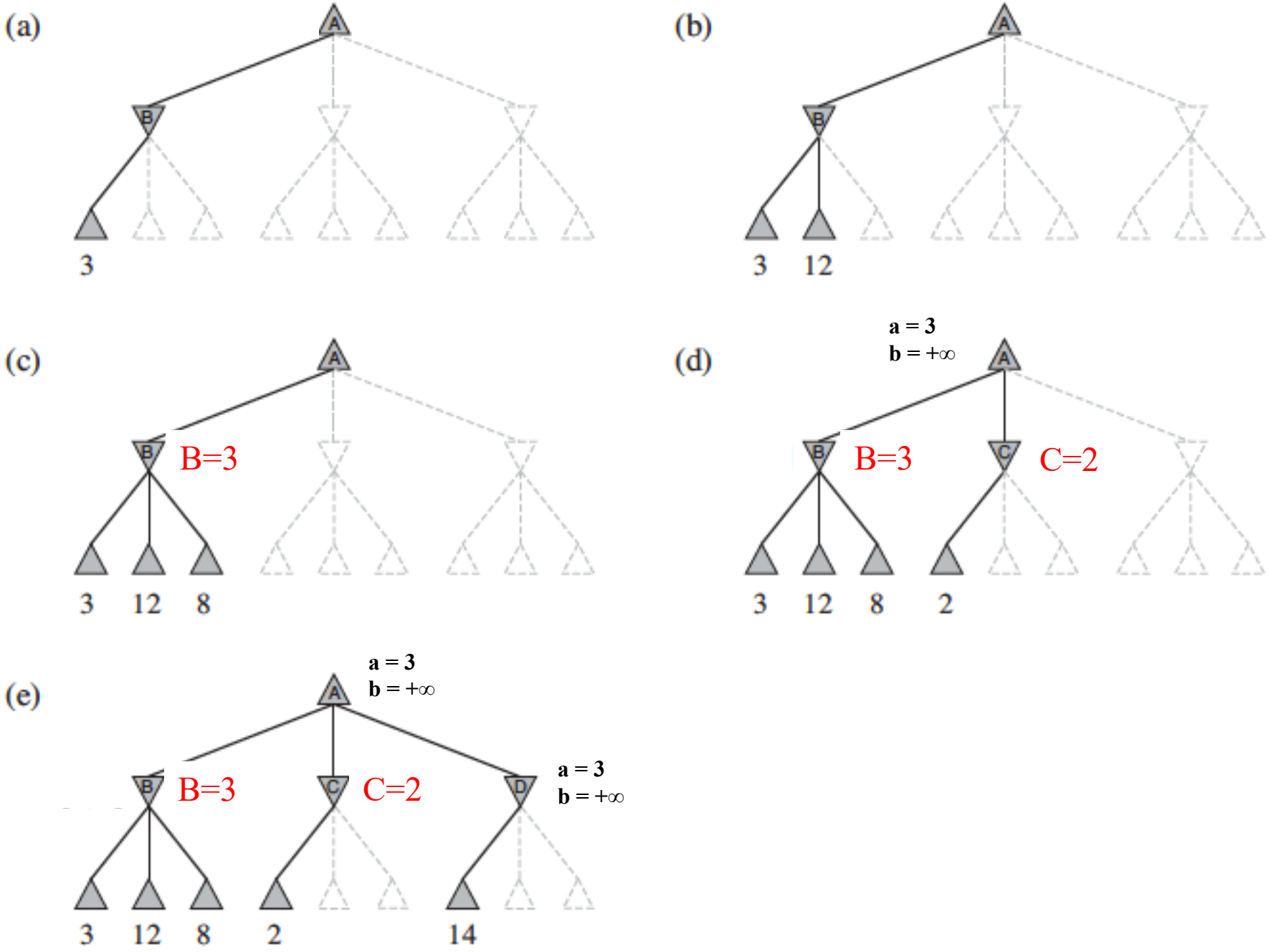


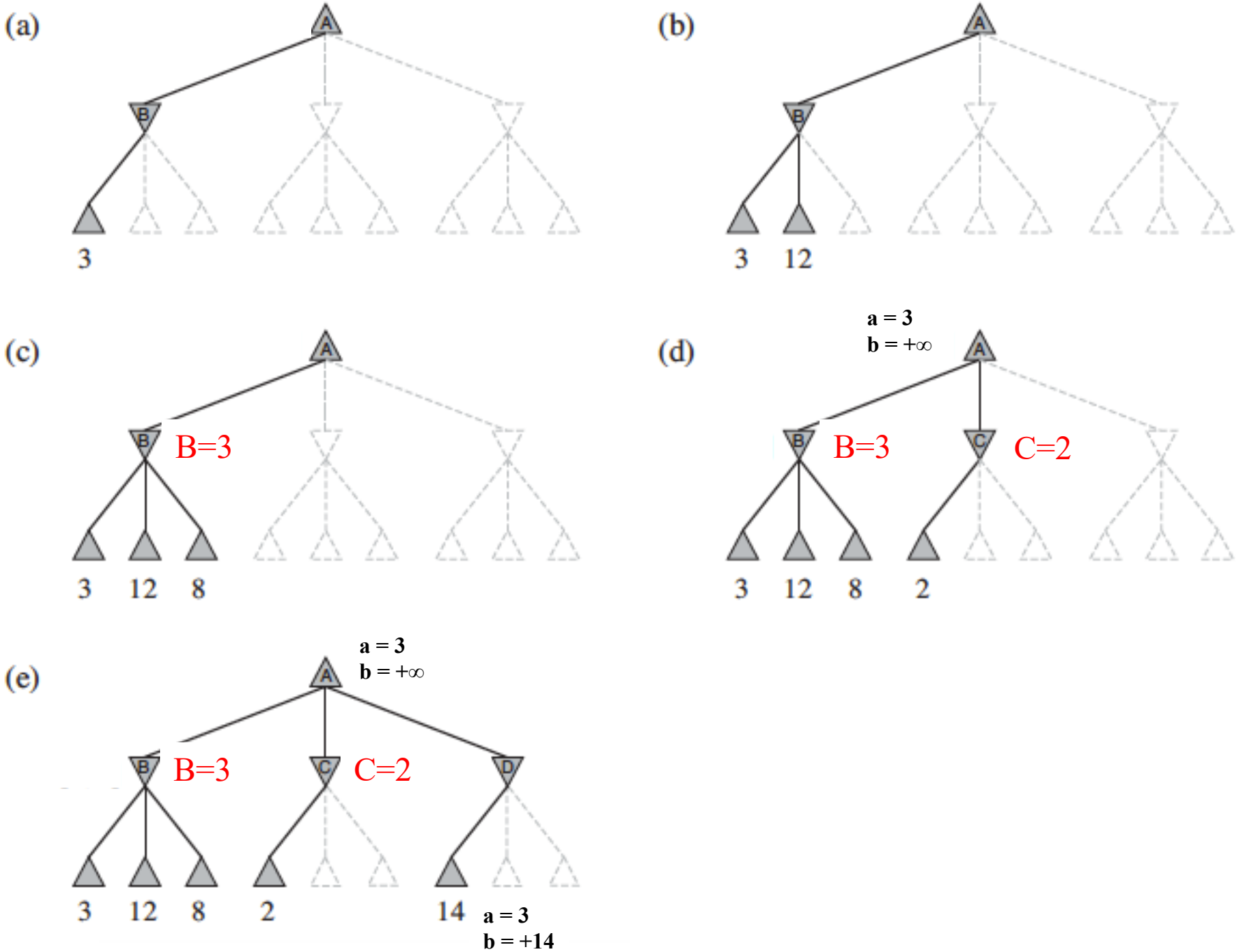

```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```

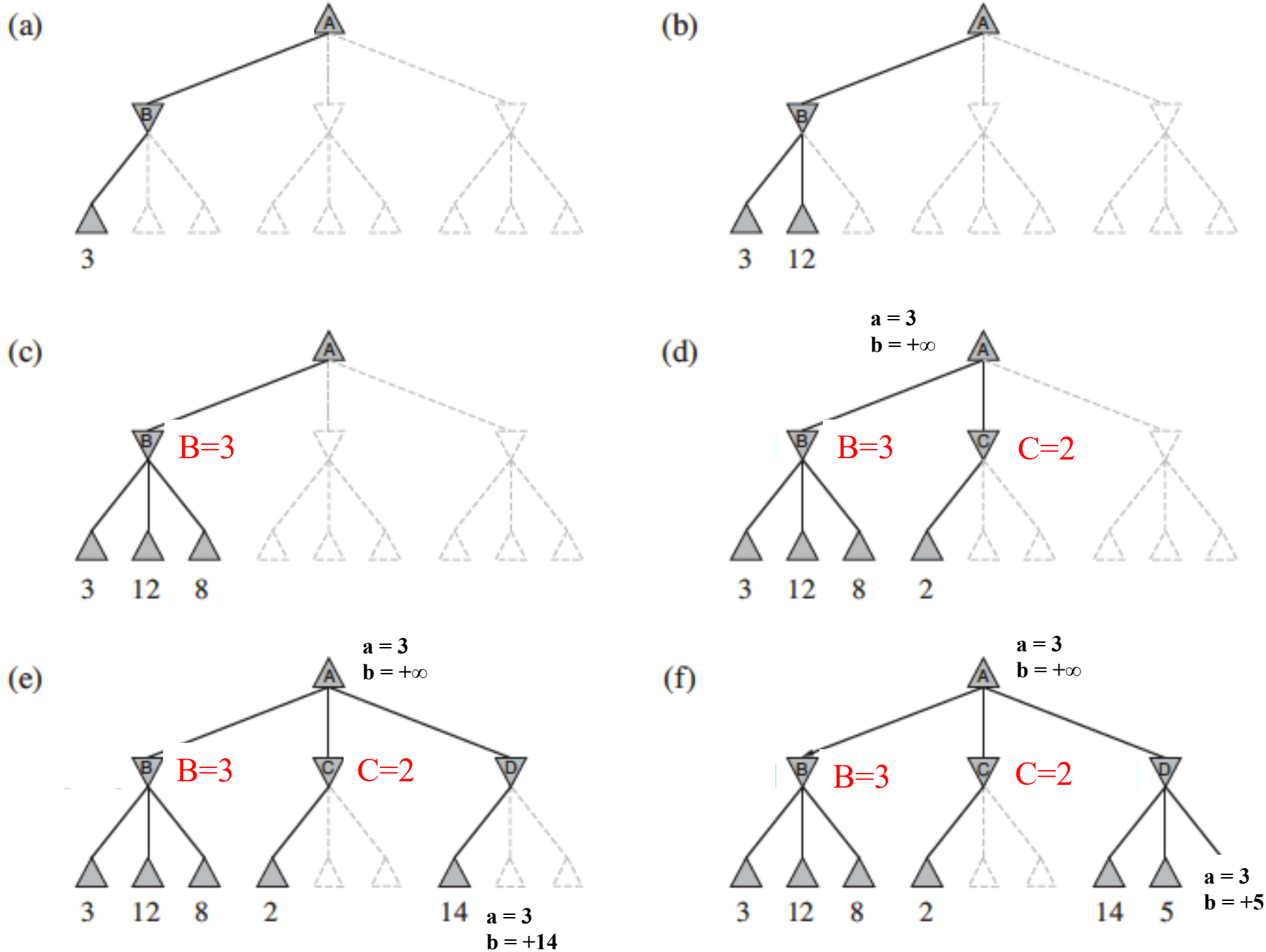


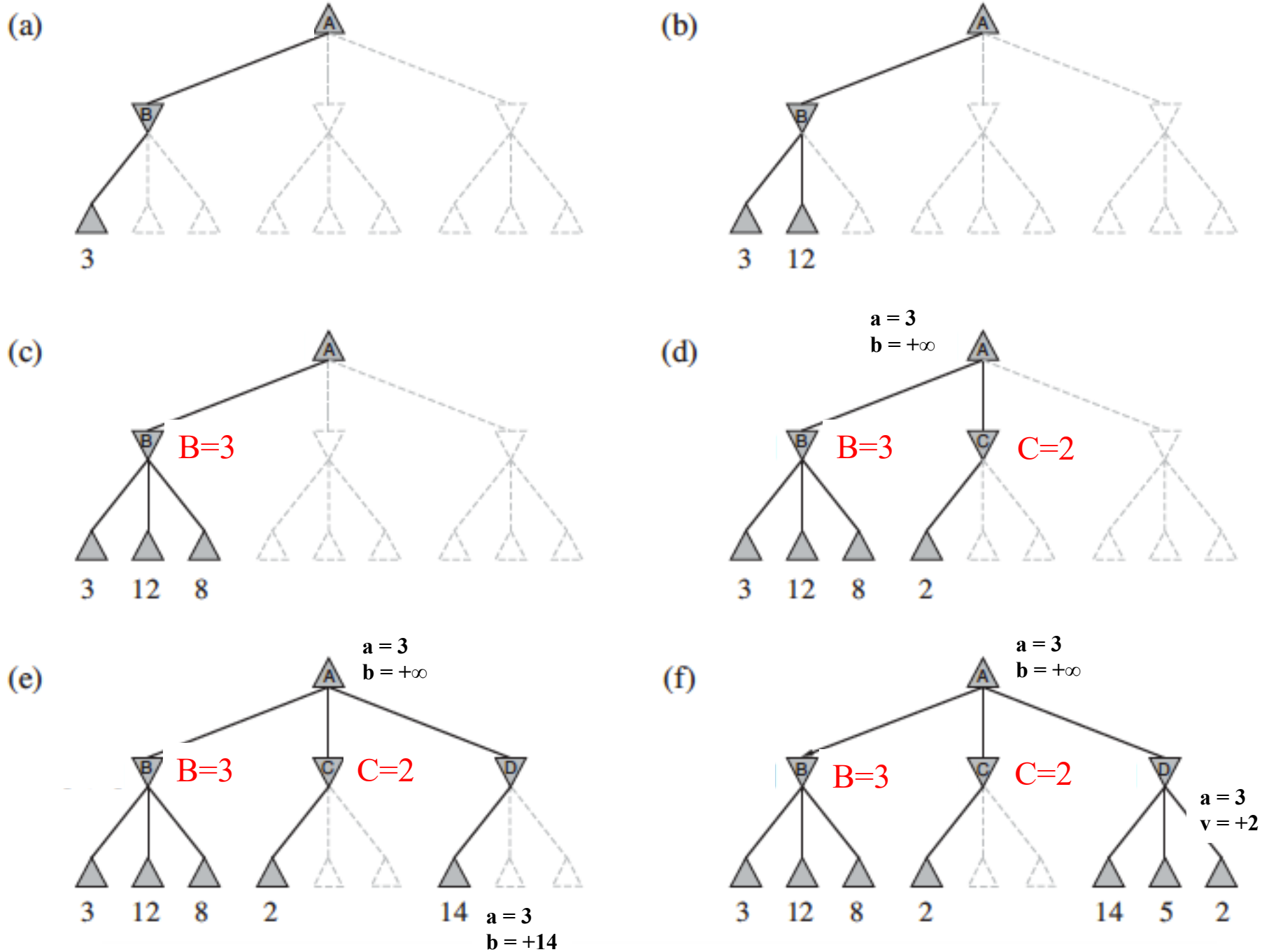
a = 3
v = 2
prunning

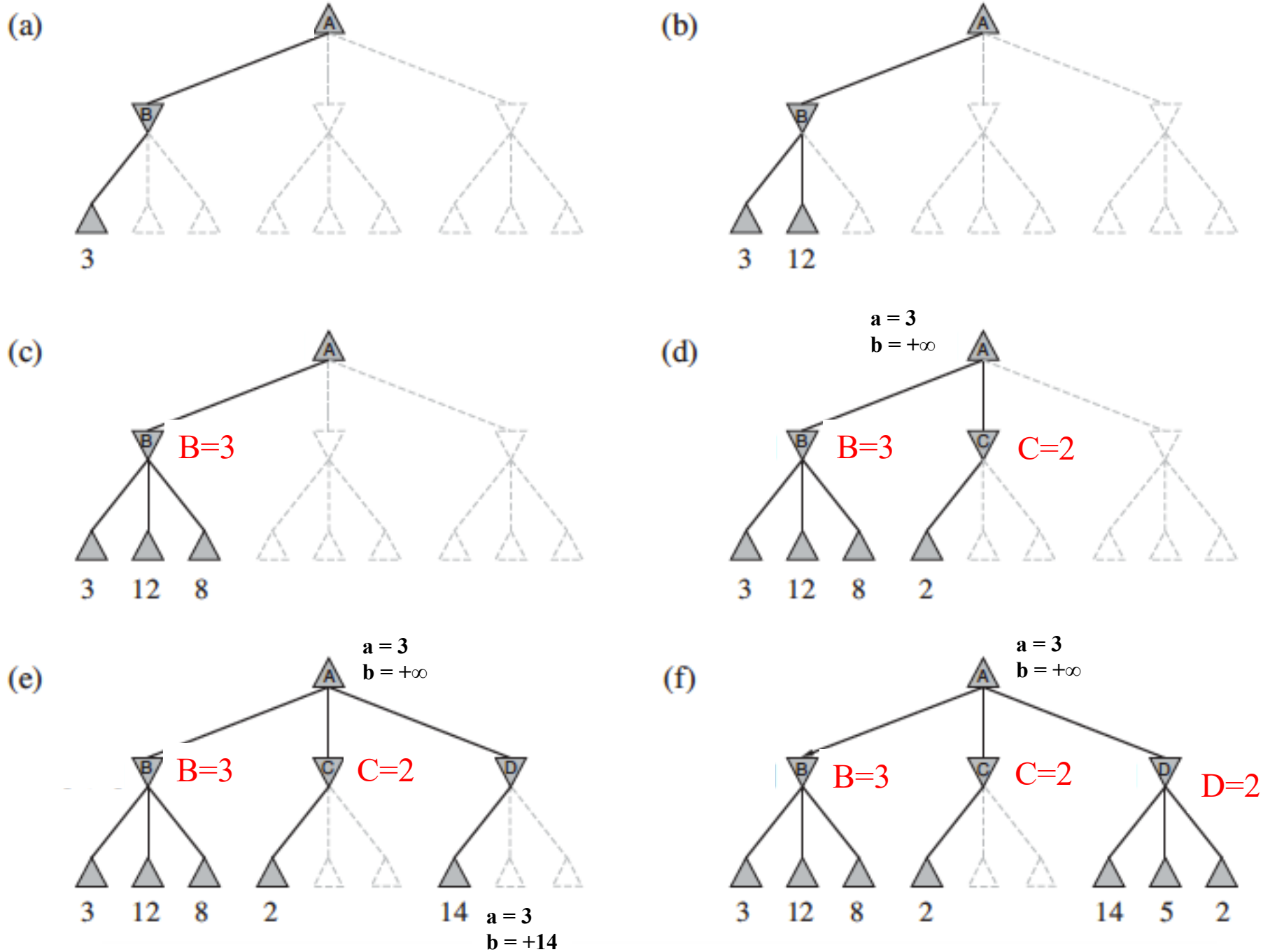
```
def min-value(stare , α, β):
    initializeaza v = +∞
    pentru fiecare succesor al stării:
        v = min(v, value(successor, α, β))
        if v ≤ α returneaza v
        β = min(β, v)
    returneaza v
```

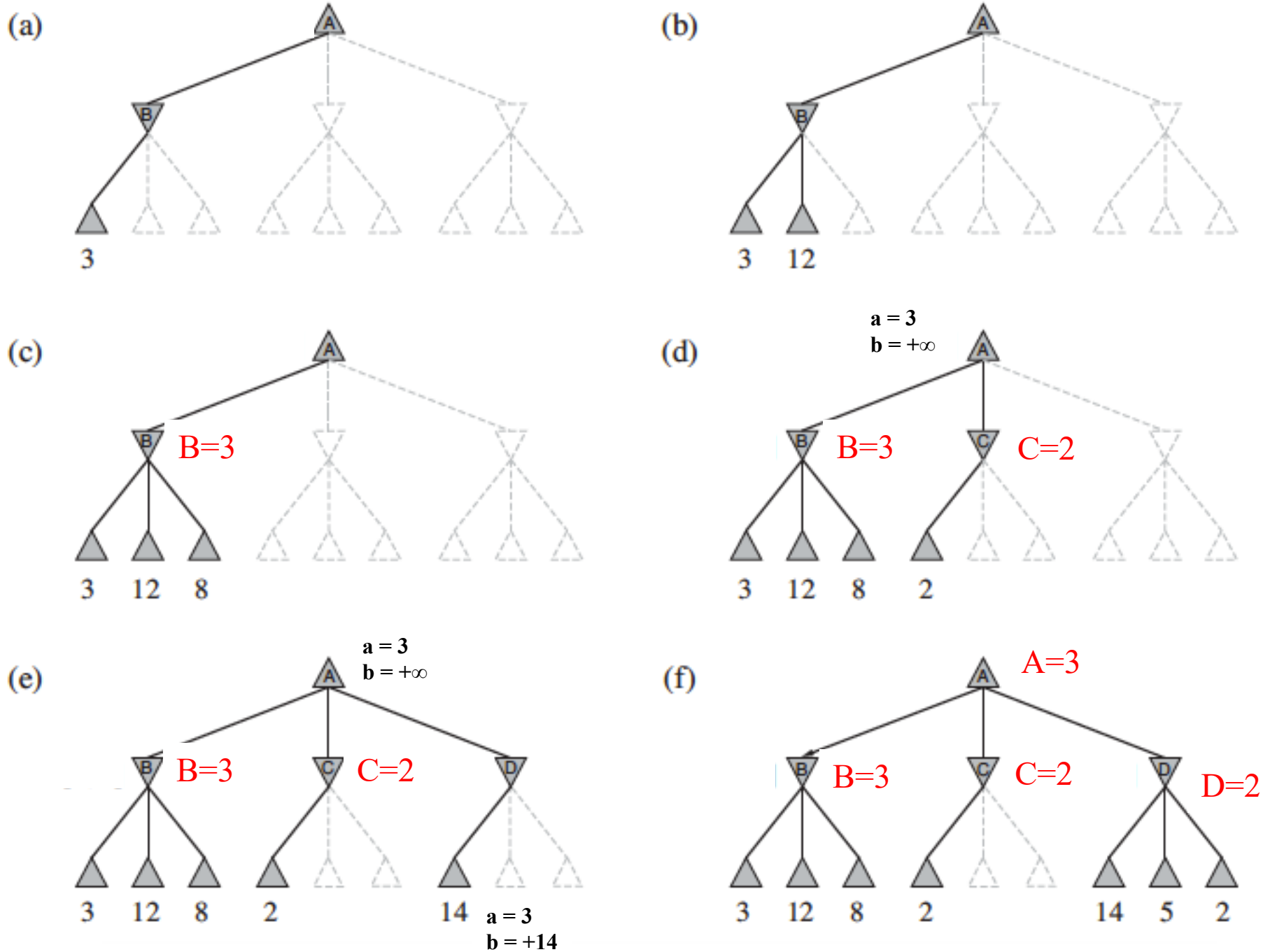

(a)
A
B
3
(b)
A
B
3 12
(c)
A
B
B=3
3 12 8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3 12 8 2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
a = 3
b = +∞
3 12 8 2 14


(a)
A
B
3
(b)
A
B
3
12
(c)
A
B
B=3
3
12
8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3
12
8
2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3
12
8
2
14
a = 3
b = +14

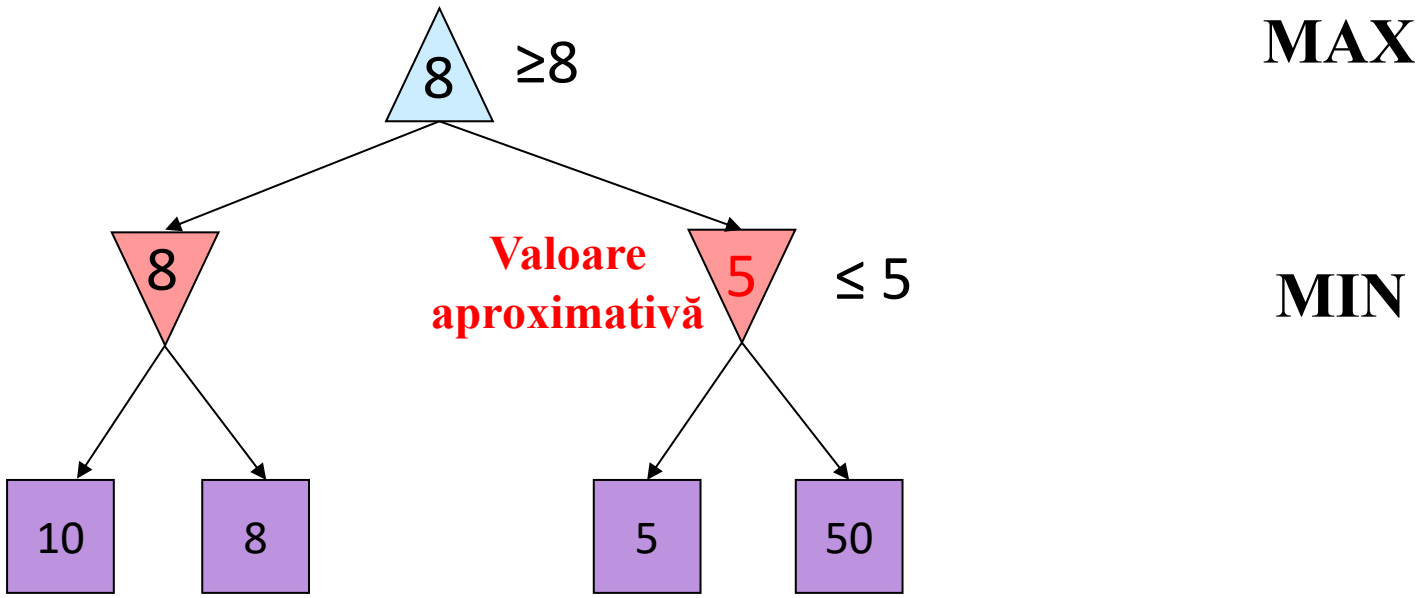

(a)
A
B
3
(b)
A
B
3 12
(c)
A
B
B=3
3 12 8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3 12 8 2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3 12 8 2 14
a = 3
b = +14
(f)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3 12 8 2 14 5
a = 3
b = +5


(a)
A
B
3
(b)
A
B
3 12
(c)
A
B
B=3
3 12 8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3 12 8 2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3 12 8 2 14
a = 3
b = +14
(f)
a = 3
b = +∞
A
B
B=3
C
C=2
D
a = 3
v = +2
3 12 8 2 14 5 2

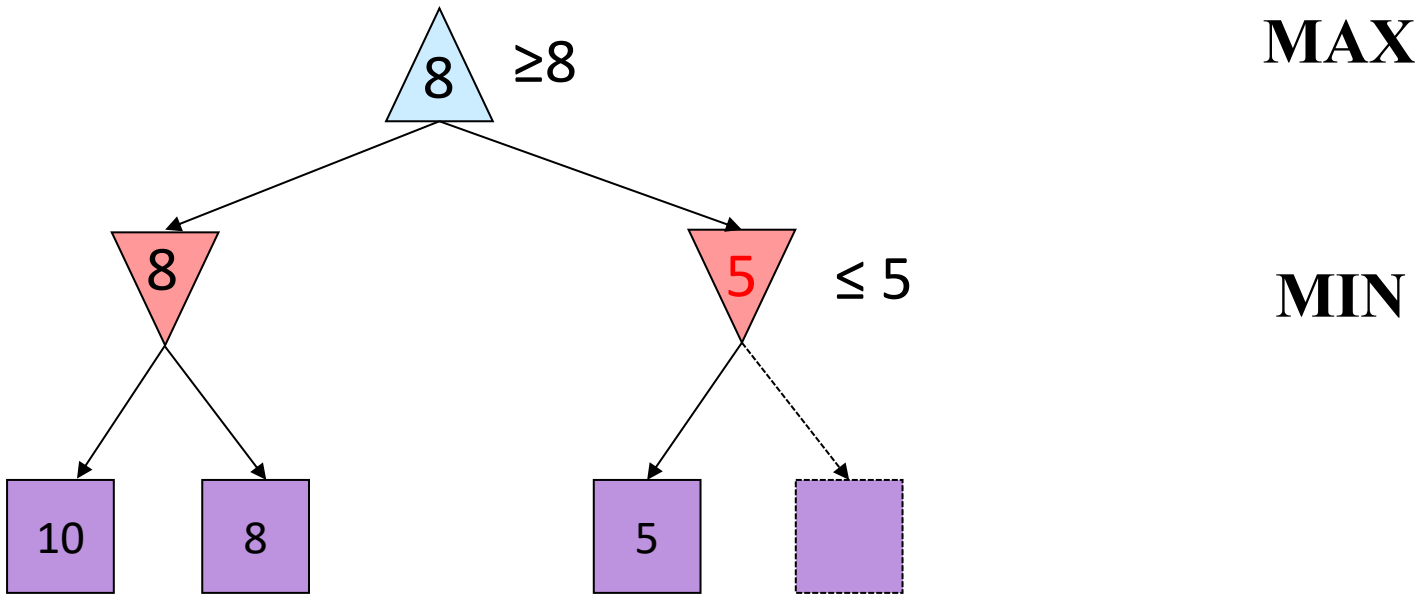

(a)
A
B
3
(b)
A
B
3 12
(c)
A
B
B=3
3 12 8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3 12 8 2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3 12 8 2 14
a = 3
b = +14
(f)
a = 3
b = +∞
A
B
B=3
C
C=2
D
D=2
3 12 8 2 14 5 2


(a)
A
B
3
(b)
A
B
3 12
(c)
A
B
B=3
3 12 8
(d)
a = 3
b = +∞
A
B
B=3
C
C=2
3 12 8 2
(e)
a = 3
b = +∞
A
B
B=3
C
C=2
D
3 12 8 2 14
a = 3
b = +14
(f)
A
A=3
B
B=3
C
C=2
D
D=2
3 12 8 2 14 5 2

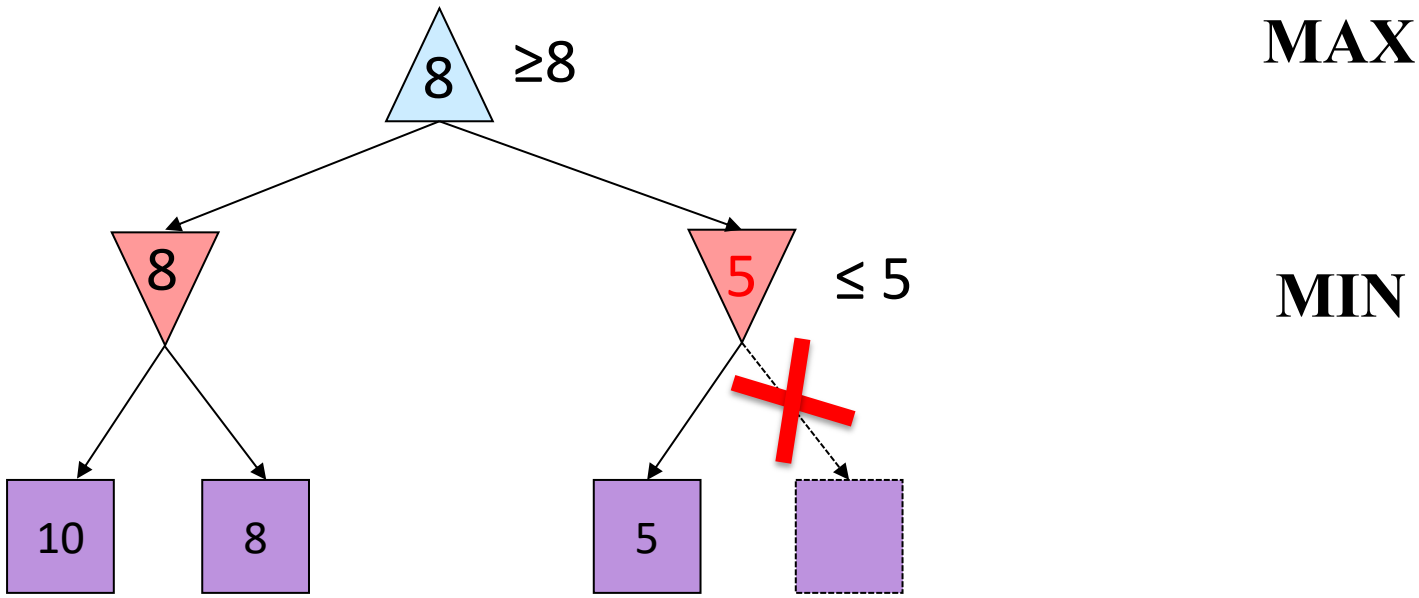
Exemplu



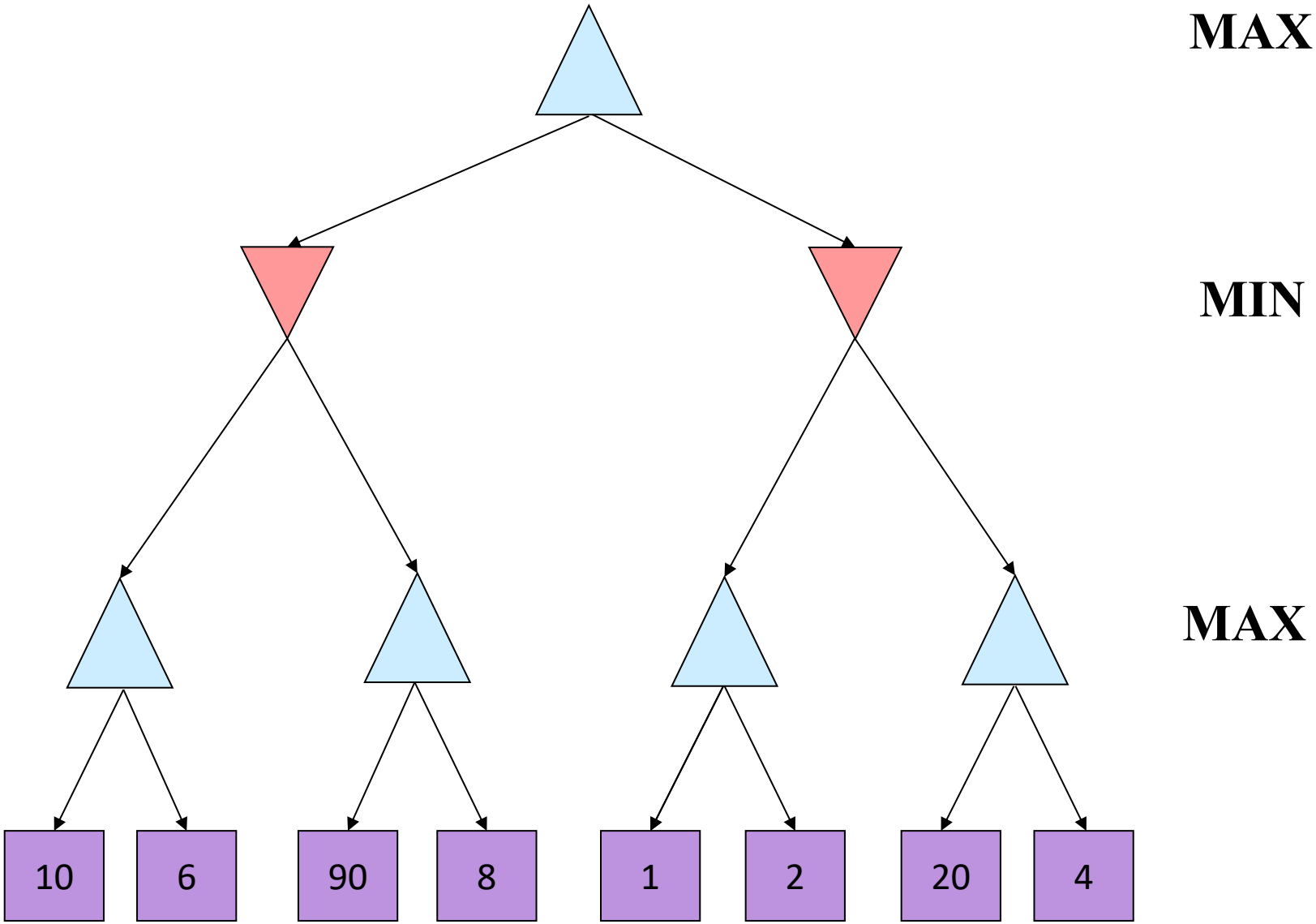
Exemplu



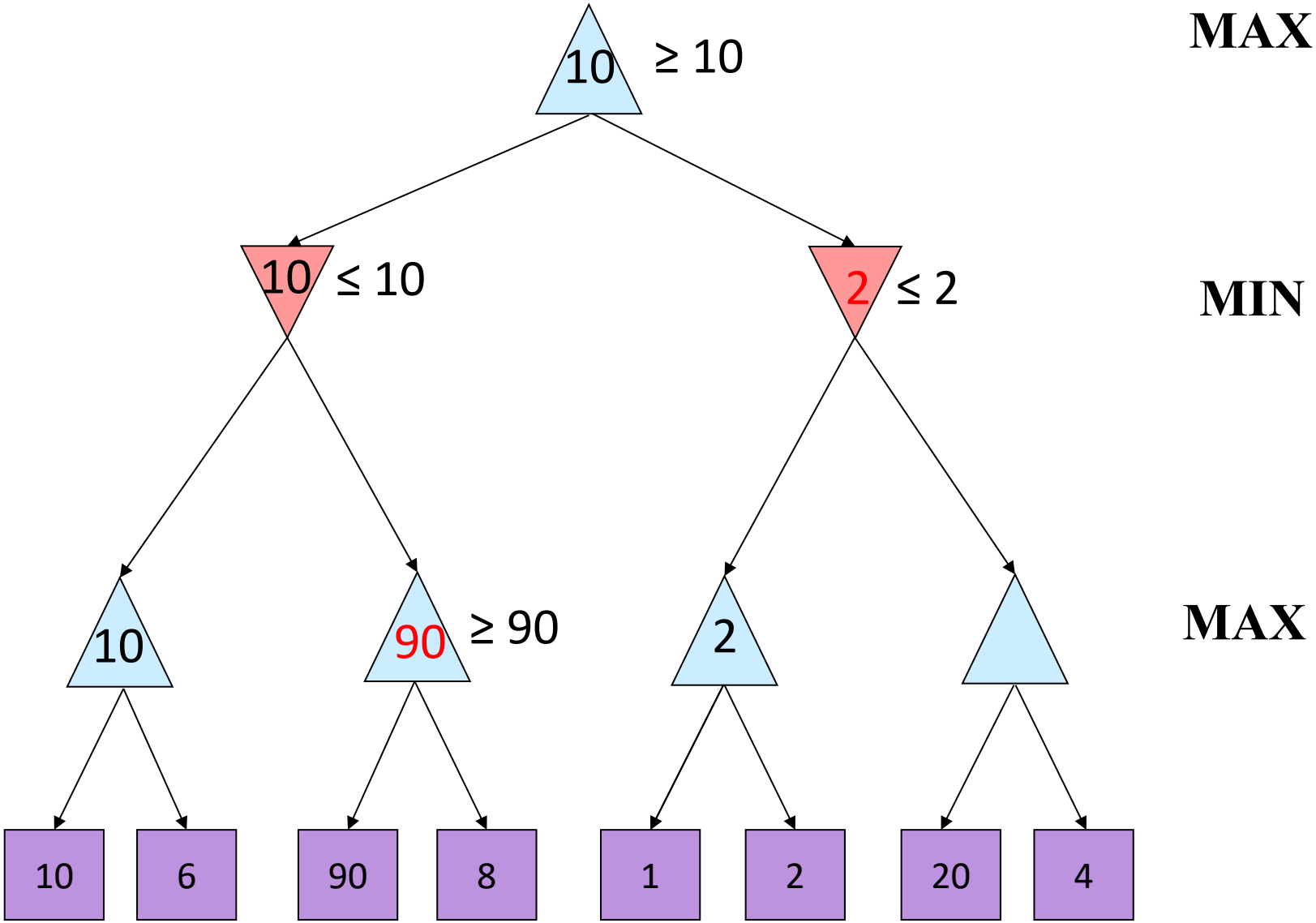
Exemplu



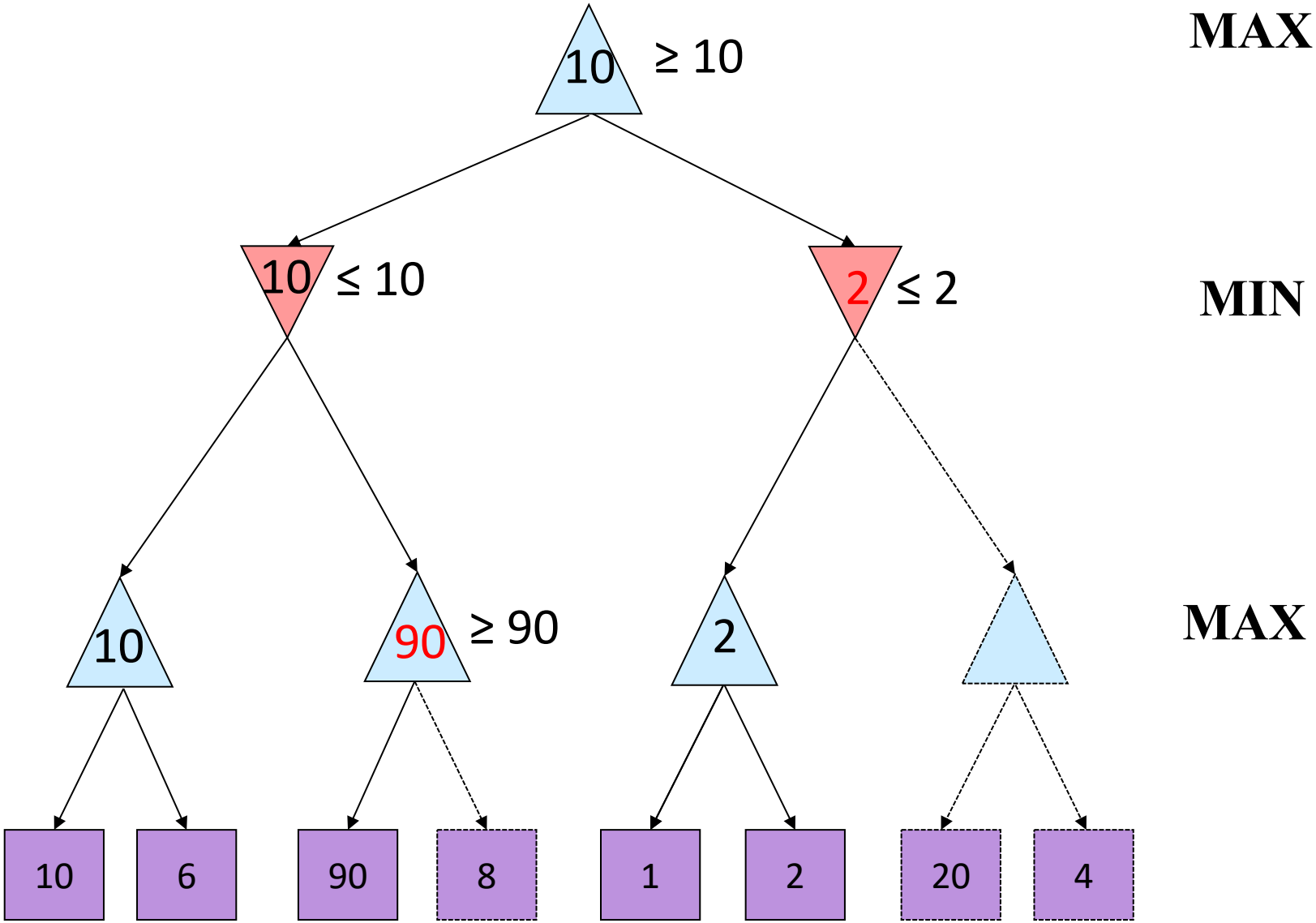
Exemplu



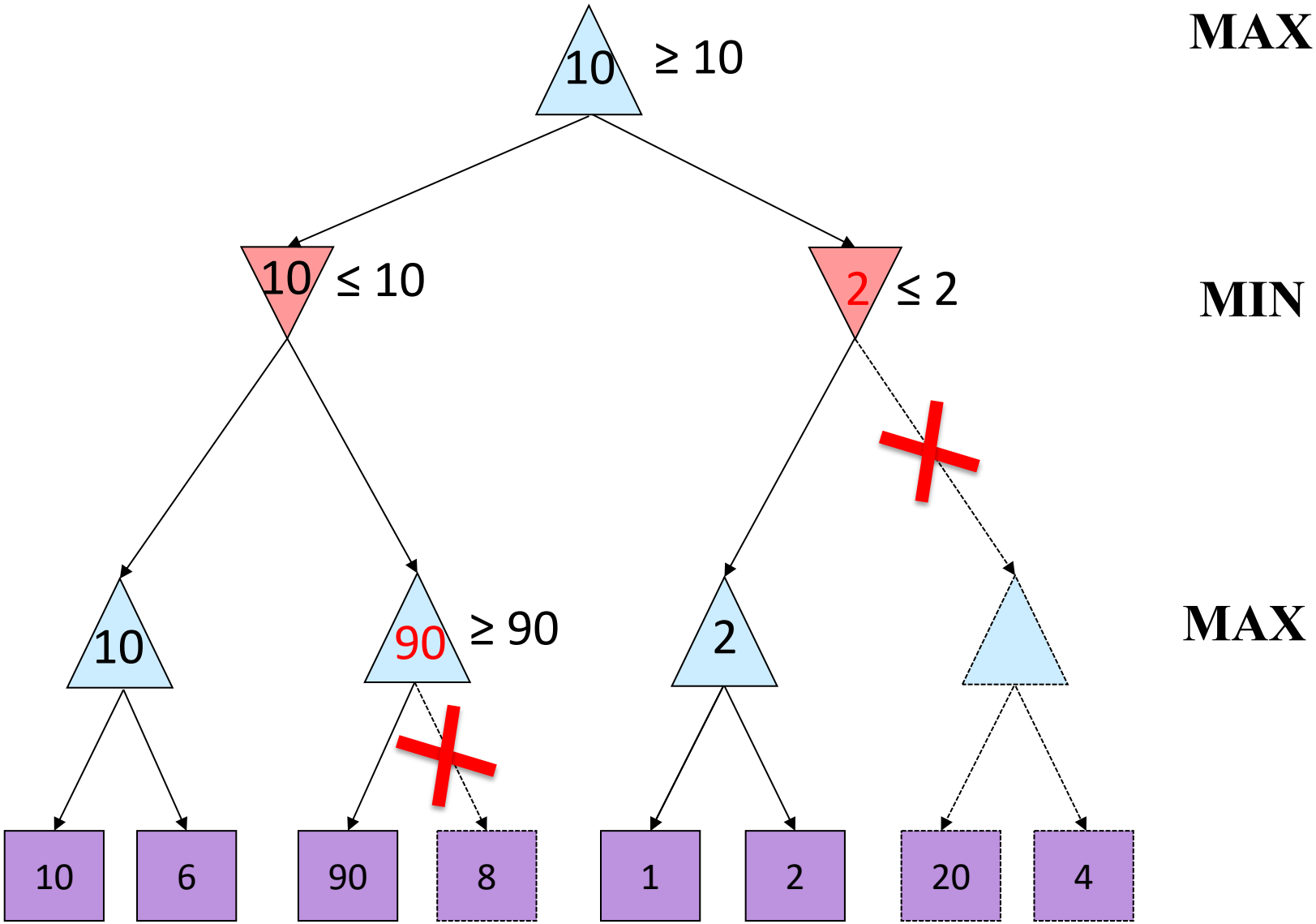
Exemplu



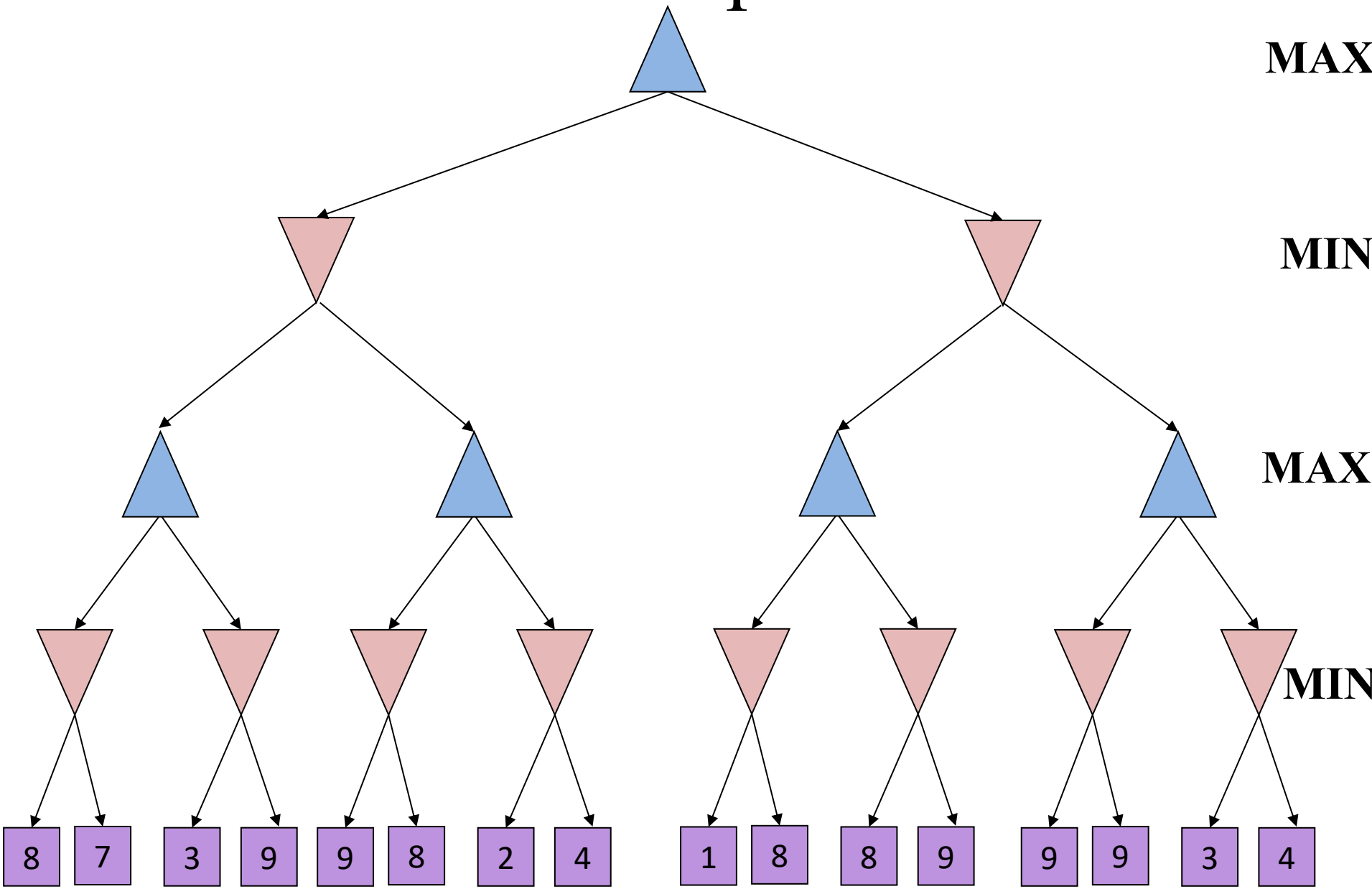
Exemplu



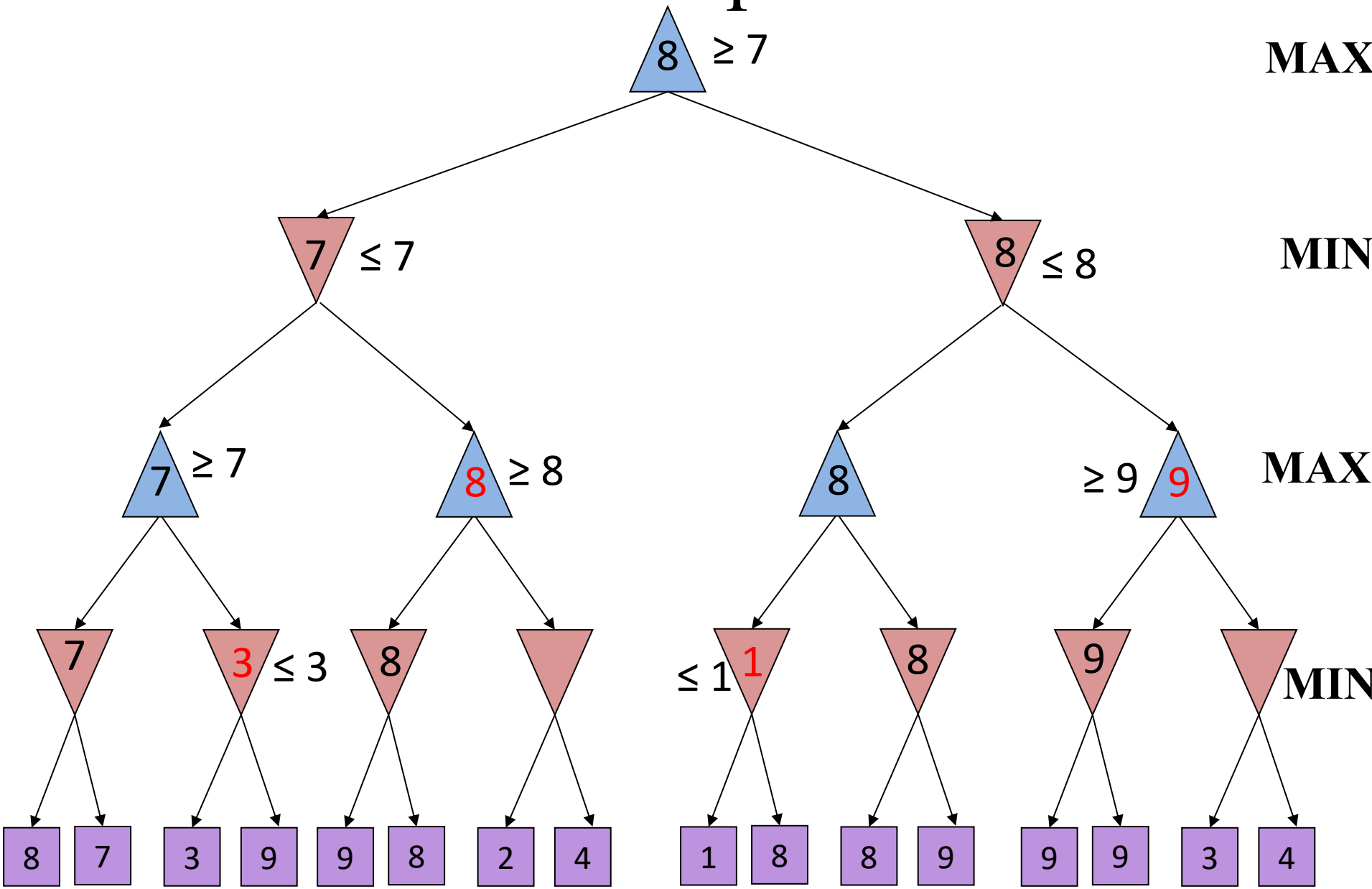
Exemplu



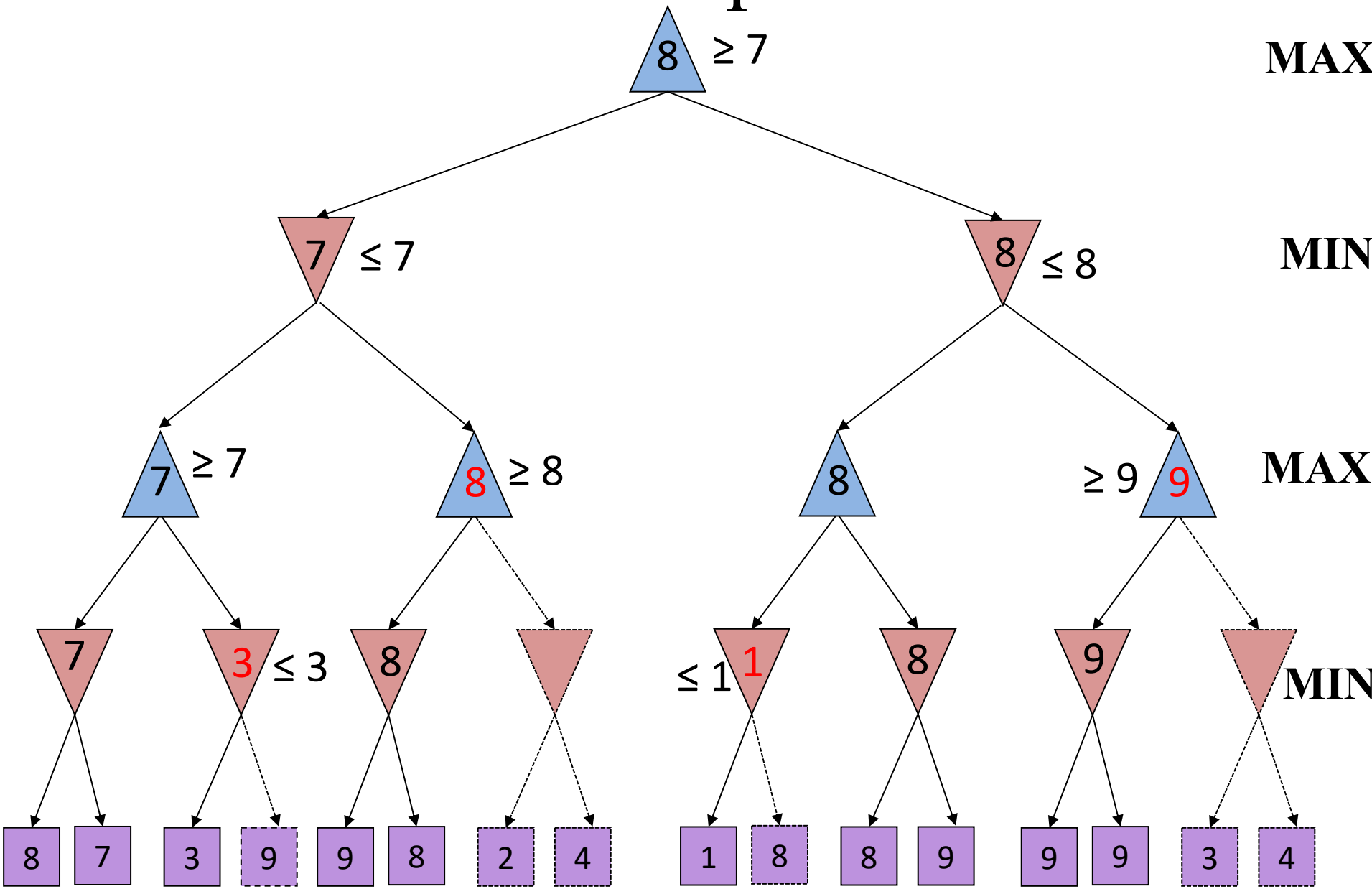
Exemplu



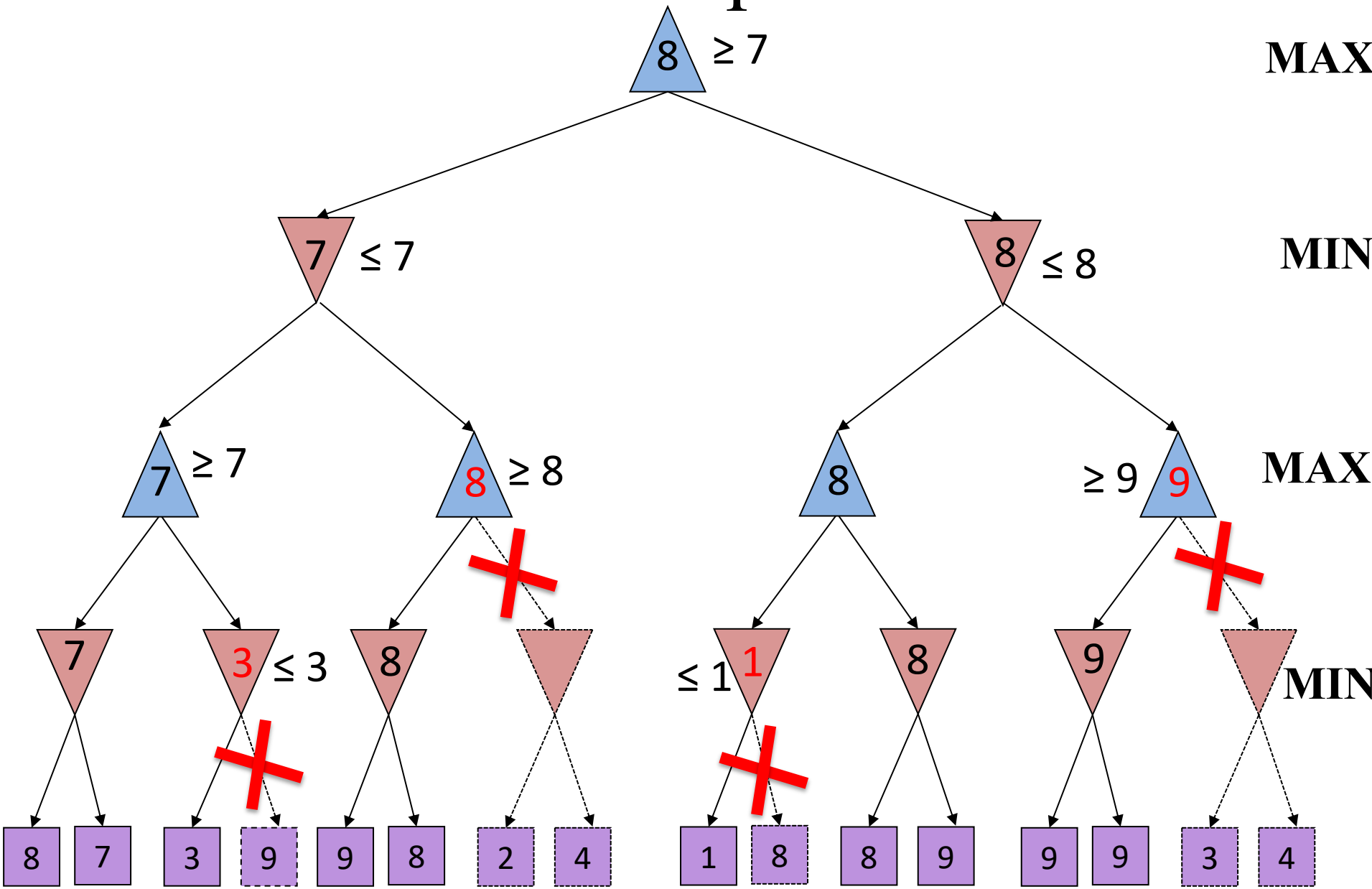
Exemplu



Exemplu



Exemplu



Proprietățile algoritmului Alpha-Beta Pruning

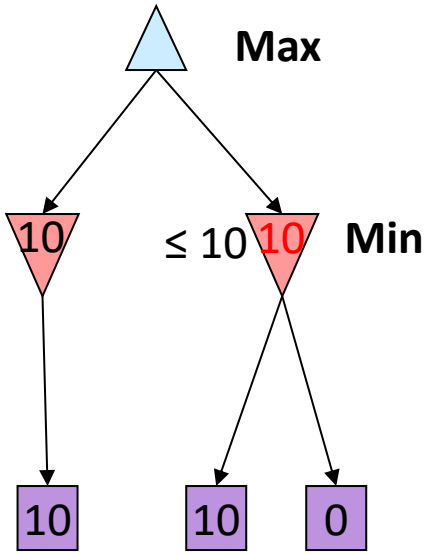
- procesul de retezare nu are niciun efect asupra valorii minimax calculate pentru nodul rădăcină (aceasta nu se schimbă)

- valorile nodurilor intermediare (aproximate) pot fi greșite:
  - important: nodurile fii ale rădăcinii pot avea valori greșite

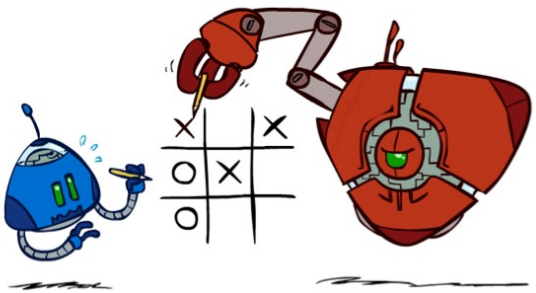
  Implementarea naivă poate conduce la erori

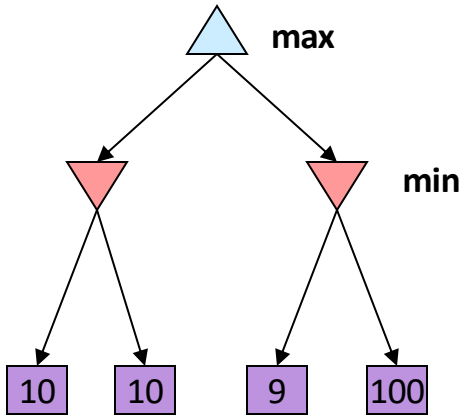
- vizitarea în ordinea bună a nodurilor fii conduce la retezări mai rapide

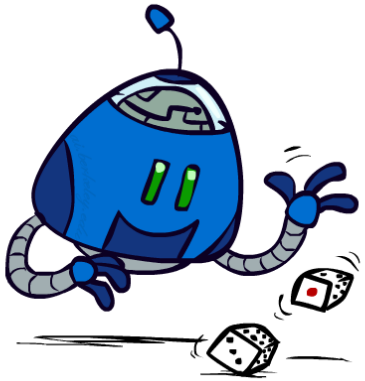
- cu "ordonare perfectă":
  - complexitatea timp poate scădea la $O(b^{m/2})$
  - algoritmul poate rula pe o adâncime dublă
  - căutarea exhaustivă nu este totuși posibilă…



Cazul cel mai defavorabil vs cazul mediu

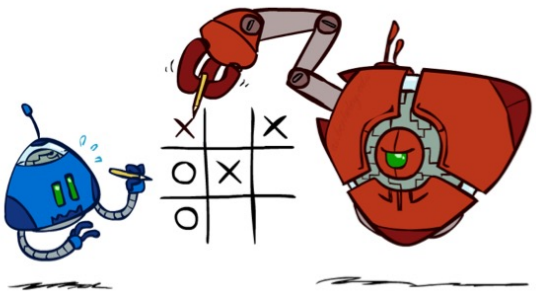


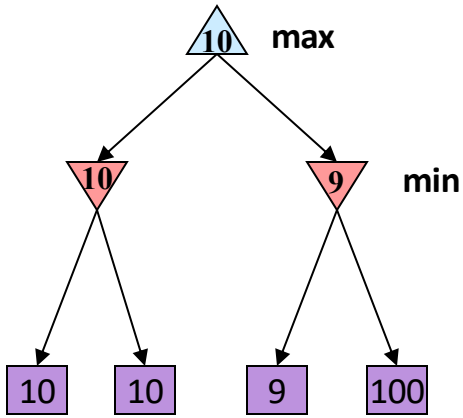


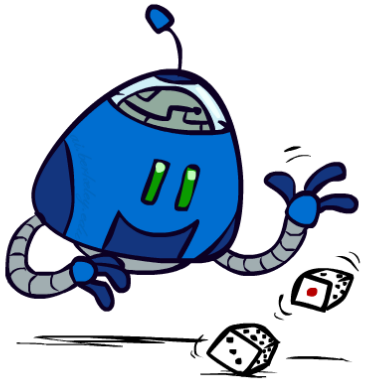


Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Cazul cel mai defavorabil vs cazul mediu

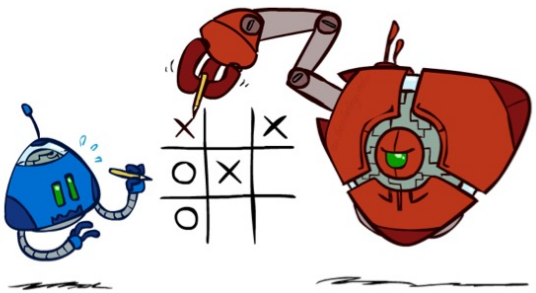


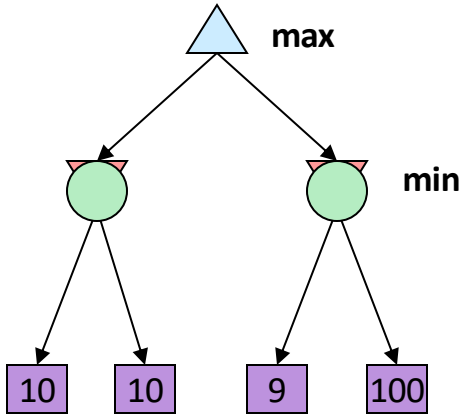


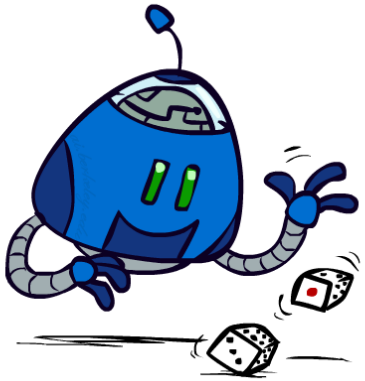


Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Cazul cel mai defavorabil vs cazul mediu

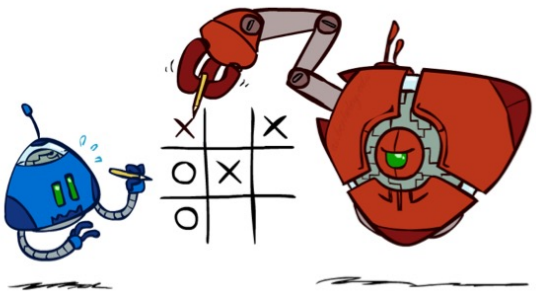


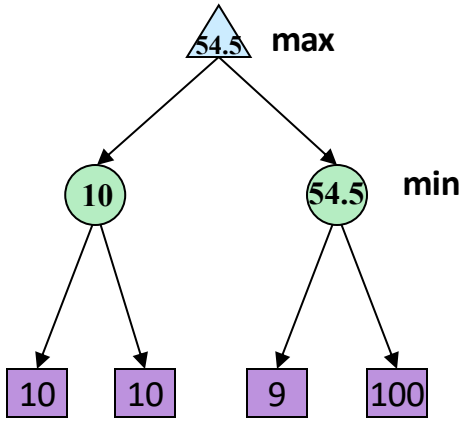


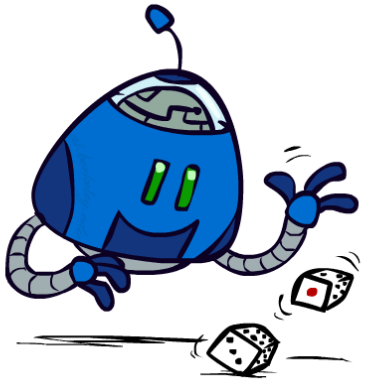


Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Cazul cel mai defavorabil vs cazul mediu



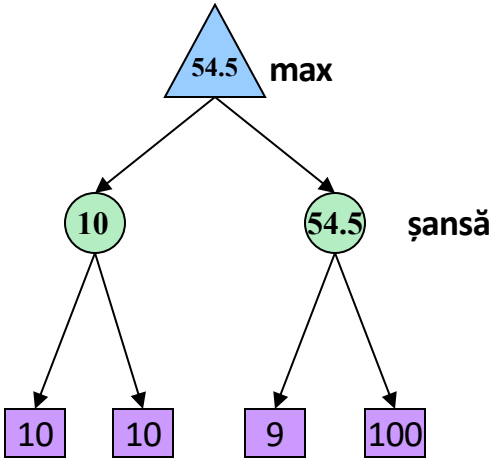




Idee: anumite acțiuni/rezultate sunt controlate de a anumită incertitudine, nu neapărat de un adversar!

Căutare ExpectiMax

- **De ce anumite acțiuni pot avea un rezultat imprevizibil:**
  - incertitudine în formă explicită: aruncarea unui zar
  - adversari impredictibili: pot avea un comportament aleator
  - anumite acțuni pot eșua: un robot care execută anumite acțiuni poate avea roțile care alunecă
- **Valorile stărilor ar trebui să reflecte în aceste situații rezultatele/utilitățile/scorul pe cazul mediu (Expectation), nu pe cazul cel mai defavorabil (Min)**
- **Căutare ExpectiMax:** calculează scorul mediu pe baza unei strategii optime a jucătorului Max
  - noduri de tip max la fel ca în căutarea MiniMax
  - noduri de tip șansă sunt un fel de noduri de tip Min dar rezultatul/utilitatea/scorul este incert, fiind guvernat de o distribuție de probabilitate
  - calculează utilitățile medii (media ponderată a nodurilor fii) – expectation (medie)
  - în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii



Implementare ExpectiMax

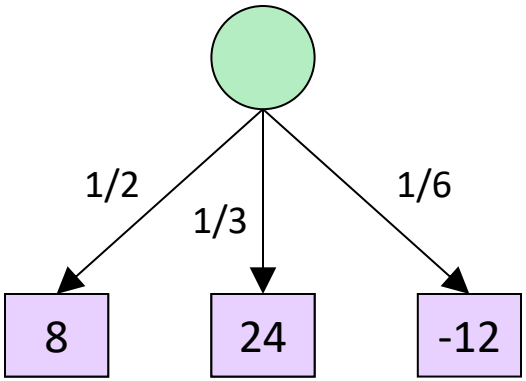
```
def value(stare):
    dacă stare este o stare terminală: returnează utilitatea
        stării
    dacă următorul agent care mută este MAX:
        returnează max-value(stare)
    dacă următorul agent care mută este EXP: returnează
        exp-value(stare)
```

```
def max-value(stare):
    initializează v = -∞
    pentru fiecare succesor al starii:
        v = max(v, value(succesor))
    returnează v
```

```
def exp-value(state):
    initializează v = 0
    pentru fiecare succesor al stării:
        v = v + p(succesor) *
            value(succesor)
    returnează v
```

Implementare ExpectiMax

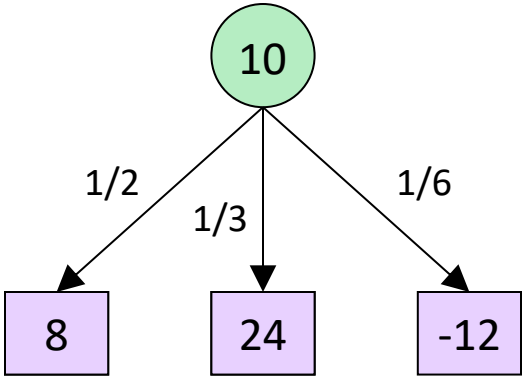
```
def exp-value(state):
    initializează v = 0
    pentru fiecare succesor al stării:
        v = v + p(succesor) *
            value(succesor)
    returnează v
```



$$v = (1/2)\,(8) + (1/3)\,(24) + (1/6)\,(-12) = 10$$

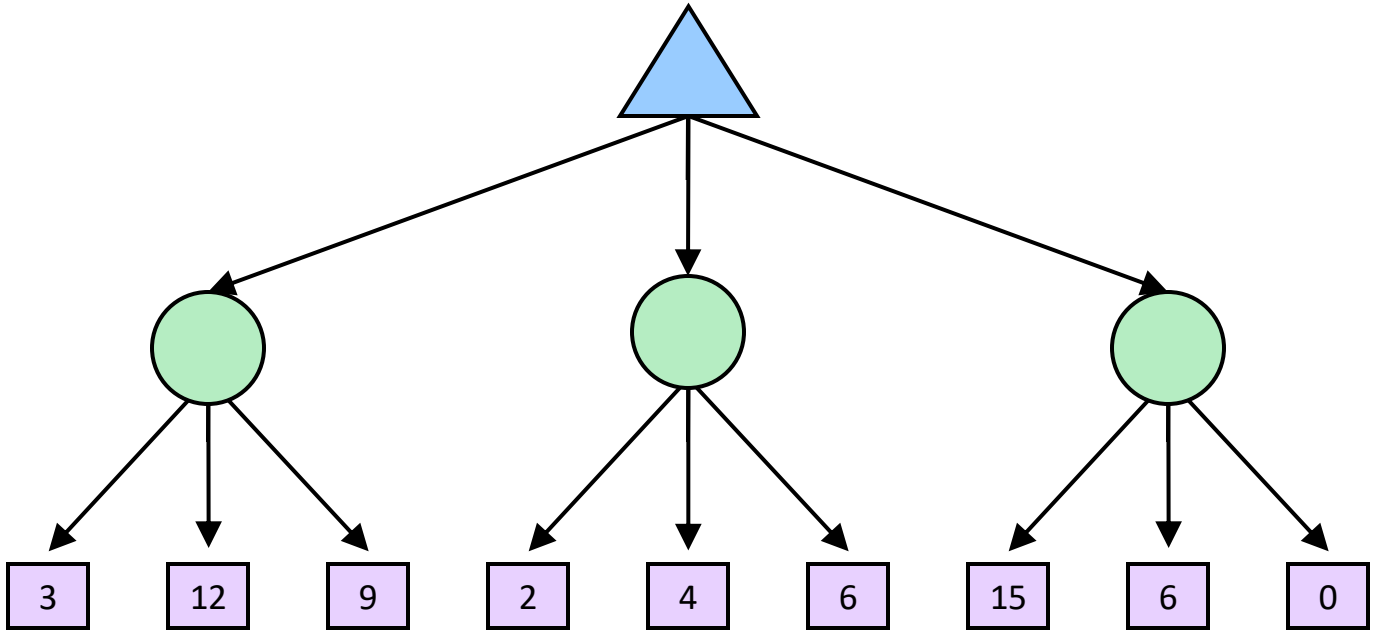
Implementare ExpectiMax

```
def exp-value(state):
    initializează v = 0
    pentru fiecare succesor al stării:
        v = v + p(succesor) *
            value(succesor)
    returnează v
```

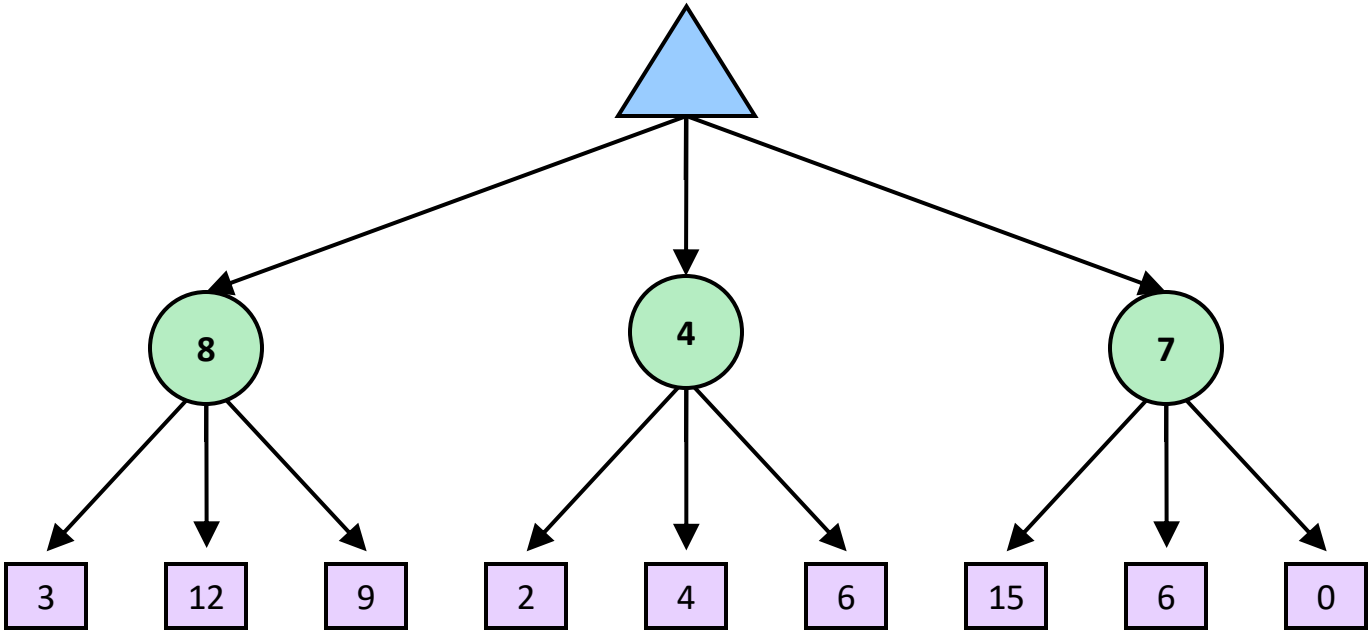


$$v = (1/2)\,(8) + (1/3)\,(24) + (1/6)\,(-12) = 10$$

Exemplu ExpectiMax



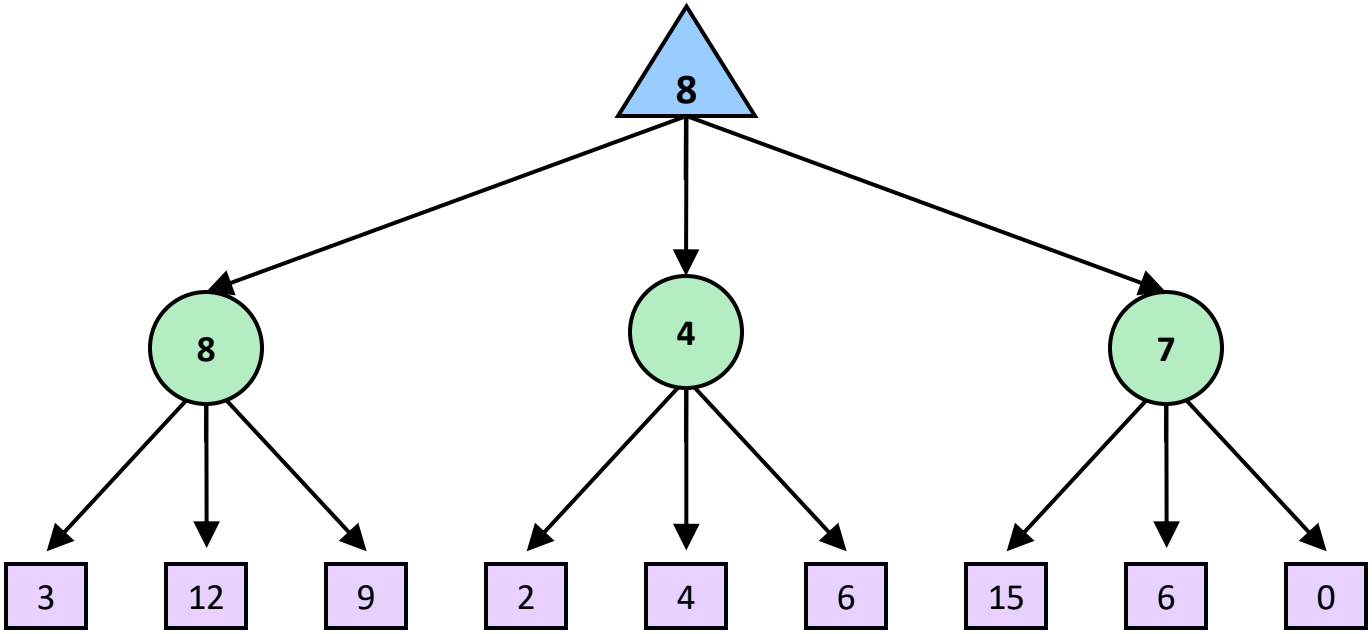
Exemplu ExpectiMax



calculează utilitățile medii (media ponderată a nodurilor fii) – expectation (medie)

în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii

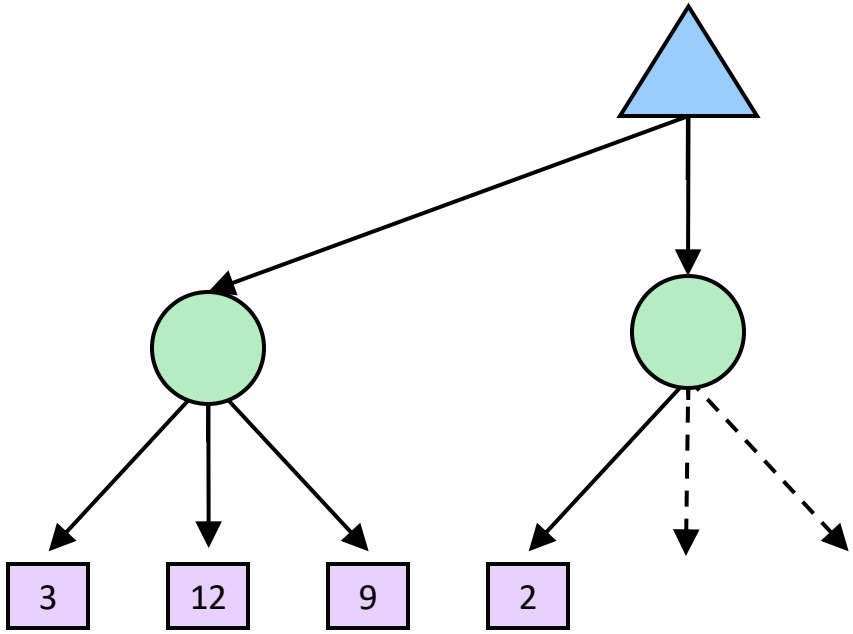
Exemplu ExpectiMax

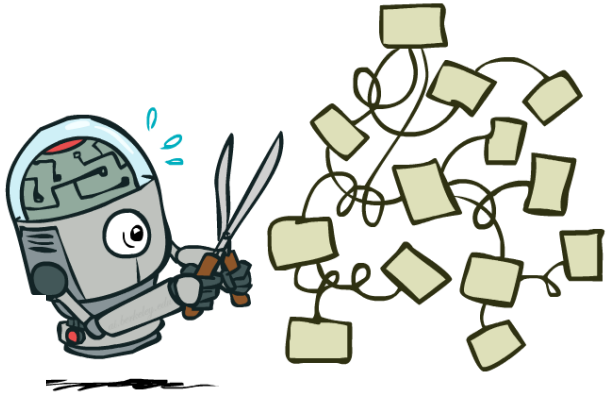


calculează utilitățile medii (media ponderată a nodurilor fii) – expectation (medie)

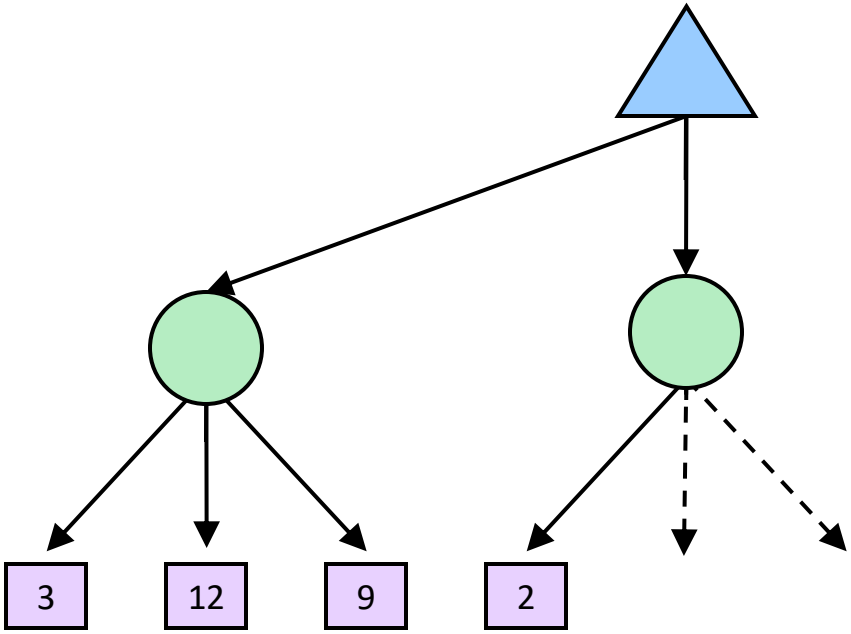
în lipsa unei distribuții de probabilitate asociate se poate calcula media nodurilor fii

Accelerarea algoritmului ExpectiMax?



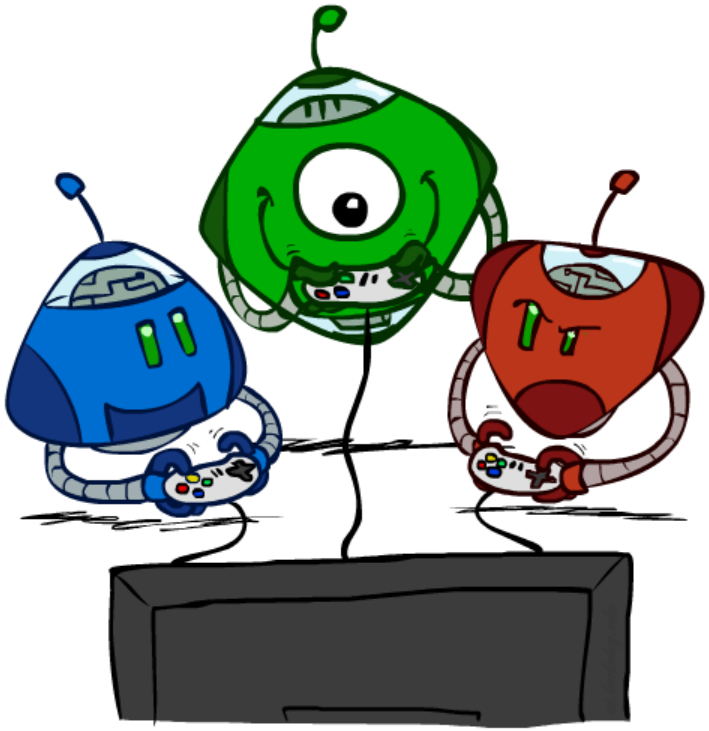


Accelerarea algoritmului ExpectiMax?



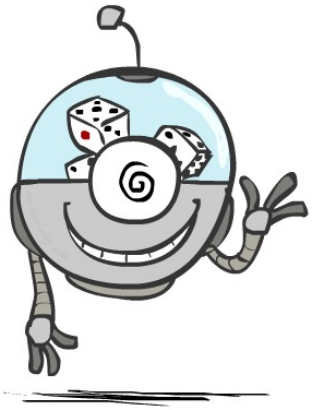
Diferit de algoritmul MiniMax care poate fi accelerat folosind alpha-beta retezare, nu putem accelera algoritmul ExpectiMax întrucât pot exista noduri la dreapta cu o valoarea foarte mare care pot influența utilitatea medie.

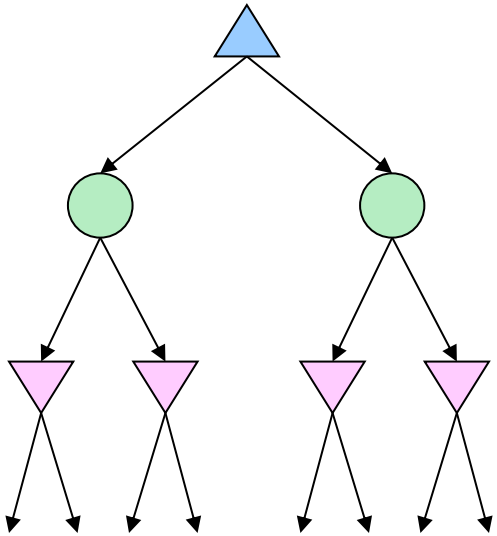
Alte tipuri de jocuri



Jocuri cu noduri mixte

- E.g. Backgammon (table)
- ExpectiMiniMax
  - mediul (environment) este un "agent aleator" care mută (se dă cu zarul) după fiecare agent Min/Max
  - fiecare nod calculează utilitatea medie a fiilor





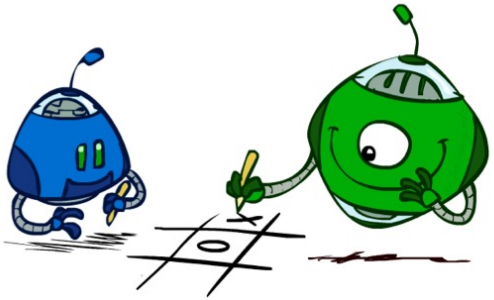
Exemplu: Backgammon (table)

- Aruncările cu zarul cresc factorul de ramificare *b*: 21 de posibilități cu 2 zaruri (1-1, 1-2, …, 6-6)
  - Backgammon/Table ≈ 20 mutări legale
  - factor total de ramificare: $b \approx 21 \times 20 \approx 400$
  - pentru o căutare de d mutări, număr de stări = $(21 \times 20)^d$

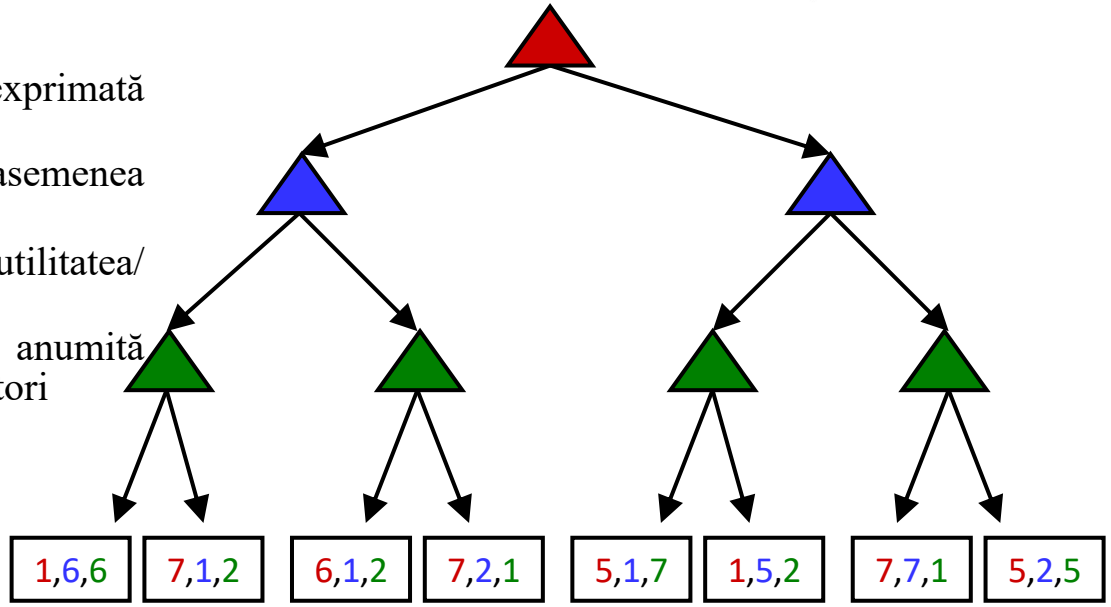
- Pe măsură ce adîncimea crește, probabilitatea de a ajunge într-un anumit nod se micșorează
  - utilitatea unei căutări se diminuează
  - căutare limitată în adâncime nu pare așa de rea
  - accelerarea unei asemenea abordări este greu de realizat

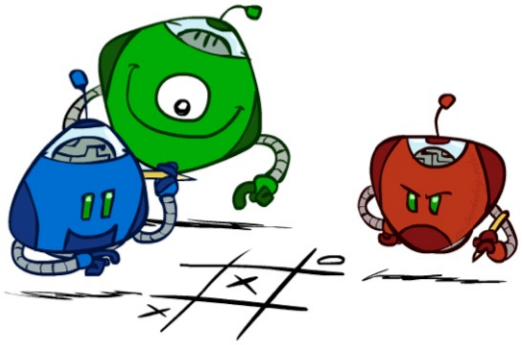
- Soluția în practică (TD-Gammon)
  - căutare superficială (adâncime mică)
  - funcție de evaluare puternică
  - reinforcement learning
  - performanță: nivel de campion mondial

Utilități multi-agent



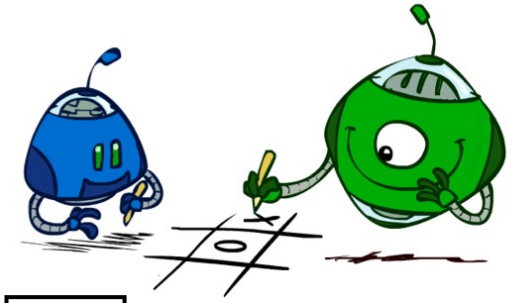
- Dacă jocul nu este cu sumă zero sau are mai mult de doi jucători?
- Generalizare pentru MiniMax:
  - nodurile terminale au utilitatea exprimată printr-un tuplu
  - valorile nodurilor interne sunt de asemenea tupluri de utilitate
  - fiecare jucător maximizează utilitatea/ componenta proprie
  - poate apărea în mod dinamic o anumită cooperare sau competiție între jucători

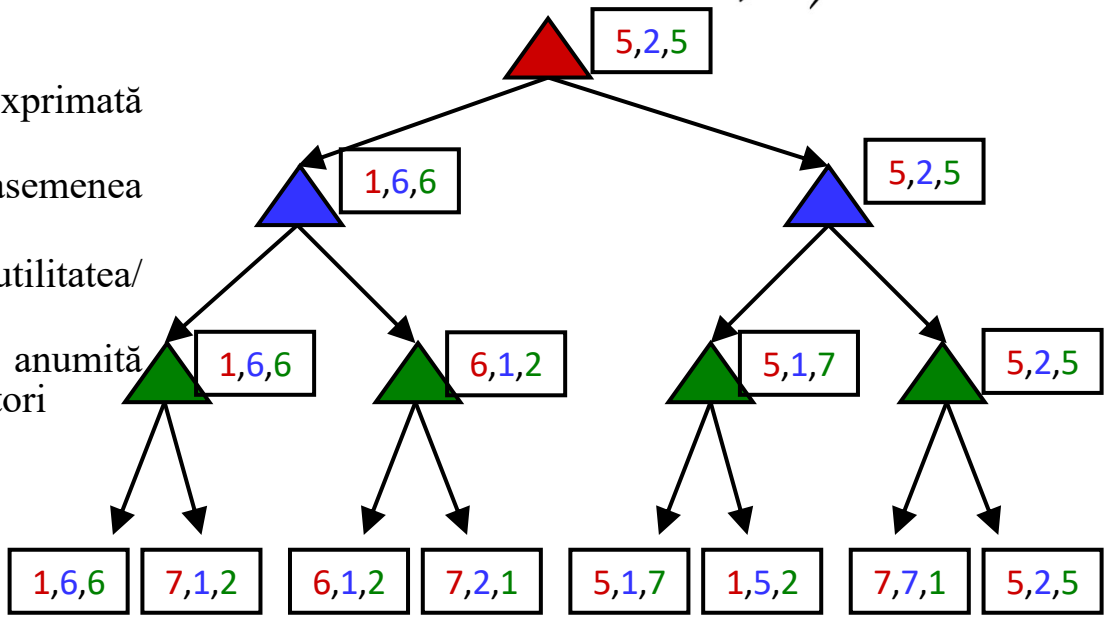


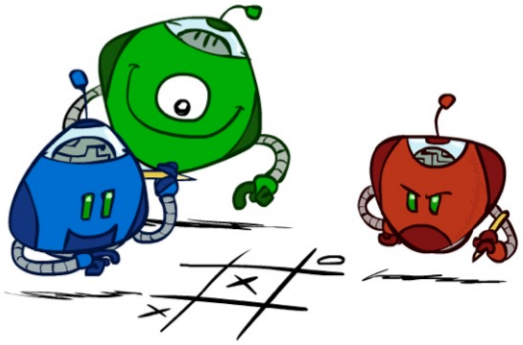


Utilități multi-agent

- Dacă jocul nu este cu sumă zero sau are mai mult de doi jucători?
- Generalizare pentru MiniMax:
  - nodurile terminale au utilitatea exprimată printr-un tuplu
  - valorile nodurilor interne sunt de asemenea tupluri de utilitate
  - fiecare jucător maximizează utilitatea/ componenta proprie
  - poate apărea în mod dinamic o anumită cooperare sau competiție între jucători







Probleme de satisfacere de constrângeri

- probleme standard de căutare:
  - spațiul stărilor, stare inițială, stare finală, acțiuni, costul unei acțiuni
  - test dacă o stare este stare-scop poate fi orice funcție de stări
  - funcția succesor poate fi orice

- căutarea unei soluții: secvență de acțiuni
  - drumul până la nodul scop este important
  - drumurile au costuri diferite, au adâncimi diferite
  - euristicile specifice problemelor ajută la găsirea soluției

- de cele mai multe ori putem folosi o reprezentare pentru fiecare stare sub forma unor variabile, fiecare luând anumite valori
  - soluția se obține când toate variabilele au asignată o valoare care respectă toate constrângerile problemei
  - drumurile au aceeași adâncime (același număr de variabile de asignat)
  - obținem probleme de satisfacere a constrângerilor (PSC)

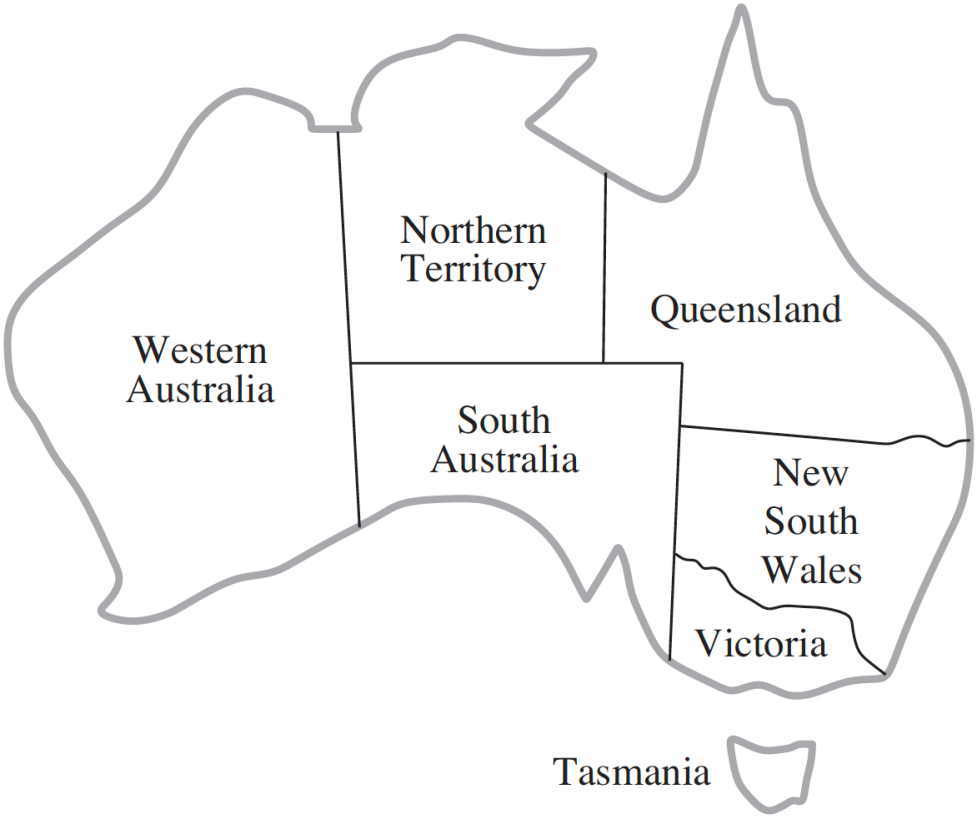
Probleme de satisfacere a constrângerilor -definiție-

O problemă de satisfacere a constrângerilor (PSC) are **trei componente**:

- X = o mulțime de variabile, $X = \{X_1, X_2, \ldots, X_n\}$
- D = o mulțime de domenii, $D = \{D_1, D_2, \ldots, D_n\}$, unde fiecare domeniu $D_i = \{v_{i1}, v_{i2}, \ldots, v_{ik_i}\}$ corespunde valorilor posibile ale variabilei X_i
- C = o mulțime de constrângeri, $C = \{C_1, C_2, \ldots, C_n\}$, unde fiecare constrângere C_i definește o relație pe care valorile anumitor variabile trebuie să o îndeplinească
  - constrângerile pot fi exprimate implicit sau explicit

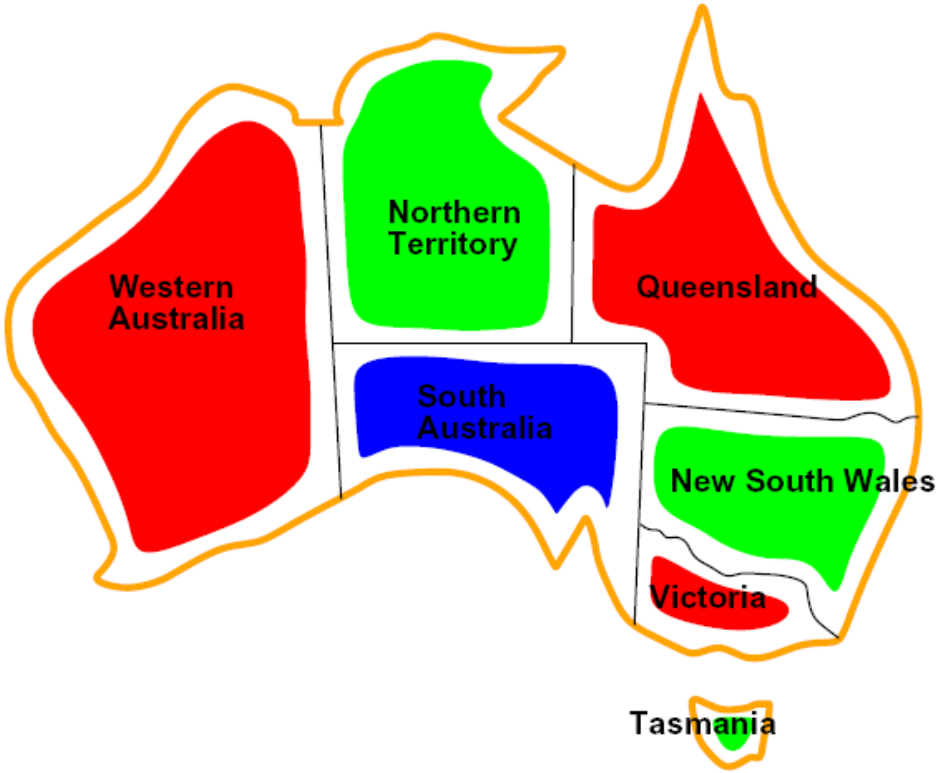
Exemplu – problema colorării unei hărți

Problema de a colora o harta cu un număr de 3 culori (**roșu**, **verde**, **albastru**) astfel încât regiunile adiacente să aibă culori diferite

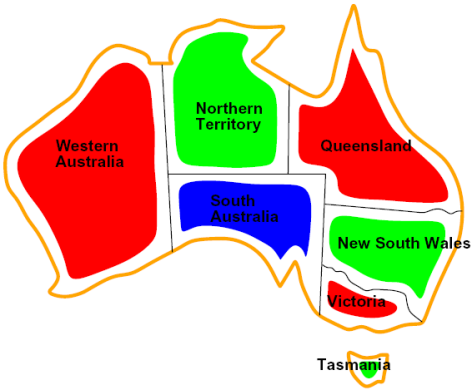


Exemplu – problema colorării unei hărți

O posibilă soluție (există mai multe)



Exemplu – problema colorării unei hărți



- Variabile: X = {WA, NT, Q, NSW, V, SA, T}
- Domenii: D = D_i = {**roșu**, **verde**, **albastru**}
- Constrângeri: regiuni adiacente trebuie să aibă culori diferite

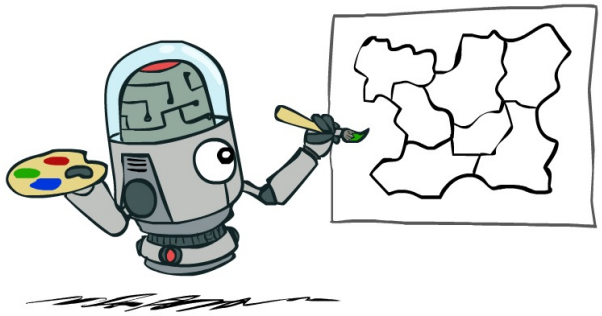
  C = {C_1, C_2, C_3, C_4, C_5, C_6, C_7, C_8, C_9}

  C_1 sub formă implicită: WA ≠ NT

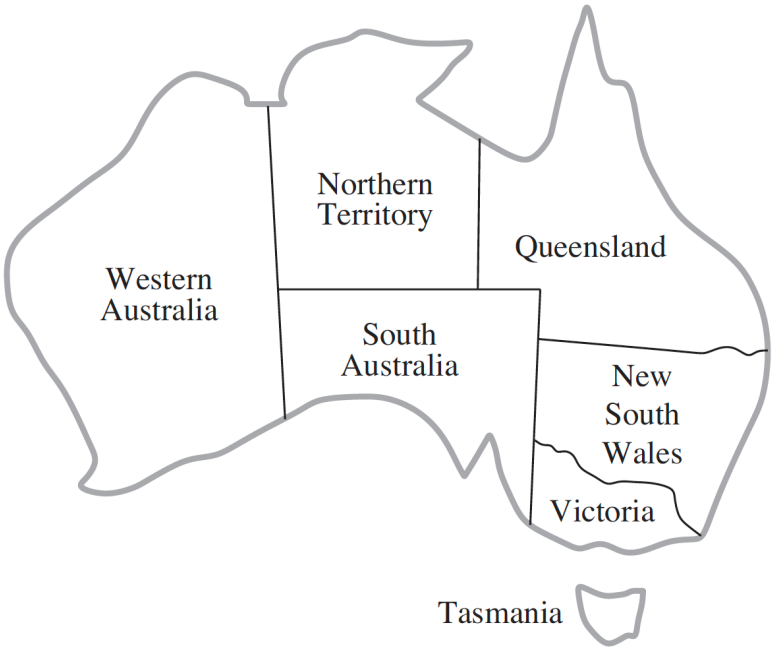
  C_1 sub formă explicită: (WA, NT) ∈ {(**roșu**, **verde**), (**roșu**, **albastru**), (**verde**, **albastru**), (**verde**, **roșu**), (**albastru**, **roșu**), (**albastru**, **verde**)}

- Soluțiile sunt reprezentate de asignarea unor valori varibilelor care satisfac toate constrângerile:

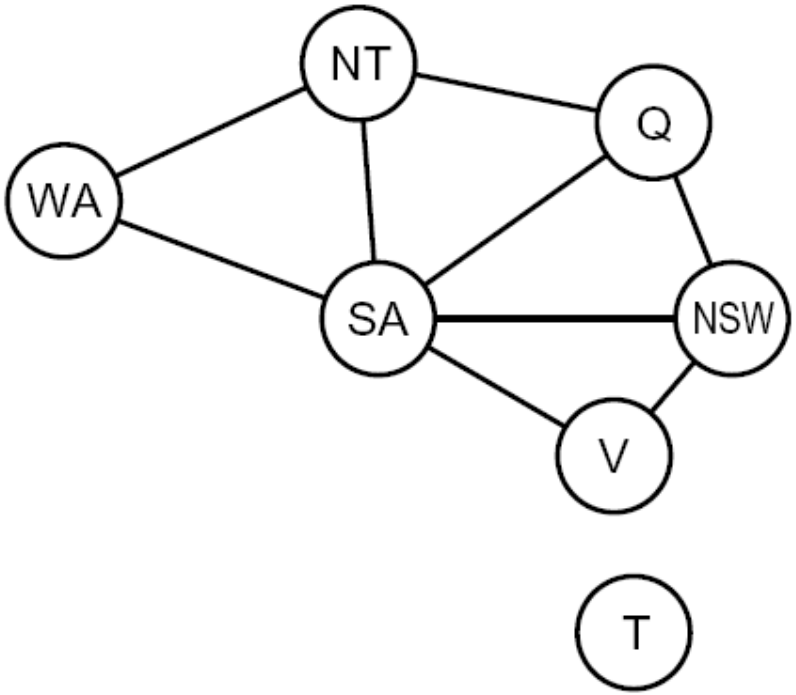
{WA = **roșu** , NT = **verde**, Q = **roșu**, NSW = **verde**, V = **roșu**, SA = **albastru**, T = **verde**}



Vizualizarea unei PSC sub forma unui graf de constrângeri



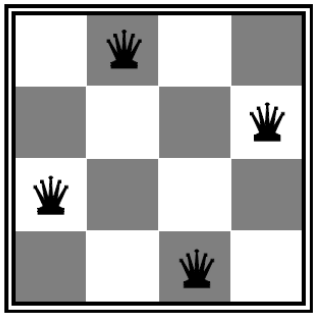
Problemă inițială

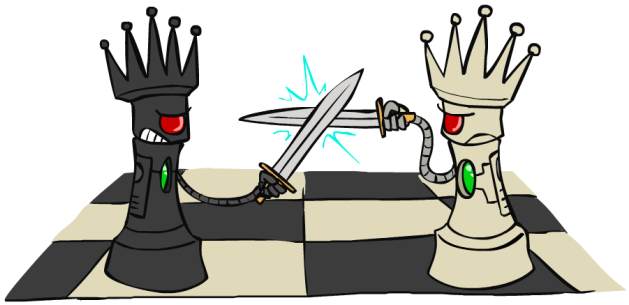


Problemă inițială reprezentată sub forma unui graf de constrângeri

Exemplu – problema celor N regine

- Formulare 1:
  - variabile: X_{ij}
  - domenii: $\{0, 1\}$
  - constrângeri:





$\forall i, j, k \;\; (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$ Piese de pe aceeași linie

$\forall i, j, k \;\; (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$ Piese de pe aceeași coloană

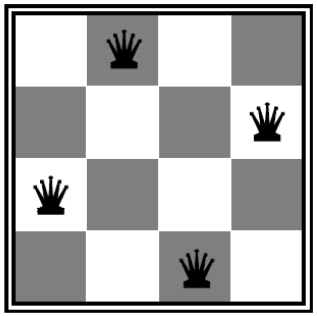
$\forall i, j, k \;\; (X_{ij}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$

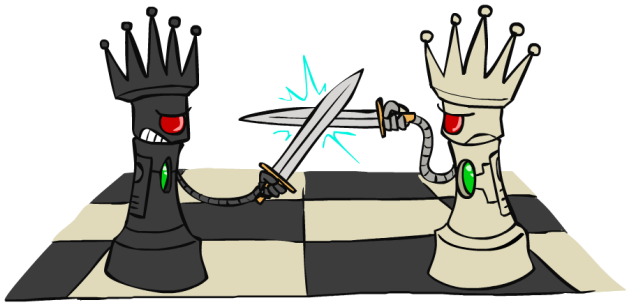
$\forall i, j, k \;\; (X_{ij}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$ Piese de pe aceeași diagonală

  - pe baza acestor constrângeri puteți să găsiți o soluție?
  - soluția banală $X_{ij} = 0$

Exemplu – problema celor N regine

- Formulare 1:
  - variabile: X_{ij}
  - domenii: $\{0, 1\}$
  - constrângeri:





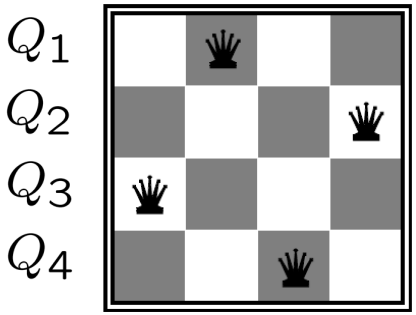
$$\forall i, j, k \;\; (X_{ij}, X_{ik}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \;\; (X_{ij}, X_{kj}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \;\; (X_{ij}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$$
$$\forall i, j, k \;\; (X_{ij}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$$

$$\sum_{i,j} X_{ij} = N$$

Exemplu – problema celor N regine

- Formulare 2:
  - variabile: Q_k
  - domenii (coloane): $\{1, 2, 3\ldots, N\}$
  - constrângeri:
    - implicite: orice două regine Q_i și Q_j nu se atacă ($i \neq j$)
    - explicite: $(Q_1, Q_2) \in \{(1,3), (1,4), (2,4), (3,1), (4,1), (4,2)\}$

      . . .



Exemplu – Criptaritmetica

$$\begin{array}{r} \text{T W O} \\ + \ \text{T W O} \\ \hline \text{F O U R} \end{array}$$

Exemplu – Criptaritmetica

- Variabile:

  $F\ T\ U\ W\ R\ O\ X_1\ X_2\ X_3$

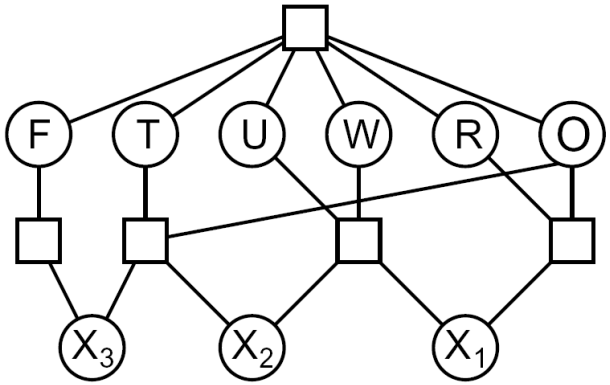
$$\begin{array}{r} \mathsf{T\ W\ O} \\ +\ \mathsf{T\ W\ O} \\ \hline \mathsf{F\ O\ U\ R} \end{array}$$

- Domenii:

  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

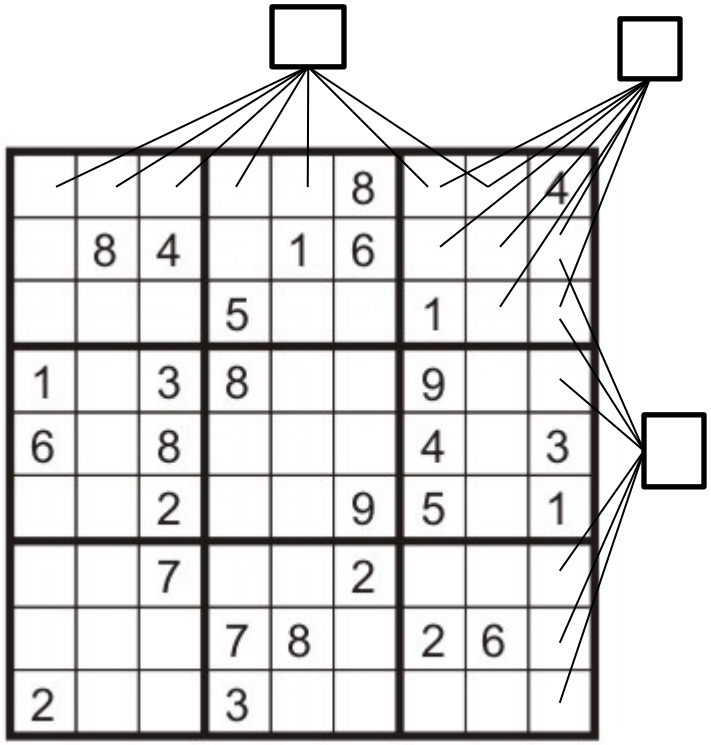
- Constrângeri:

$$\mathsf{alldiff}(F, T, U, W, R, O)$$
$$O + O = R + 10 \cdot X_1$$
$$W + W + X_1 = U + 10 \cdot X_2$$
$$T + T + X_2 = O + 10 \cdot X_3$$
$$F = X_3$$



Problemă inițială reprezentată sub forma unui hiper-graf de constrângeri
Apar constrângeri de aritate 2, 3, 4, 6.

Exemplu – Sudoku



- Variabile:
  - fiecare pătrat gol
- Domenii:
  - {1,2,…,9}
- Constrângeri:

  Pe fiecare coloană sunt 9 cifre diferite

  Pe fiecare linie sunt 9 cifre diferite

  În fiecare pătrat 3 × 3 sunt 9 cifre distincte

  (sau putem adăuga multe inegalități de tipul A1 ≠ A2, A1 ≠ A3, etc.)

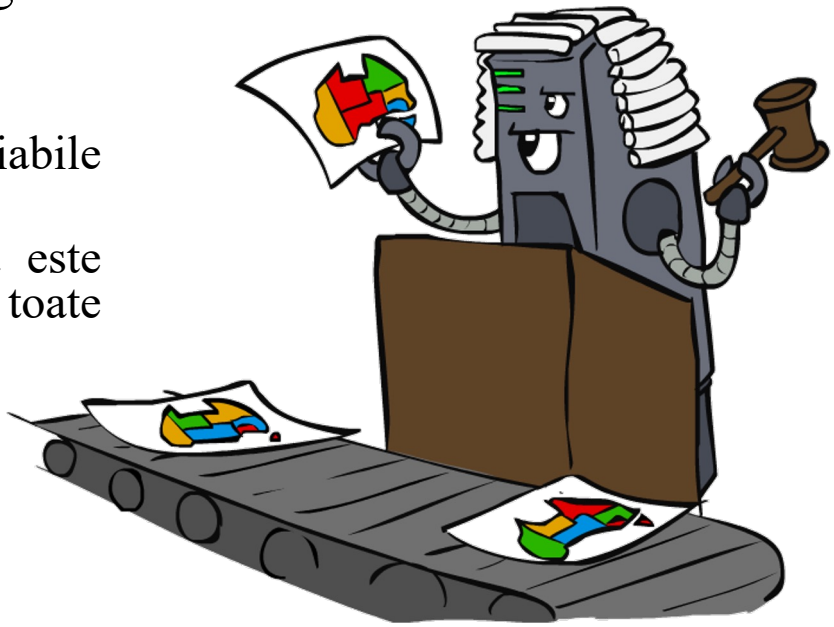
Zebra puzzle

6.7 Consider the following logic puzzle: In five houses, each with a different color, live five persons of different nationalities, each of whom prefers a different brand of candy, a different drink, and a different pet. Given the following facts, the questions to answer are "Where does the zebra live, and in which house do they drink water?"

The Englishman lives in the red house.
The Spaniard owns the dog.
The Norwegian lives in the first house on the left.
The green house is immediately to the right of the ivory house.
The man who eats Hershey bars lives in the house next to the man with the fox.
Kit Kats are eaten in the yellow house.
The Norwegian lives next to the blue house.
The Smarties eater owns snails.
The Snickers eater drinks orange juice.
The Ukrainian drinks tea.
The Japanese eats Milky Ways.
Kit Kats are eaten in a house next to the house where the horse is kept.
Coffee is drunk in the green house.
Milk is drunk in the middle house.

Formularea standard a unei PSC pentru căutare

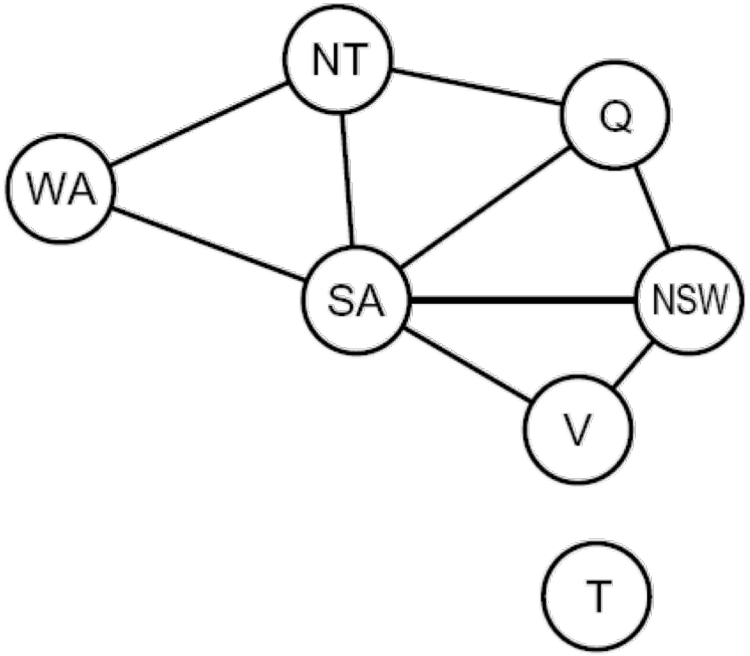
- stările sunt definite de valorile asignate până la momentul curent (fac parte din asignări parțiale)
  - stare inițială : asignarea vidă, {}
  - funcția succesor: asignez o valoare unei variabile neasignate
  - testul de stare-scop: dacă asignarea curentă este completă și consistentă (satisface toate constrângerile)



- pornim cu o soluție simplă, naivă pe care o vom îmbunătăți

Strategii de căutare

- Care este rezultatul căutării în lățime?

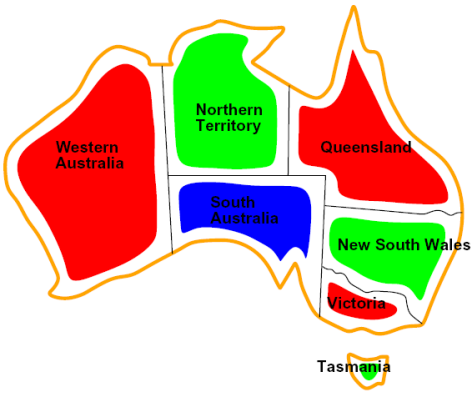


Strategii de căutare

- Care este rezultatul căutării în lățime?

{}

{WA=g} *{WA=r}* … *{NT=g}* …



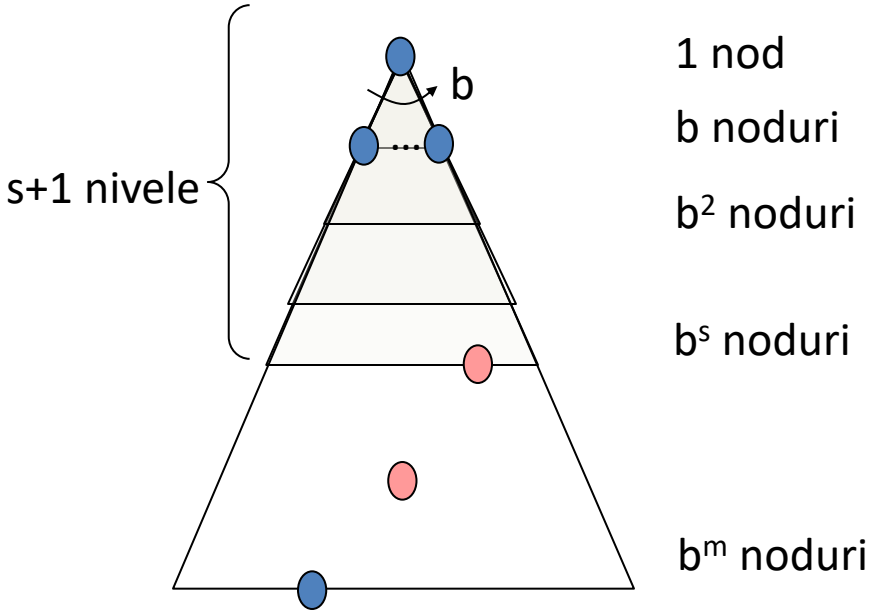
Cursul 2: Proprietățile căutării în lățime

- Ce fel de noduri expandează BF?
  - procesează toate nodurile deasupra soluției de adâncime minimă
  - fie s = adâncimea minimă a unui nod scop
  - complexitate timp $O(b^s)$
  - verifică dacă un nod succesor generat este nod scop - reduce complexitatea timp de la $O(b^{s+1})$ la $O(b^s)$

- Cât de mult spațiu necesită BF?
  - este spațiul pentru memorarea frontierei $O(b^s)$ + memorarea nodurilor explorate $O(b^{s-1})$
  - limită superioară dată de ultimul nivel
  - complexitate $O(b^s)$

- Completitudine?
  - dacă o soluție există atunci s este finit, deci DA!

- Optimalitate?
  - în general NU!
  - uneori DA (spre exemplu dacă costurile fiecărei muchii sunt egale)



Strategii de căutare

- Care este rezultatul căutării în lățime?
  - Generează întreg arborele de căutare, soluțiile sunt pe ultimul nivel
  - La nivelul 1, factor de ramificare $b = n \times d$ (avem n variabile și fiecare poate lua d valori)
  - La nivelul 2, factor de ramificare $b = (n-1) \times d$
  - Avem n nivele (= numărul de variabile)
  - Numărul de frunze este $n! \times d^n$, mult mai mare decât d^n

Căutare locală

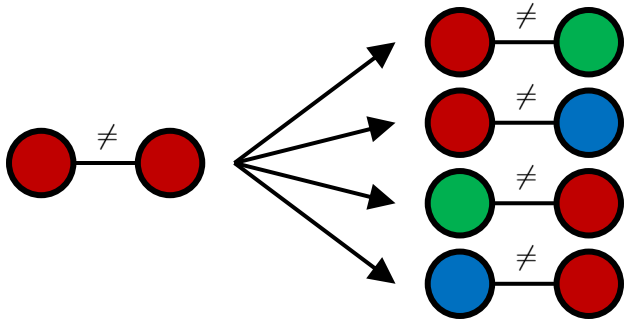
Căutare locală

- în multe probleme de căutare nu contează drumul către o soluție, ci numai starea finală = soluția (problema celor N regine, problema colorării hărților)

- arborii de căutare păstrează în frontieră altenative neexplorate (asigură completitudinea)

- **căutare locală**: îmbunătățește o stare actuală până când nu mai poți (nu există conceptul de frontieră!)

- funcția succesor: schimbări locale



- în general, o asemenea abordare este mult mai rapidă și mai eficientă din punct de vedere al memoriei (dar este incompletă și suboptimală)

Hill climbing

- Idee generală:
  - pornește de oriunde (stare actuală = asignare aleatoare)
  - repetă: îmbunătățește starea actuală căutând local cea mai bună stare într-o vecinătate
  - dacă nu există vecini mai buni te oprești

- Dezavantaje ale unei asemenea abordări?
  - nu există garanția de completitudine
  - nu există garanția de optimalitate

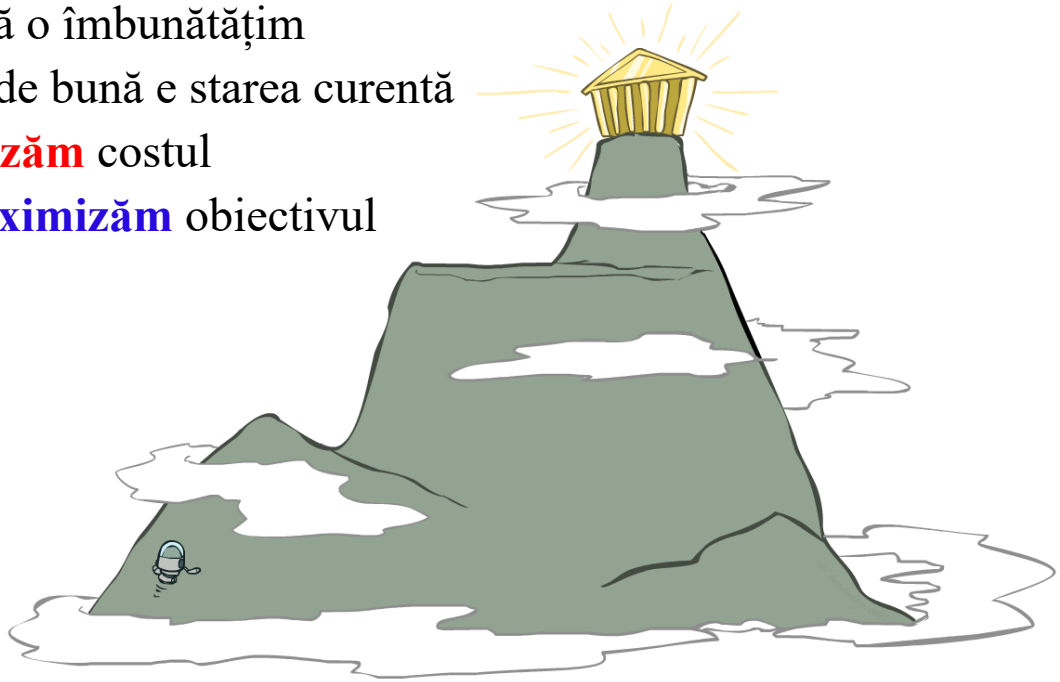
- Avantaje
  - o putem aplica pentru orice problemă
  - foarte rapid și eficient dpdv al memoriei

- Se mai numește *căutare prin ascensiune*

Hill climbing

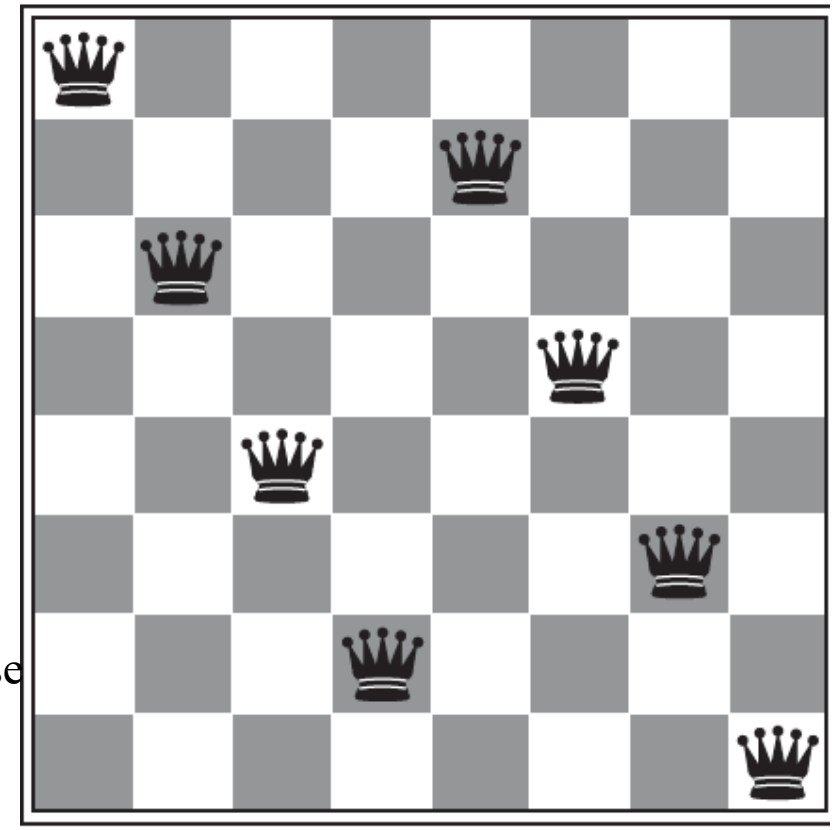
- Idee generală:
  - pornește de oriunde (stare actuală = asignare aleatoare)
  - repetă: **îmbunătățește starea actuală** căutând local cea mai bună stare într-o vecinătate
  - dacă nu există vecini mai buni te oprești

- Îmbunătățirea unei stări?
  - există o funcție pe care vrem să o îmbunătățim
  - funcția cuantifică numeric cât de bună e starea curentă
  - **funcție cost** – vrem să **minimizăm** costul
  - **funcție obiectiv** – vrem să **maximizăm** obiectivul



Hill climbing pentru problema celor 8 regine

- **căutare locală**: îmbunătățește o stare actuală până când nu mai poți
- o stare actuală = asignarea pe fiecare din cele 8 coloane a unei regine pe o linie
- funcția succesor (vecinătatea unei stări) = toate stările în care putem ajunge mutând o regină pe o coloană
- sunt 56 de succesori pentru orice stare
- **funcție cost h** = numărul de perechi de regine care se atacă
- vrem să **minimizăm această funcție cost**, minimul global al funcției h = 0, corespunde unei soluții (stări finale)



Hill climbing pentru problema celor 8 regine

- **căutare locală**: îmbunătățește o stare actuală până când nu mai poți

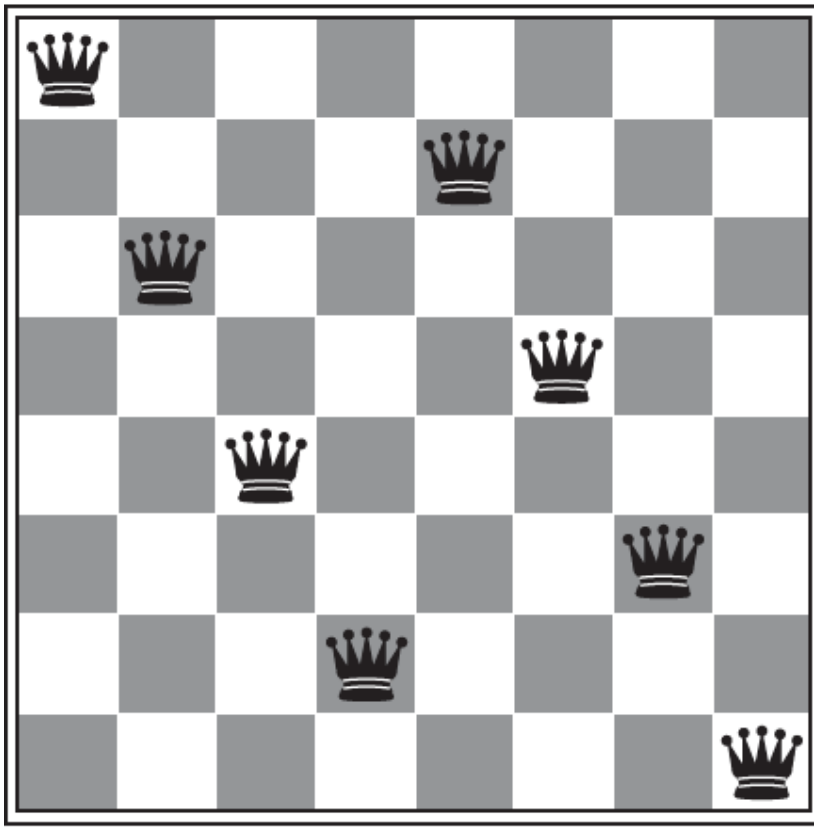
- o stare actuală = asignarea pe fiecare din cele 8 coloane a unei regine pe o linie

- funcția succesor (vecinătatea unei stări) = toate stările în care putem ajunge mutând o regină pe o coloană

- sunt 56 de succesori pentru orice stare

- **funcție obiectiv g** = numărul de perechi de regine care NU se atacă

- vrem să **maximizăm această funcție obiectiv**, maximul global al funcției g = 28, corespunde unei soluții (stări finale)



Hill climbing pentru problema celor 8 regine

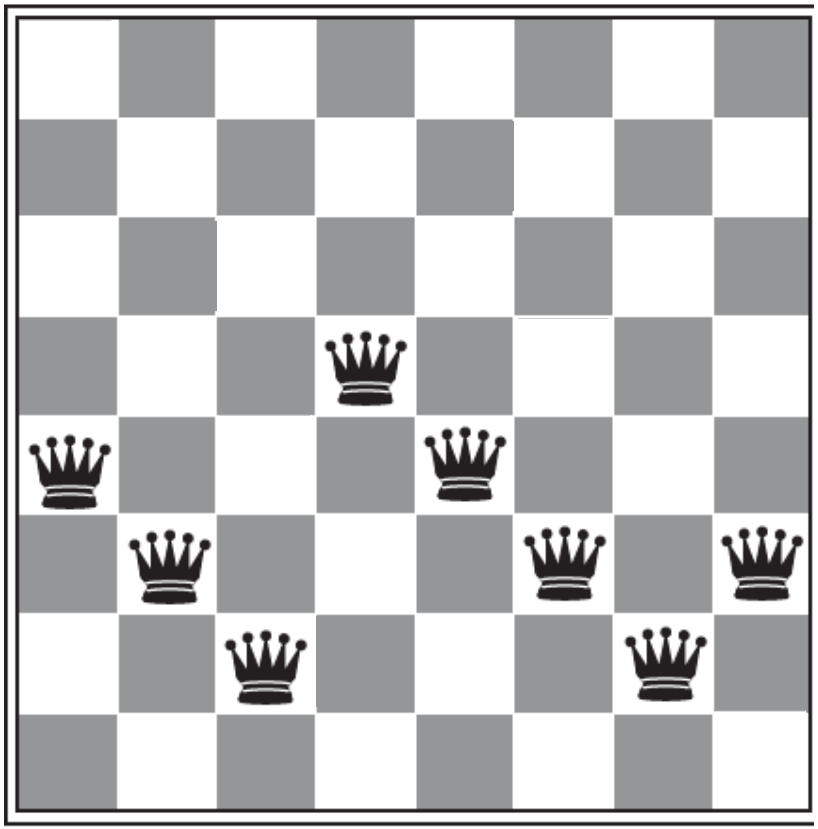
- **funcție cost h** = numărul de perechi de regine care se atacă

- vrem să **minimizăm această funcție cost**, minimul global al funcției h = 0, corespunde unei soluții (stări finale)

- cât este valoarea lui h în starea actuală?

- h = #perechi regine pe aceeași coloană + #perechi regine pe aceeași linie + #perechi regine pe aceeași diagonală

- h = 0 + 5 + 12 = 17



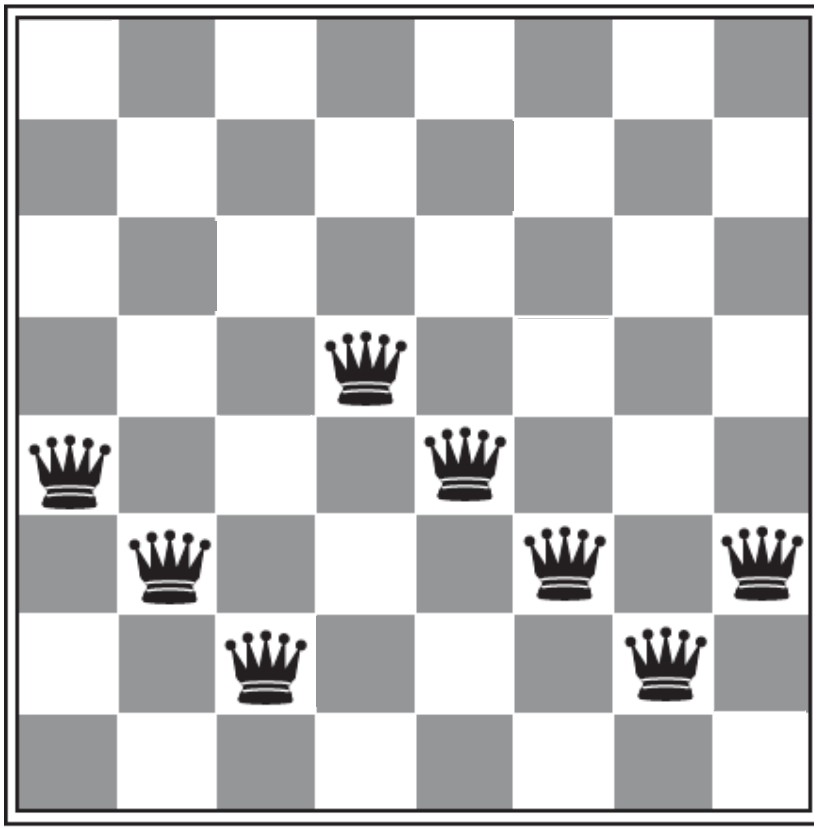
Hill climbing pentru problema celor 8 regine

- **funcție cost h** = numărul de perechi de regine care se atacă

- vrem să **minimizăm această funcție cost**, minimul global al funcției h = 0, corespunde unei soluții (stări finale)

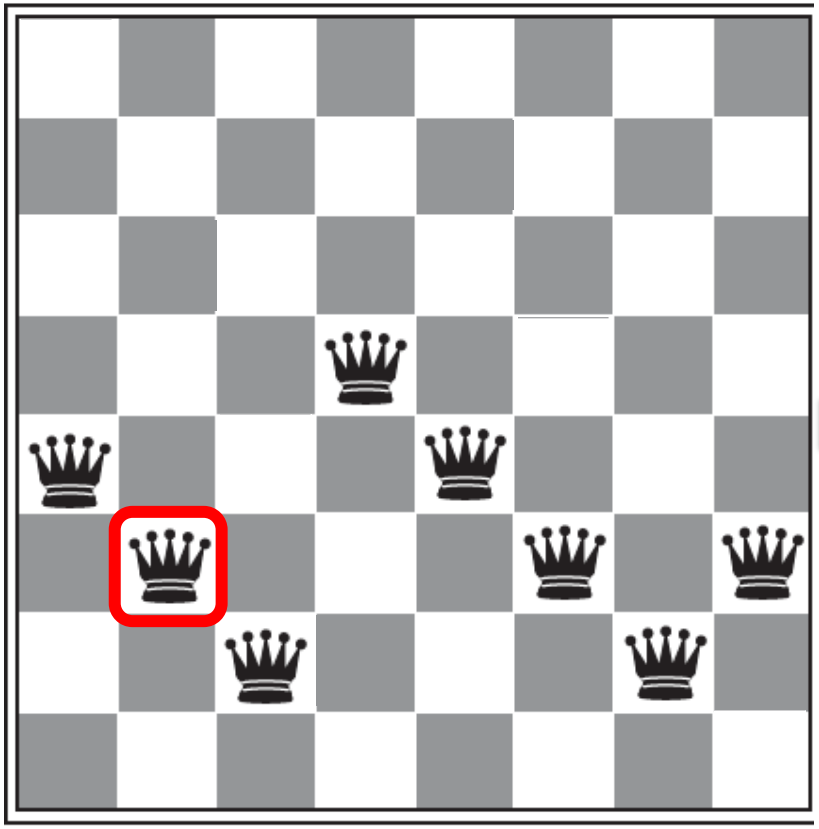
- sunt 56 de succesori pentru stare actuală

- calculăm funcția h pentru fiecare succesor

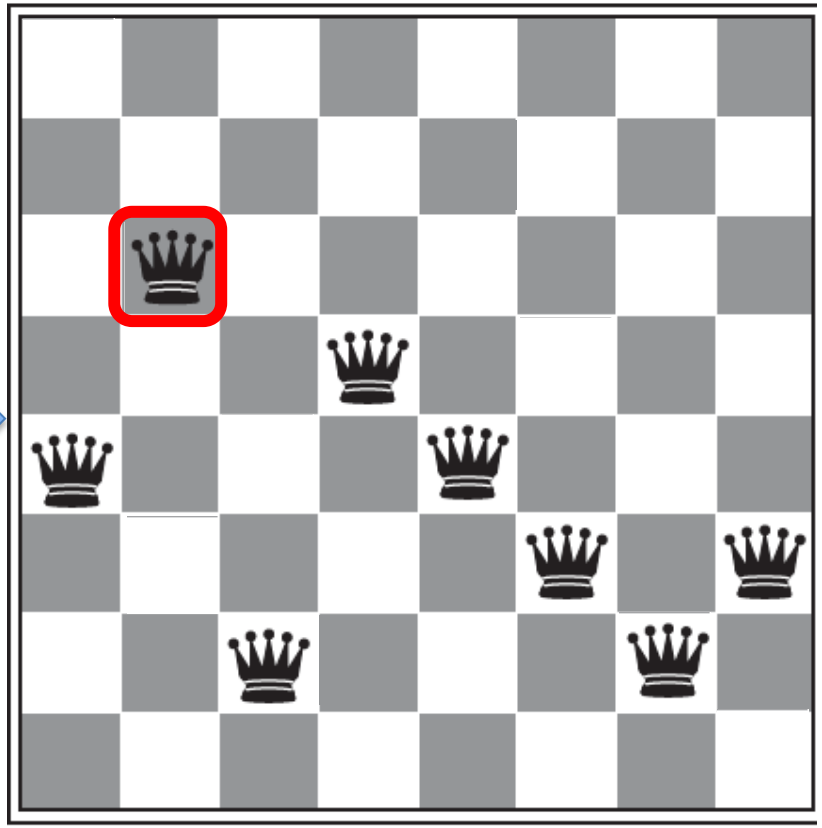


Hill climbing pentru problema celor 8 regine

Stare actuală



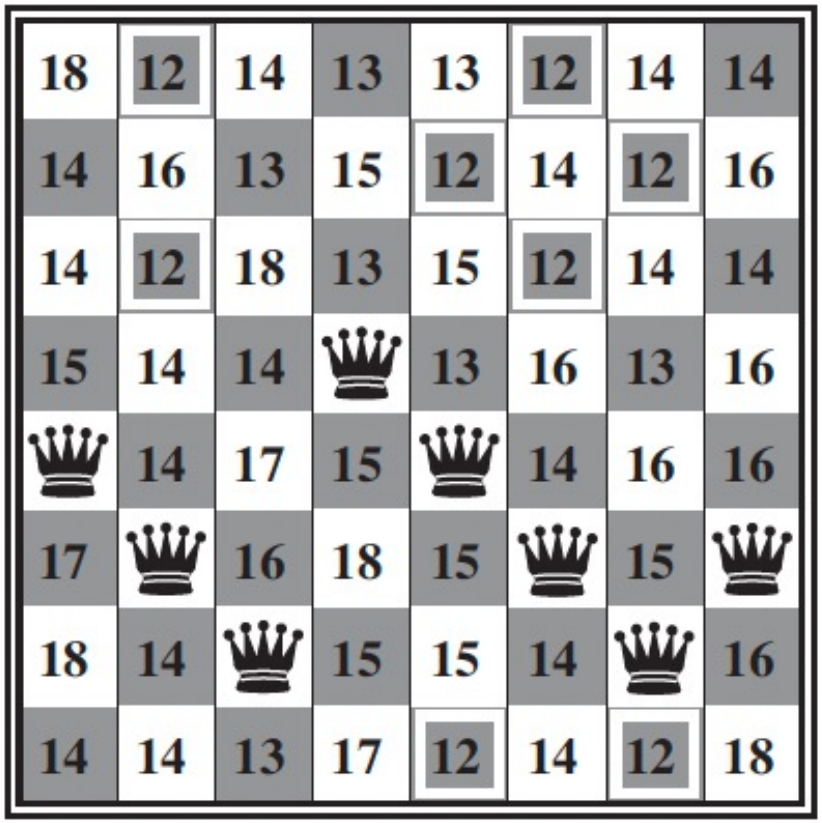
Posibil succesor



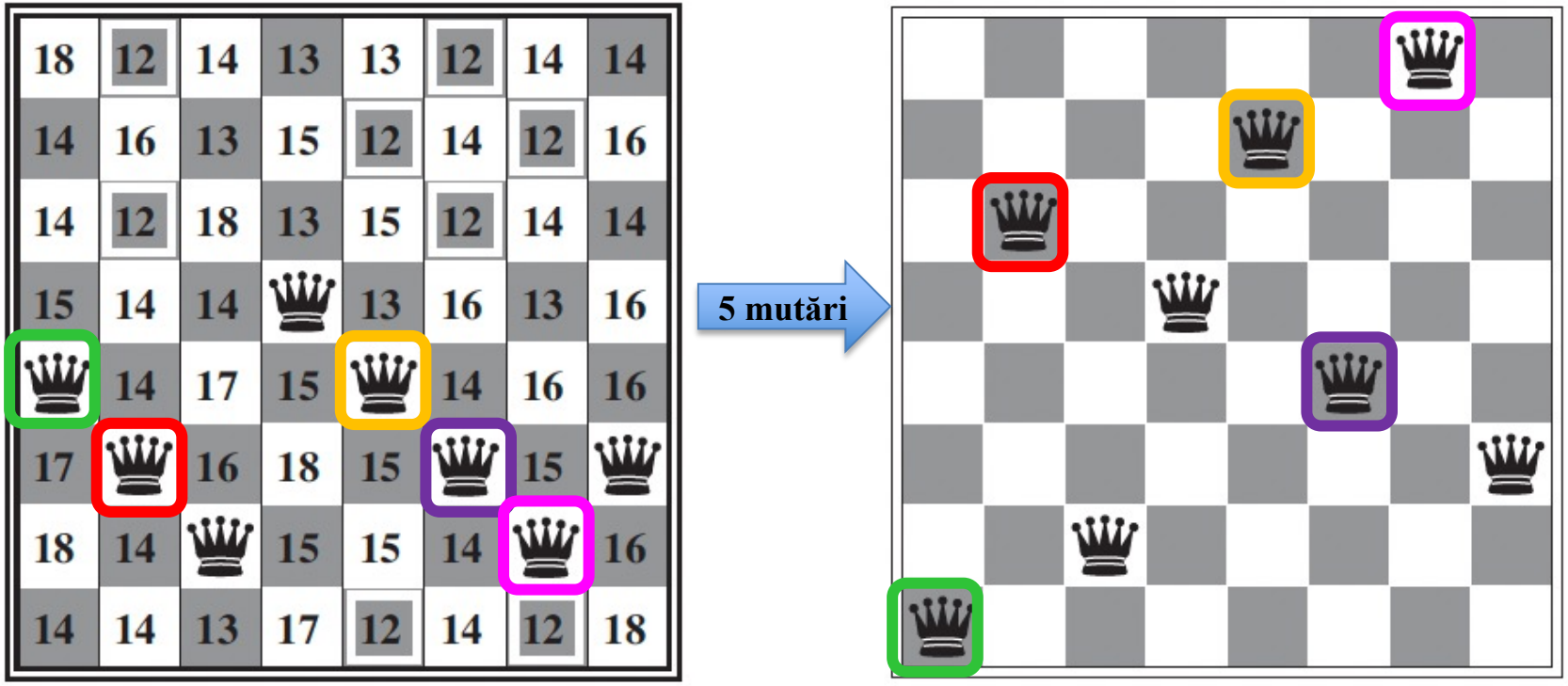
- h = #perechi regine pe aceeași coloană + #perechi regine pe aceeași linie + #perechi regine pe aceeași diagonală = 0 + 3 + 9 = 12

Hill climbing pentru problema celor 8 regine

- sunt 56 de succesori pentru stare actuală
- calculăm funcția h pentru fiecare succesor
- există 8 succesori cu h = 12
- hill climbing alege aleator între cei 8 succesori
- hill climbing = căutare locală greedy = alege un succesor optim local fără să aibă o privire de ansamblu globală

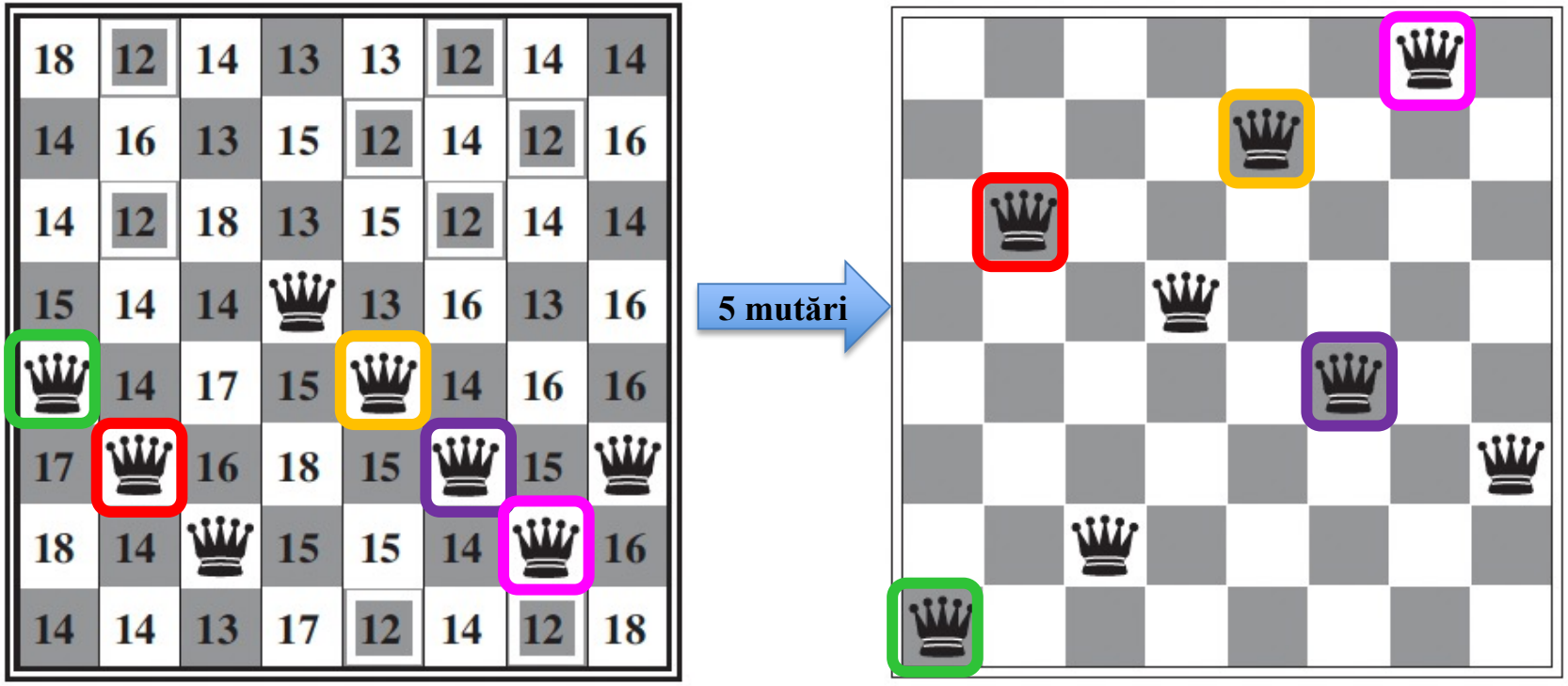


Hill climbing pentru problema celor 8 regine



- hill climbing = căutare locală greedy = alege un succesor optim local fără să aibă o privire de ansamblu globală
- realizează un progres foarte rapid către o posibilă soluție (h = 1) întrucât e foarte ușor să îmbunătățim starea curentă (foarte proastă, h = 17)
- starea la care se ajunge (h = 1) este stare de minim local, nu este minim global. Nu mai poate fi îmbunătățită, dar nu am ajuns la o soluție (h = 0)

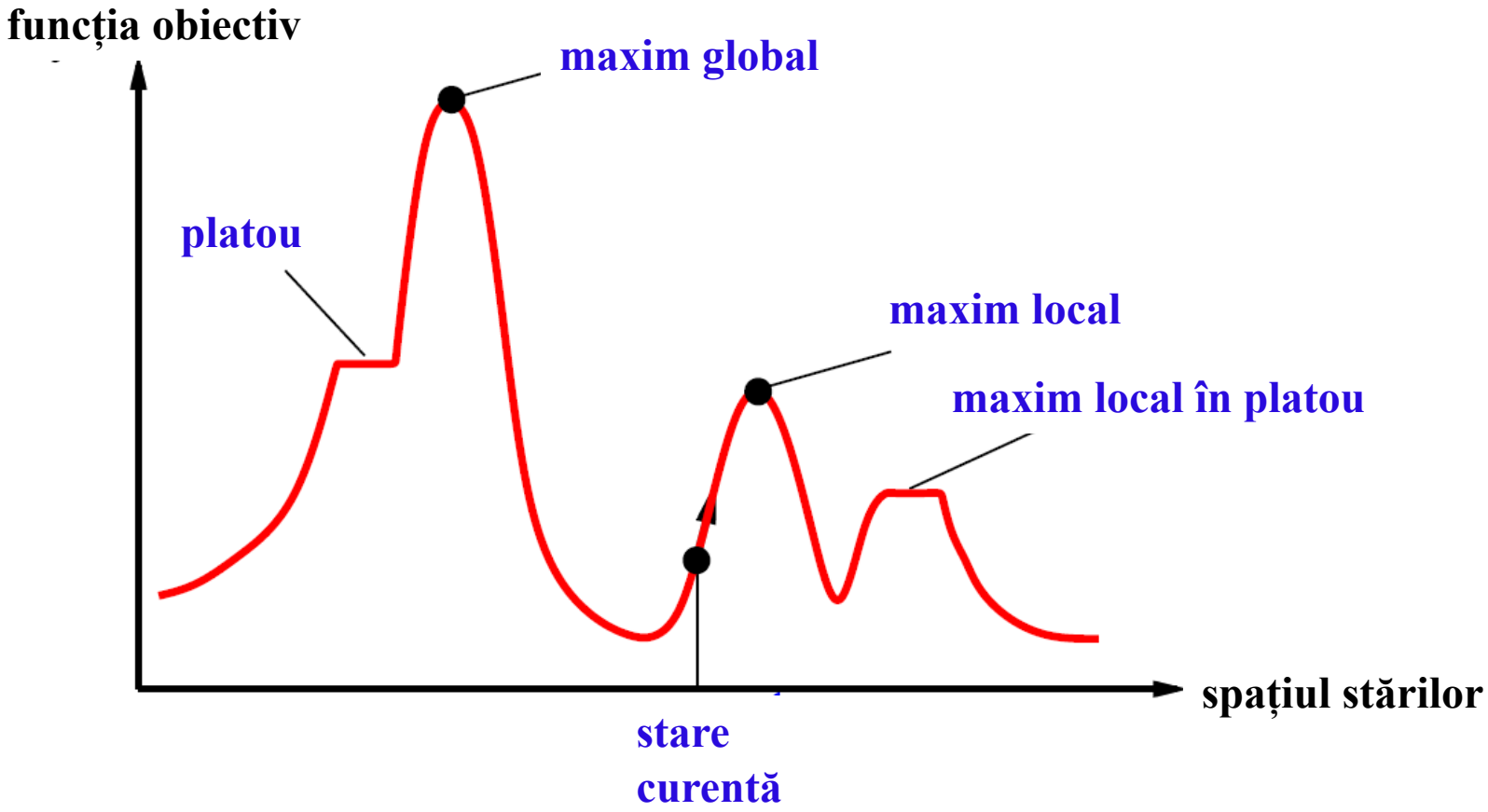
Hill climbing pentru problema celor 8 regine



- pe problema celor 8 regine, pornind de la o stare aleatoare, hill climbing găsește o soluție globală (h = 0) în 14% din cazuri în medie în 4 mutări. Când se blochează într-un minim local o face în 86% din cazuri în medie în 3 mutări.

- spațiul stărilor are $8^8 \approx 17{,}000{,}000$ configurații posibile

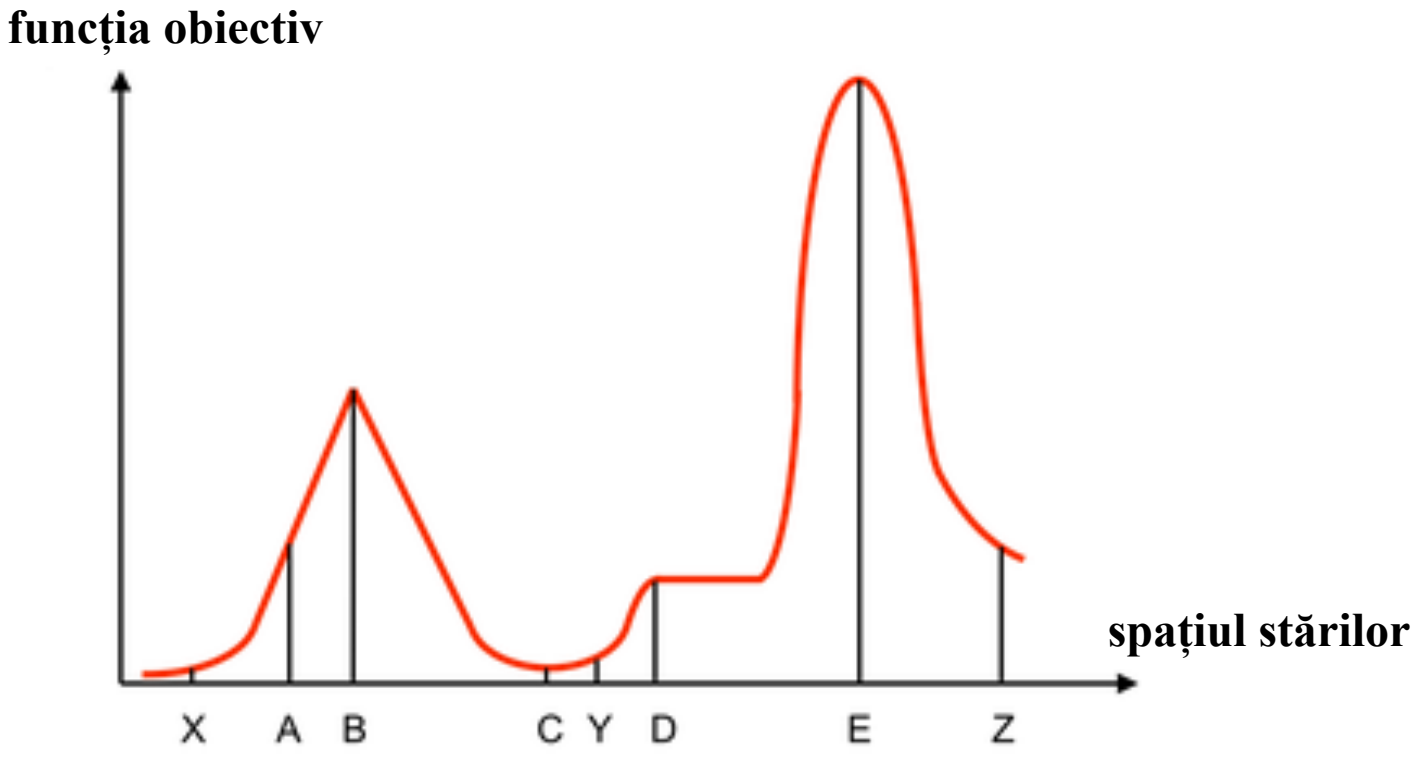
Diagramă pentru Hill climbing



Problema de optim local: maxim global vs maxim local

- inițializări multiple în paralel + hill climbing + ia maximul după toate rulările
- evadează din punctele de maxim local cu anumite metode de a explora spațiul – ***algoritmi genetici***, simulated annealing, etc.

Instanțieri pentru Hill climbing



Dacă starea curentă este X, unde ajungem ?

Dacă starea curentă este Y, unde ajungem ?

Dacă starea curentă este Z, unde ajungem ?

Calcul evolutiv – motivație și idee generală

În multe probleme de optimizare:

- spațiul soluțiilor este foarte mare;
- gradientul nu este disponibil sau nu poate fi calculat;
- funcția cost/obiectiv prezintă multe minime/maxime locale;
- metodele clasice (hill climbing, gradient descent) se blochează ușor.

Avem nevoie de o metodă care să exploreze mai multe regiuni ale spațiului soluțiilor, în paralel.

Ideea centrală a **calculului evolutiv**

- inspirat de evoluția naturală;
- menținem o **populație de soluții candidat**;
- evaluăm fiecare soluție printr-o **funcție de fitness**;
- selectăm soluțiile cele mai bune;
- generăm soluții noi prin variație (**încrucișare și mutație**);
- repetăm procesul până când calitatea soluțiilor devine satisfăcătoare.

Calculul evolutiv echilibrează:

- **explorarea** (caută zone noi din spațiul soluțiilor);
- **exploatarea** (rafinează soluțiile bune existente).

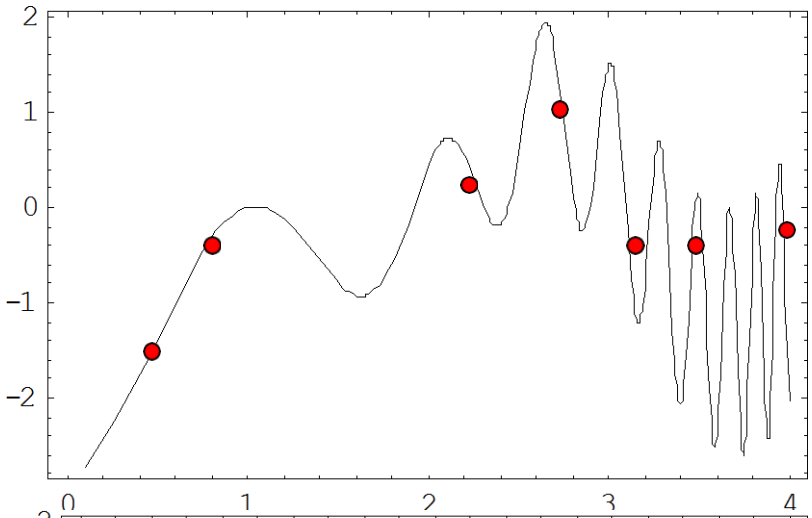
Acest „duet” permite rezolvarea unor probleme unde algoritmii determinist-greedy eșuează.

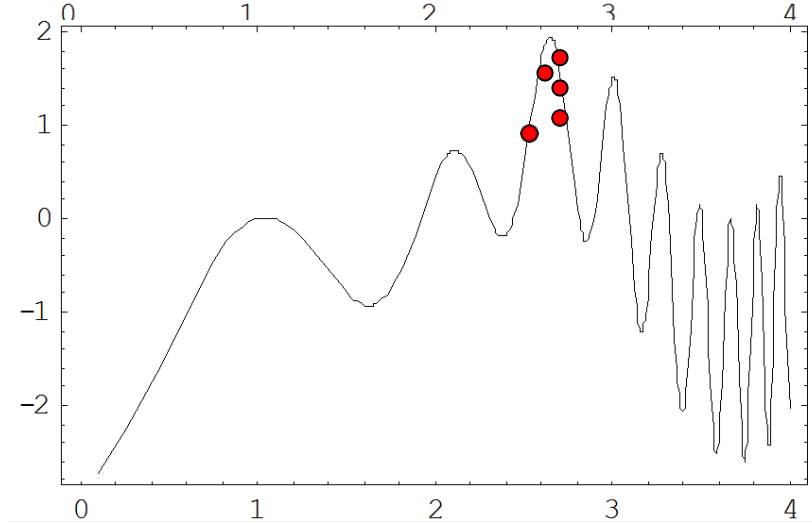
Specificul calculului evolutiv

Procesul de căutare se bazează pe două mecanisme principale:

explorare = parcurgerea diferitelor regiuni din spațiul soluțiilor și colectarea de informații

exploatare = rafinarea soluției (exploatarea informațiilor colectate în procesul de explorare)





Trecerea naturală către Algoritmi Genetici

În cadrul calculului evolutiv există mai multe familii de algoritmi inspirați din procese naturale:

• ES – Evolution Strategies/Strategii evolutive - metode evolutive pentru optimizare continuă, bazate în principal pe mutație gaussiană și adaptarea distribuțiilor.

• GA – Genetic Algorithms/**Algoritmi Genetici** - cea mai utilizată abordare evolutivă. folosește populații, selecție, încrucișare și mutație pentru a evolua soluții.

• GP – Genetic Programming/Programare genetică - evoluează programe sau expresii (sub formă de arbori), este utilizat pentru descoperirea de funcții, formule sau algoritmi.

• PSO – Particle Swarm Optimization/Optimizare prin roi de particule - algoritm inspirat de dinamica roiurilor, soluțiile („particulele”) se deplasează ghidate de propria experiență și de cea globală.

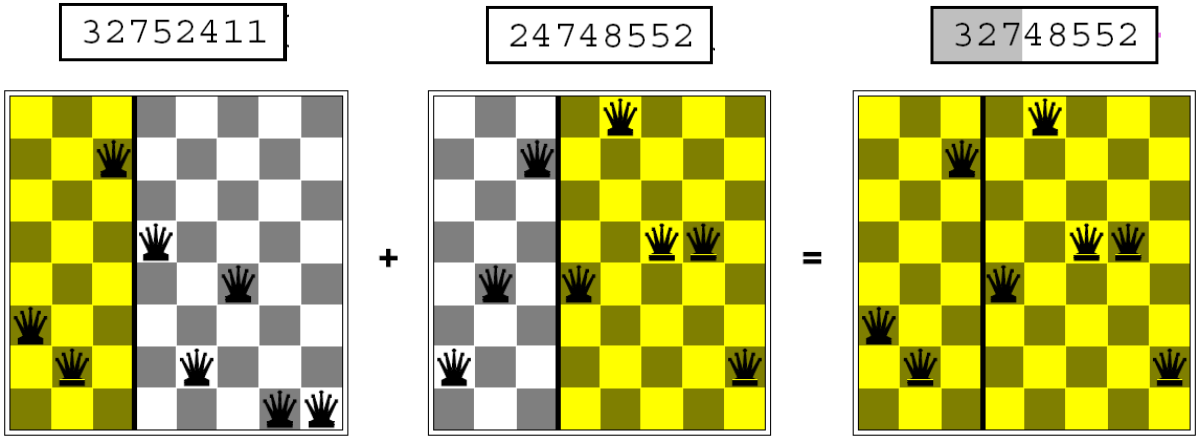
De ce ne concentrăm pe **Algoritmii Genetici (AG)?**

- sunt simpli, generali și ușor de implementat;
- pot lucra cu multe tipuri de codificări;
- nu necesită gradient;
- funcționează bine în optimizare, control, jocuri și învățare automata.

Structura unui algoritm genetic

Inițializare populație (aleatoare)
Evaluare populație (calcul fitness pt fiecare element)
REPETĂ
    Selecție părinți
    Generarea urmașilor prin încrucișare
    Modificarea urmașilor prin mutație
    Evaluare urmași
    Selecție supraviețuitori (vor forma populația din noua generație)
PÂNĂ CÂND <condiție de oprire>

Algoritmi genetici – exemplu



Populație = mulțime de indivizi (ipoteze, stări, asignări, configurări)

Funcție de evaluare (fitness) = evaluează calitatea unei ipoteze

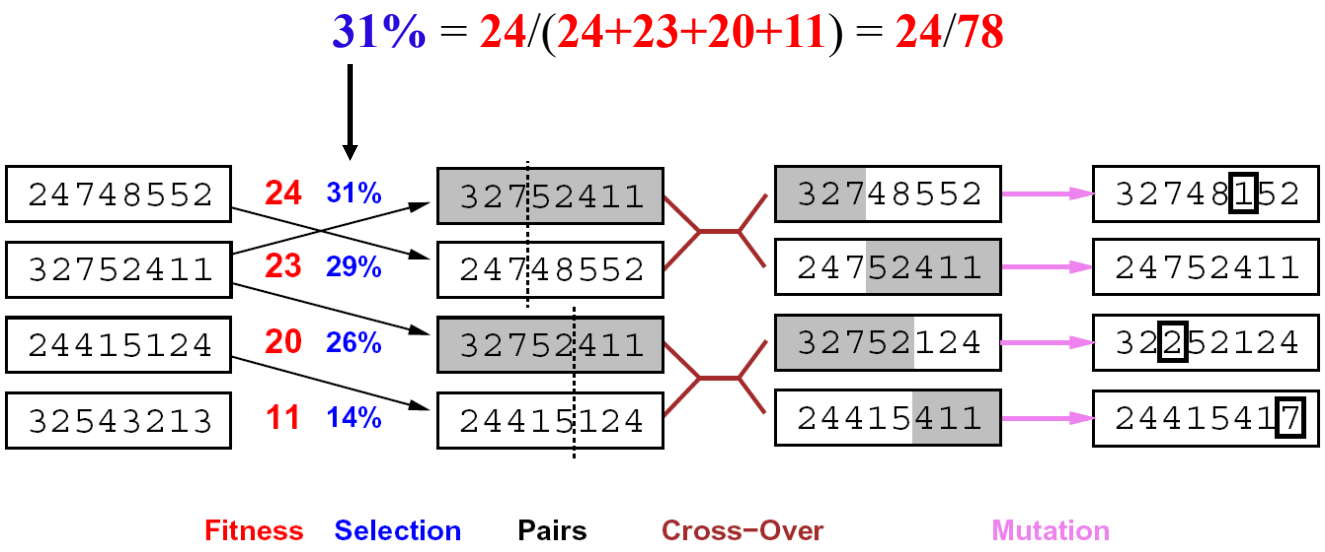
-exemplu în problema de N-regine: numărul de perechi de regine care nu se atacă

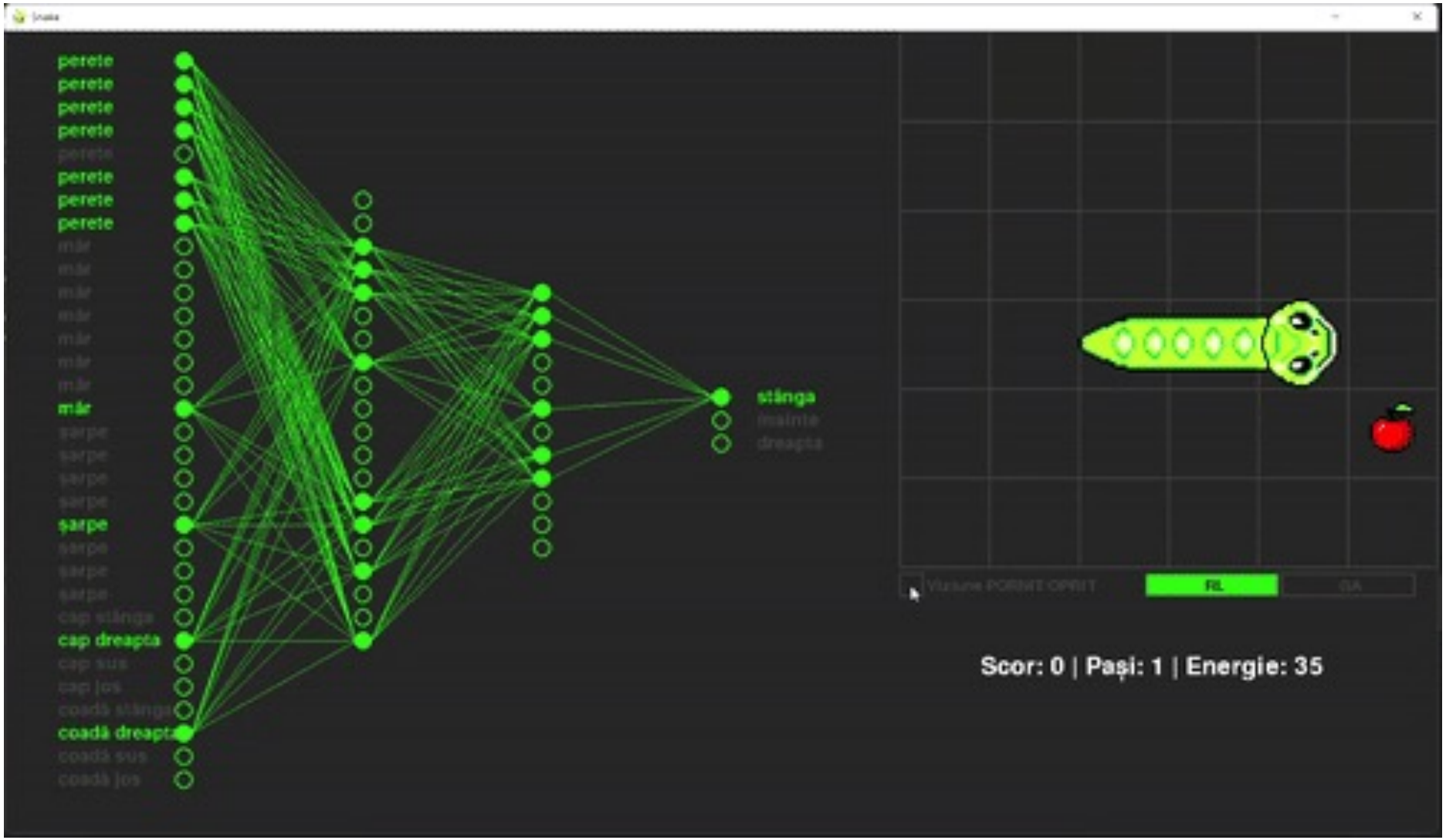
f (32752411) = 23 = 28 - 5

f (24748552) = 24 = 28 - 4

Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție după o anumită distribuție de probabilitate și împerechere bazată pe operațiile de încrucișare (cross-over) și mutație.

Algoritmi genetici – exemplu



- Obține o nouă populație combinând indivizi folosind un proces care se bazează pe o selecție (selection) după o anumită distribuție de probabilitate bazată pe funcția de evaluare (fitness) si împerechere (**pairs**) bazată pe operațiile de **încrucișare (cross-over)** și mutație (mutation)

Algoritmi genetici – exemplu Snake